

Preprocessing for Life: Dishonest-Majority MPC with a Trusted or Untrusted Dealer

Elette Boyle
Reichman University and
NTT Research
eboyle@alum.mit.edu

Niv Gilboa
Ben-Gurion University
gilboan@bgu.ac.il

Matan Hamilis
Reichman University
matan@hamil.is

Yuval Ishai
Technion
yuvali@cs.technion.ac.il

Ariel Nof
Bar-Ilan University
ariel.nof@biu.ac.il

May 3, 2025

Abstract

We put forth a new paradigm for practical secure multiparty computation (MPC) in the preprocessing model, where a feasible one-time setup can enable a lifetime of efficient online secure computations.

Our protocols match the security guarantees and low costs of the cheapest category of MPC solutions, namely 3-party protocols (3PC) secure against a single malicious party, with the qualitative advantages that one party communicates data sublinear in the circuit size, and can go offline after its initial messages. This “2+1”-party structure can alternatively be instantiated between 2 parties with the aid of a (possibly untrusted) dealer. Within such existing protocols, we provide comparable online performance while improving the storage and offline dealer-to-party communication requirements by more than 3 orders of magnitude.

At the technical level, we build on a novel combination of the Fully Linear Interactive Oracle Proof (FLIOP)-based protocol design of Boyle et al. (CRYPTO 2021) and pseudorandom correlation generators. We provide an extensive assortment of algorithmic and implementation-level optimizations, design efficient distributed proofs of well-formedness of complex FLIOP correlations, and make them circuit-independent. We implement and benchmark our end-to-end system against the state of the art in the 2+1 regime, a dealer-aided variant of SPDZ for Boolean circuits.

We additionally extend our techniques to the $(n + 1)$ party setting, where a dealer aids general dishonest-majority MPC, and provide a variant of the protocol which further achieves security with identifiable abort.

1 Introduction

Can we design everlasting batteries? This work discusses the analogue of this question in the setting of *secure multiparty computation* (MPC) [1, 2]. MPC is a general-purpose tool for computing across secret inputs while revealing only the computation output. MPC can be used for mitigating single

points of failures in a variety of applications; see, e.g., [3] for current real-world use cases. Here we focus on the case of computing Boolean circuits¹ with security against a *malicious* adversary.

Perhaps the most commonly studied setting of MPC is secure *2-party* computation (2PC). The last decades have witnessed tremendous advances in the concrete efficiency of 2PC protocols. Unfortunately, the costs of existing solutions is still quite high for large-scale computations, typically due to their demanding communication requirements. Alternative approaches offer online protocols with low communication by means of useful correlated randomness produced in an offline preprocessing phase; however, the large size of these correlations forms a bottleneck to scaling, even just for this data to be communicated.

1.1 Our Contributions

We put forth a new paradigm for MPC in the preprocessing model, dubbed “*preprocessing for life*,” where a feasible one-time setup can support a lifetime of efficient online 2PC (or MPC) executions. For large-scale applications, one can imagine possession of cryptographic correlations as a form of “battery charge,” enabling the parties to continue performing secure computations until the correlations are consumed. We correspondingly seek MPC batteries with lifetime charge.

Jumping ahead, in the simpler *semi-honest* security model (where corrupted parties are still assumed to follow the protocol), this goal can be achieved via the use of pseudorandom correlation generators (PCGs) [4]. However, known PCGs that support security against malicious parties [5, 6] are slower by orders of magnitude, especially for Boolean circuits or in the multi-party setting.

Our starting point is a theory-oriented line of work [7, 8, 9] that achieves malicious security via sublinear zero-knowledge proofs based on “Fully Linear Interactive Oracle Proofs” (FLIOPs). The FLIOP-based approach can yield MPC protocol in the aforementioned preprocessing model, where a trusted dealer is used to generate and distribute correlated randomness [8]. The advantage of the FLIOP-based approach is that the amount of extra communication by the dealer required for achieving malicious security grows *sublinearly* with the size of the circuit being computed. This should be contrasted with the traditional “SPDZ-style” approach [10, 11] in which the extra communication is roughly 3σ bits per AND gate, for a statistical security parameter σ .

Unfortunately, the FLIOP-based approach to dealer-aided MPC from [8] also has several downsides. First, in exchange for improved communication, it incurs significantly greater computation costs. In spite of efforts to minimize the computational overhead of FLIOPs [12], applying the best known constructions within the framework of [8] still incurs a significant slowdown. A second limitation is that the FLIOP-based correlations distributed by the dealer have to depend on the circuits computed in the online phase, severely limiting the possible use cases. Finally, the security of the protocol strongly relies on the honest behavior of the dealer. A malicious (or even “selfish”) dealer could lead to incorrect results without detection. Prior approaches for mitigating this trust by distributing the role of the dealer [8, 9] incur high concrete costs, defeating the goal of the FLIOP-based approach for efficient large-scale secure computation.

In this work, we initiate a systematic effort to eliminate the above disadvantage of the FLIOP-based approach, making it a viable basis for secure 2PC with preprocessing for life. This encompasses significant optimizations and modifications to the protocol of [8], including circuit-independent correlations, new procedures for efficiently verifying a possibly corrupt dealer, and

¹While our efficiency optimizations focus on Boolean circuits, results of this paper can be generalized to richer circuit models, such as arithmetic circuits and beyond, using the same high level blueprint.

end-to-end system implementation and benchmarks. We further provide extensions to the n -party setting, where we boost security to identify a cheating party in case the online protocol aborts. We next proceed to describe our contributions in more detail.

Protocol design & optimization We present an optimized protocol that realizes our main goal in the “2+1”-party setting, where a (trusted or untrusted) dealer provides the 2 online parties with the corresponding (*short*) correlations. Alternatively, this can be viewed as a 3PC protocol in the honest-majority setting, with the qualitative advantages that one party is non-interactive, only communicates data sublinear in the online computation size, and can go offline after its initial messages.

Our resulting protocol and implementation comprise the first practical secure computation solution whose online phase nearly matches current state-of-the-art in the preprocessing model (based on the SPDZ line of work [11, 13, 14]; with just 2 bits of communication per party per AND gate in the 2+1 setting), with correlation size that grows *sublinearly*—in fact, logarithmically—in the circuit size.

On the technical level, we build on the FLIOP-based protocol of [8], providing several kinds of improvements:

- *Circuit-independent correlations.* In [8], the correlation generated by the dealer is circuit dependent, meaning that one must commit in advance to the topology of all future circuits to be securely computed, and must further reveal this information to the dealer party. We modify the correlation and protocol to remove this limitation.
- *Optimizations.* We incorporate several algorithmic and implementation-level optimizations to the baseline protocol from [8]. For example, we identify a special structure in how FLIOPs are used that enables us to cut the computational cost of the verification phase of the protocol by a factor of 2. We additionally employ several implementation-level optimizations, including cache-friendly implementations for PCG expansion, higher “folding factors” for the recursive FLIOP structure to improve memory access costs (as in [12]), efficient exploitation of independent arithmetic logic units (ALU) on the CPU, and lazy modular reduction for operations over binary extension fields.² These optimizations are detailed in appendix C.
- *Untrusted dealers.* We present an approach for enhancing the security of dealer-aided protocols against a potentially malicious dealer. In particular, we provide new efficient procedures for parties to verify correctness of dealer-provided correlations, for both FLIOP-based and SPDZ-based protocols. This verification can be accomplished by a local, efficient computation followed by single round of interaction between non-dealer parties.

Our verification procedures themselves rely on FLIOP machinery, which supports sublinear-communication zero-knowledge proofs on distributed data [7]. A naive application of applying FLIOPs to the correlation structure leads to heavy constraints of a high algebraic degree, which translates to expensive verification. Through a combination of modifications to the protocol of [8] and careful representation choices, we present new solutions which enable the dealer’s correlation to be verified with a comparably sized collection of *degree-2* constraints, admitting highly

²We remark that, contrary to the findings of [12], the benchmarks within our computer architecture resulted in faster zk-FLIOP over binary extension fields as opposed to the comparable Mersenne prime field; see discussion in Section 6.

efficient FLIOP-based verification. This degree reduction is done without requiring additional communication by the prover.

Implementation & benchmarks Our protocols offer a flexible tradeoff between correlation compression and computation costs. We provide full benchmarking of one such data point against an optimized state-of-the-art (2+1) variant of SPDZ.

- In the online phase, consuming correlations for secure distributed computation of 1,024 parallel AES-128 encryptions, consisting of 6.5 million Boolean AND gates, is 1.6x slower than the online phase of SPDZ on a LAN network with a single core. The overhead mainly comes from FLIOP-based verification, which is slightly more costly in computation and communication rounds than SPDZ. This gap vanishes with parallelization or when computing circuits of a higher depth.
- In exchange, our correlation size is smaller than SPDZ by *almost 4 orders of magnitude*. While SPDZ can support secure computation of $6.64 \cdot 10^4$ AND gates per MB of correlated randomness (for $\sigma = 40$ bits of statistical security), at this tradeoff point our protocol can support $4.97 \cdot 10^8$ gates/MB. For example, extending to secure computation of 1 million AES-128 circuits, the SPDZ correlation would be 100GB, whereas ours is *just 12.8MB*.

The computation cost for generating our correlation is higher than (the trusted-dealer variant of) SPDZ but parallelizes naïvely, whereas just communicating the large correlation in SPDZ yields a parallelization bottleneck due to network saturation. Even with just 5 cores, the end-to-end offline phase runtime in our protocol is shorter than SPDZ (and continues parallelizing to ~ 2000 cores; see Section 6.2, Figure 3).

Already at this correlation compression rate, where our computation costs are comparable to state-of-the-art alternative solutions, this enables the first practical feasibility of *preprocessing for life*, requiring just 2MB of storage per 10^9 gates. For example, 1TB of storage can support nearly 5×10^{14} gates. This correlation will be composed of several independent correlation units, which can be expanded and consumed piece by piece on demand (e.g., for 6.5 million gates at a time).

Extensions: $(n + 1)$ -PC & identifiable abort Beyond our core contribution, we investigate two additional related extensions: accommodating more than 2 online parties and enabling security with identifiable abort.

While our main implementation is optimized for the (2+1)-party setting, our approach applies more generally to the $(n+1)$ -party setting, where *either* the dealer or any strict subset of the online parties can be corrupted. This requires two main changes: (1) Extending the procedure for verifying an untrusted dealer’s correlations; unlike the 2+1 case, here we cannot bring the verification degree down to 2, but keep it independent of the circuit size. (2) Extending the underlying pseudorandom correlation generator (PCG) to the $(n + 1)$ setting; here we opt for using fast two-party PCGs for OT [4, 15, 16], at the price of the online cost scaling quadratically with n . However, even without attempting to leverage compression provided by PCGs, our FLIOP-based protocol will still provide at least $\times 3\sigma/(n - 1)$ improvement in the correlation size over SPDZ, where we lose the $n - 1$ factor due to our reliance on pairwise OT correlations.

We present a transformation for boosting the above protocols from standard “security with abort” to the stronger security notion of *identifiable abort* [17] (IA) against a malicious adversary

who either corrupts the dealer or any strict subset of online parties. This stronger guarantee can help mitigate DDoS attacks by enabling parties to identify a cheater whenever the protocol aborts. In particular, a cheating dealer will be identified and potentially eliminated by the online parties, without a risk of an honest dealer being framed by a malicious party. We note that IA security is the best kind of security one could hope for in our setting, in light of the impossibility of achieving fairness with an offline dealer (see Section 2.1). Unlike prior works in the area (e.g., [17, 18, 19, 20]), which address the more general n -party corruption setting, we do not need strong tools such as generic zero knowledge, non-committing encryption, or homomorphic signatures.

Summary of contributions To summarize, we provide the following main contributions in our undertaking toward MPC with *preprocessing for life*.

- **Protocol design & optimizations:** We propose the first practical secure $(2+1)$ -party protocol with SPDZ-like online cost and *compressible* correlated randomness.
- **Implementation & benchmarks:** We compare our implementation with an optimized state-of-the-art $(2+1)$ variant of SPDZ for secure computation of 1,024 parallel AES-128 encryptions. For this task, our online phase is roughly $1.6\times$ more expensive than SPDZ; but, in exchange, our correlated randomness is smaller by nearly *4 orders of magnitude*.
- **Extensions:** We extend our protocol to the $(n+1)$ -party setting, and to the stronger notion of security with identifiable abort.

Why is a dealer realistic? We conclude this section with a discussion on the $(n+1)$ -party model and its possible realizations.

First, we observe that the dealer can only be corrupted at a singular point in time: during the offline phase when generating the correlations. As such, we can view the offline phase as a semi-trusted setup. It is a “setup” in the sense that the outputs of this phase may serve the parties throughout the whole lifetime and it is “semi-trusted” in the sense that the only assumptions parties should make is that no collusion between the dealer and any of the parties may occur. This can be realistically implemented by an open-source external cryptographic service provider, special-purpose hardware, a personal computer started in “safe mode,” or other means as proposed in [21, 22, 23, 24]. We stress again that after this singular point time of generating the correlations any corruption of the dealer alone is essentially meaningless and has no impact on the security of our model.

Second, the practicality of dealer-aided architectures has been embraced and utilized within many recent secure systems, in application regimes such as secure computations on weak mobile phones [25, 26], medical and genomic data processing [21, 22, 27], and broader forms of privacy-preserving AI [28, 29, 30, 31, 32]. Among the contributions of this paper is (efficiently) removing the need for trust in the dealer entity itself.

Finally, the simple communication structure of the $(n+1)$ model also minimizes its attack surface, making feasibility of a non-colluding dealer more realistic. Namely, the dealer can perform its computations disconnected from the network, joining only to send out its computed values, then disconnecting for the remainder of the protocol. In fact, given the potential of our $2+1$ protocol to enable a *lifetime supply* of correlated randomness in a single message (for arbitrary future secure computations with a given other party), then for many applications the dealer can even be the

hardware maker, with the correlation hardcoded into system firmware. In fact, one could argue that in any realistic execution environment for MPC protocols, the process of manufacturing the hardware or installing the operating system effectively employs an external, semi-trusted “dealer.”

1.2 Related Work

A central bottleneck in existing 2PC solutions is their large communication requirement. For example, protocols using garbled circuits require roughly security parameter (at least 128) bits communicated per Boolean multiplication gate for weak semi-honest security; for the malicious case, the situation is even worse.

2PC with uncompressed correlations. Alternative approaches—most notably, the successful “SPDZ” line of work [11, 13, 14] and followups—offer online protocols with low communication by means of useful correlated randomness. However, in most existing protocols the required correlated randomness scales *linearly* with the size of the circuit that should be evaluated in the online phase. This size is further multiplied by a statistical security parameter in the case of Boolean circuits. The goal of eliminating the latter overhead was most recently studied by the work of Karmakar et al. [33]; they also achieve fairness, but at the (inherent) cost of requiring the dealer party to remain active and communicate in the online phase, conflicting with our goal of lifetime preprocessing.

As an illustrative example, using standard SPDZ-based protocols to support secure computation of one million AES-128 symmetric-key encryption circuit executions (e.g., for securely searching within the same number of AES-encrypted items) with 40 bits of statistical security (i.e., 2^{-40} failure probability), the required communication from the dealer just to send the authenticated triples would be roughly *100 GB*. While some progress has been made on the size of the correlated randomness within the SPDZ framework (e.g., [34, 35]), this came at the price of a significant increase in the online costs.

2PC with compressed correlations. To the best of our knowledge, there are only two relevant protocols that offer malicious security and support correlated randomness that can compress sublinearly in the circuit size with practical decompression.

- The TinyOT protocol of Burra et al. [36, 37, 38] and the MASCOT protocol of Keller et al. [39] achieve malicious security with a good concrete overhead for Boolean and arithmetic circuits, respectively, using only *oblivious transfer* (OT) correlations. Concretely efficient pseudorandom correlation generators (PCG) [4] for OT can be used for compressing the correlated randomness of both protocols. However, both the online communication and the number of OTs consumed by these protocols are significantly larger than the baseline semi-honest OT-based protocol [2] (and SPDZ-type online protocols), roughly by $\times 10$ in the Boolean case.
- As discussed, the recent theory-oriented work of Boyle et al. [8] showed how to adapt the FLIOP-based distributed zero-knowledge proofs of Boneh et al. [7] to obtain a dealer-based protocol that achieves efficient online protocols with short, compressible correlations. However, the dealer in this protocol is fully trusted, the correlations are circuit dependent, and the protocol was not optimized for concrete efficiency.

We remark that for securely computing arithmetic circuits over a large field, the correlated randomness for SPDZ can be compressed in the 2-party case using the PCG construction for

“Oblivious Linear Evaluation” (OLE) correlations from [40]. This was recently extended to general fields [41, 6]. However, the concrete computational overhead compressing SPDZ-style correlations is significantly higher than for OT correlations (using e.g. [4, 15, 16, 42]). Furthermore, there are no concretely efficient PCGs for SPDZ-style correlations with $n > 2$ parties (despite partial progress in [43, 41]).

Honest-majority 3PC & server-assisted 2PC. As a result, many implementations turn to a weaker setting of *honest-majority* secure 3-party computation (3PC), with 3 parties interacting throughout the protocol, only 1 of whom can be corrupt. Related works have also designed systems in a model interpolated between (2+1)-PC and 3PC, where 2 parties have access to a third “helper” party or server who is more active than a one-shot dealer (e.g., [23, 33, 44]). In these settings, protocols exist with significantly lower communication requirements of a few bits per multiplication gate. For example, due to their efficiency, such protocols are commonly used for privacy-preserving AI [45, 46, 47, 48, 12, 49, 50, 51], including secure DNN training and inference, logistic regression and more.

On the downside, these protocols involve a more complex interaction pattern and require more coordination than (2+1)-PC protocols, and additionally have a larger attack surface. In particular, the third (or helper) party must remain present and communicate with the other parties during the online phase of the protocol. Further, the data communicated by this entity to the other parties is *linear* in the circuit size being securely computed. Both disadvantages are contrary to the goal of preprocessing for life.

Alternative approaches to untrusted dealers. An orthogonal question is the effect of maliciously generated correlated randomness on security. Genkin et al. [52] showed that Beaver’s protocol can still maintain security against an untrusted dealer at a relatively low asymptotic cost, but suffers high concrete overhead in the case of Boolean circuits. Muth and Katzenbeisser [23] propose a cut-and-choose based mechanism for detecting malicious behavior of the dealer in their SPDZ-based protocols, but with greater communication compared to our FLIOP-based approach for verification.

One could alternatively use SNARKs to enforce honest dealer behavior; but, disregarding technical issues that may arise by doing so, even the baseline performance of state-of-the-art general-purpose SNARKs (see, e.g., [53]) would lead to roughly 3 orders of magnitude slowdown compared to the FLIOP-based technique considered in this work.

1.3 Structure of the Paper

We begin in Section 2 with necessary preliminaries. In Section 4 we present our method for verifying dealer correlations in the setting of SPDZ, providing an optimized adaption of the SPDZ framework to the $(n + 1)$ -PC model. Section 5 contains our main protocol. In Section 6 we discuss our implementation and benchmarks. We defer further material on concrete instantiations of the underlying zero knowledge FLIOP tool, the formal description and security proof of our protocol, and more detailed discussions on implementation optimizations to the Appendix.

2 Preliminaries

Notation. Let $P_0, P_1, \dots, P_n$ be the parties participating in the protocol. In our setting, P_0 is an external party known as the dealer and thus is denoted many times by $\mathcal{D}$. Let R be a finite ring

which is either a finite field $\mathbb{F}$ or the ring $\mathbb{Z}_{2^k}$ and let $|R|$ be its size. We do not consider other rings in this work. Finally, we let κ and σ denote computational and statistical security parameters, respectively. We commonly use C to denote a circuit, when such circuit is in context. We also use m to denote the number of non-linear gates in the circuit C , which we also refer to as the *size* of the circuit.

We use $\vec{x}$ to denote a vector x where $x[j]$ is the j th component of x . The notations $\langle \vec{a}, \vec{b} \rangle$ and $\vec{a} \cdot \vec{b}$ are used to denote the inner-product between $\vec{a}$ and $\vec{b}$. We use $[n]$ to denote the set $\{1, \dots, n\}$. We denote by $a \leftarrow \$ A$ a uniform sampling of a value a from a finite set A . For a tuple of n values $(v_1, \dots, v_n)$ and a subset $T \subseteq [n]$ we denote by n_T the tuple $(v_{i_1}, \dots, v_{i_{|T|}})$ for $T = \{i_1, \dots, i_{|T|}\}$ and $i_1 < i_2 < \dots < i_{|T|}$.

Network model. We assume secure point-to-point channels within a synchronous network. Unless otherwise stated, we do not assume a public-key infrastructure or broadcast channel. Broadcast and PKI will be assumed for some of our results achieving a stronger security notion of identifiable abort. To implement a broadcast channel, one can use any known result, e.g., Dolev-Strong [54], but it is possible to achieve higher efficiency via signatures; see discussion in Section 5.5.

$\mathcal{F}_{\text{RO}}$ - random oracle functionality. We define $\mathcal{F}_{\text{RO}}$ to be a global random oracle that models a random function which can be queried by the parties.

Commitments. In some of our protocols, we make use of commitment schemes, which are computationally binding and hiding. The commitment algorithm is denoted by Com . To commit to a value x , the sender chooses a random r and sends $c = \text{Com}(x, r)$ to the receiver. To open the commitment, the sender sends r and x . Then, the receiver can check that c is indeed a commitment to x with randomness r . In our implementation, we realize this commitment using a hash function. Specifically, $\text{Com} = H(x||r)$, where H is a hash function, modelled as a random oracle. The computational binding property is achieved by sampling r randomly from a sufficient large space $\{0, 1\}^{\text{poly}(\kappa)}$.

Secret sharing. We use the following notations:

- **Additive Sharing:** We use $\llbracket x \rrbracket$ to denote an additive sharing of $x \in G$ for some abelian group G between $P_1, \dots, P_n$, i.e., party P_i holds $x_i \in G$ such that $x = \sum_{i=1}^n x_i$. We also use the notation $\llbracket x \rrbracket_i$ for the share held by P_i .
- **Authenticated Sharing:** We use $\langle\langle x \rangle\rangle$ to denote an authenticated additive sharing of x across the parties. We also denote by $\langle\langle x \rangle\rangle_i$ to denote the share of party P_i . The authentication property should allow reconstruction of the secret even in the presence of malicious parties. The number of times we use this scheme is sublinear in the circuit size, so any implementation should suffice. Concretely, for cases where abort is allowed, we use the following simple scheme: P_i holds r_i, x_i and $(c_1, \dots, c_n)$ under the constraint that $x = \sum_{i=1}^n x_i$ and for each $i \in [n]$, and c_i is a commitment to x_i with randomness r_i . To open a secret, each party P_i sends x_i and r_i to the other parties. Using the commitment it holds, each party can verify the correctness of the received share and abort in case of incorrectness. Note that parties who do not hold authenticated shares but only the commitment can still participate in the authenticated opening of the share-holding parties and ensure their correct opening using the commitments only.

$\text{AuthGame}(\kappa, n, T, x, \mathcal{A})$

```

1:  $m \leftarrow |T|$ 
2: Let  $\langle\langle x \rangle\rangle_1, \dots, \langle\langle x \rangle\rangle_n \leftarrow \text{AuthShare}(1^\kappa, x, n)$ 
3:  $\langle\langle x \rangle\rangle_T = \{\langle\langle x \rangle\rangle_i\}_{i \in T}$ 
4:  $\langle\langle x \rangle\rangle_{\bar{T}} = \{\langle\langle x \rangle\rangle_1, \dots, \langle\langle x \rangle\rangle_n\} \setminus \langle\langle x \rangle\rangle_T$ 
5: Let  $X_{\mathcal{A}}^* \leftarrow \mathcal{A}(1^\kappa, \langle\langle x \rangle\rangle_T)$ 
6: Let  $b \leftarrow \text{Reconstruct}(\langle\langle x \rangle\rangle_{\bar{T}} \cup X_{\mathcal{A}}^*)$ 
7: if  $b \in \{x, \perp\}$  then
8:   | return 1
9: else
10: | return 0

```

Figure 1: The authentication game AuthGame

We provide a formal definition for authenticated secret sharing below.

Definition 1 (Authenticated Secret Sharing). *Let κ be a computational security parameter. An authenticated secret sharing scheme over a finite set S is a pair of algorithms (Share, Reconstruct) defined as follows:*

- $\text{AuthShare}(1^\kappa, x, n)$ for a value $x \in S$ this algorithm outputs n shares $\langle\langle x \rangle\rangle_1, \dots, \langle\langle x \rangle\rangle_n$.
- $\text{AuthReconstruct}(\{v_1, \dots, v_n\})$ given a set of n shares $v_1, \dots, v_n$ the algorithm reconstructs a previously shared value or $\perp$ if reconstruction fails.

The scheme should satisfy the following constraints:

- **Correctness:** $\text{AuthReconstruct}(\text{AuthShare}(x)) = x$ for all $x \in S$ with probability 1.
- **Privacy:** Given any two distinct values $x_1, x_2 \in S$ and any $n \geq 2$ and any strict subset $T \subsetneq [n]$ the distribution $\{\text{AuthShare}(1^\kappa, x_1, n)_T\}$ (i.e. only the shares matching indices in T) and the distribution $\{\text{AuthShare}(1^\kappa, x_2, n)_T\}$ are computationally indistinguishable.
- **Authenticity:** For any $x \in S$, any n any strict subset $T \subsetneq [n]$ and any poly-time adversary $\mathcal{A}$ the probability that $\text{AuthGame}(\kappa, n, T, x, \mathcal{A})$ outputs 0 is negligible in κ .

The security game AuthGame : As part of the definition we consider a security game AuthGame taking a security parameter κ , a number of parties n , a strict subset of parties' indices T , a secret x from a set of secrets S and an adversary $\mathcal{A}$. The security game essentially samples an n -party authenticated secret sharing and hands the adversary a set of shares corresponding to the shares indices in the set T . The goal of the adversary is to output some (altered) set of authenticated shares that should result in the successful reconstruct of a secret different from x , in which case the game outputs 0 and we say that the adversary has won the game. Otherwise (i.e. if the same secret x is reconstructed or if simply the reconstruction fails), that game outputs 1 and we say that the adversary has lost the game.

Remark 2 (Statistically Secure Authenticated Secret Sharing). *The definition above takes a computational security parameter. Instead, one can opt for a security definition with statistical security guarantees (i.e. secure against unbounded adversaries). In that case the definition, the security game, the adversary and the algorithm `Share` take a statistical security parameter σ . As a result, the (statistical) privacy property requires that $\{\text{Share}(x_1, n)_T\}$ and $\{\text{Share}(x_2, n)_T\}$ are statistically indistinguishable. Moreover, the adversary in the authenticity property is now computationally unbounded and its probability at winning the security game is in $\text{negl}(\sigma)$.*

Generating random coins. Let $\mathcal{F}_{\text{coin}}$ be an ideal functionality that hands random coins to the parties. Note that when the parties need many coins, they can call $\mathcal{F}_{\text{coin}}$ for a short random key, from which they can locally derive an effectively unlimited number of pseudorandom coins using a pseudorandom function. Hence, any secure realization of $\mathcal{F}_{\text{coin}}$ will suffice.

2.1 The $n+1$ Model

We begin by providing a formal definition for the $(n+1)$ -model under which our protocol works. We consider a standard security definition of multiparty computation based on the ideal/real model paradigm [55, 56], adapted to the $n+1$ -party communication structure setting. By default we consider security against malicious adversaries with security with (non-unanimous) abort. For simplicity, we consider here the case of evaluating a single circuit C ; the definition can be naturally extended to (sequentially) evaluating many circuits using the same setup.

Definition 3 (MPC in the $(n+1)$ Model). *Let R be a finite ring (the field $\mathbb{F}_2$ by default) and let $\mathcal{F}$ be the class of n -party functionalities represented by arithmetic circuits C over R . Let κ be a security parameter. Let $P_1, \dots, P_n$ (the “online parties”) be a set of n parties and let $\mathcal{D}$ (the “dealer”) denote an additional party. A protocol in the $n+1$ model for $\mathcal{F}$ is defined by a pair of protocols $\Pi = (\Pi_{\text{off}}, \Pi_{\text{on}})$ such that:*

- Π_{off} involves $\mathcal{D}$ and $P_1, \dots, P_n$. It is invoked with public inputs $(1^\kappa, m)$, where m is an upper bound on the number of multiplication gates in C . In the protocol the dealer sends a single message to each party P_i and does not receive any messages. The parties $P_1, \dots, P_n$ can then interact. Eventually, each P_i has a local random output v_i and $\mathcal{D}$ has no output.
- Π_{on} involves $P_1, \dots, P_n$. It is invoked on public inputs $(1^\kappa, C)$ and local inputs (v_i, x_i) held by each P_i . The protocol ends with each P_i holding an output y_i .

The protocol Π , obtained by first running Π_{off} and then Π_{on} , is said to be a secure protocol in the $(n+1)$ model if it realizes $\mathcal{F}$ in the presence of malicious adversaries controlling $\mathcal{D}$ or any subset of $\{P_1, \dots, P_n\}$ of size $\leq n-1$.

Definition 4 (Lifetime Preprocessing). *A protocol Π in the $(n+1)$ model is said to provide lifetime preprocessing if the protocol Π_{off} on inputs $(1^\kappa, m)$ outputs correlated randomness of size $\text{poly}(\kappa, \log m)$.*

We further consider a stronger notion of security with *identifiable abort* [17] within the $n+1$ setting. Intuitively, this guarantees that in the case of **abort**, the honest (non-dealer) parties can output a consistent identity of a corrupted party within the protocol execution. We refer the

reader to [17] for a formal definition. Note that as this stronger property no longer tolerates non-unanimous selective abort (as parties must agree on the corrupted party), achieving identifiable abort will inherently make use of a further network assumption, such as PKI or broadcast channel. We stress that any broadcast carried during the online protocol takes as input values that could violate security if obtained by the dealer. Therefore, the dealer, who does not participate in the online protocol, does not participate in particular in any broadcast subprotocols involved in it.

We remark that full security, with guaranteed output delivery, is not generally possible in the $n + 1$ setting. Note that Cleve’s impossibility [57] does not directly apply in this setting, since the dealer can just provide the parties with a common unbiased coin. However, the impossibility of fully secure MPC with a dishonest majority holds even when the parties have access to a trusted source of correlated randomness. While this appears to be folklore, and was noted in passing in [58], we could not find a proof of this fact in the literature. Therefore, a proof is provided in Appendix A.

2.2 Pseudorandom Correlation Generators

The following serves as an high-level and informal introduction to PCGs and their purpose in this work. We refer the reader to Definitions 10–12 from [4] for a formal definition of PCGs.

A *Pseudorandom Correlation Generator* (PCG) is cryptographic tool that allows correlated randomness to be generated in a compressed form of PCG *seeds* such that a share of the correlated randomness is associated with a single seed. Each of these seeds can later by *silently* expanded into a large number of pseudorandom correlations. Therefore, in the context of this paper we can think of a PCG as a pair of algorithms $\text{PCG.Gen}(1^\kappa, m, n)$ and $\text{PCG.Expand}(\text{seed})$. The PCG.Gen takes a computational security parameter κ , the number of correlations to be generated m and the number of parties n and outputs n seeds $\text{seed}_1, \dots, \text{seed}_n$. Later, each seed_i can be expanded into shares of m pseudorandom correlations using the $\text{PCG.Expand}(\text{seed}_i)$ algorithm.

In this work we consider the $(n + 1)$ -model from Definition 3 in which the dealer provides the parties with correlated randomness that facilitates lifetime preprocessing (as in Definition 4). Towards minimizing the size of the correlated randomness and achieving lifetime preprocessing, we use PCGs for multiplication triples where the dealer provides each party with a PCG seed that can be later expanded into shares of m multiplication triples. Slightly more formally, let seed_i be the PCG seed handed to party P_i generated from $\text{PCG.Gen}(1^\kappa, m, n)$. Then, in the case of PCGs for multiplication triples the $\text{PCG.Expand}(\text{seed}_i)$ algorithm outputs m shares of multiplication triples for party P_i : $\left\{ \left(a_j^i, b_j^i, c_j^i \right) \right\}_{j \in [m]}$ such that for all $j \in [m]$:

$$\sum_{i=1}^n a_j^i \cdot \sum_{i=1}^n b_j^i = \sum_{i=1}^n c_j^i$$

2.3 Fully Linear Proof Systems

A main technical building block in our protocols is a *fully linear* proof system [7], which was shown to enable information-theoretic sublinear-communication zero-knowledge proofs on secret-shared input statements [7].

More concretely, our protocols can use any (public-coin) *zero-knowledge fully linear interactive oracle proof* (zk-FLIOP), as defined in Definition 6. In a nutshell, a zk-FLIOP is an information-theoretic proof system in which a prover P wishes to prove some statement over an input x to

a verifier V . In each round of the protocol, P produces a proof which, together with x , can be queried by V using *linear queries* only. Then, a public random challenge is generated, from which the linear queries are computed, and the parties proceed to the next round. At the end, the verifier V accepts or rejects based on the answers to its queries.

Definition 5 (Degree of a Language). *Let R be a ring or a field and let $\mathcal{L} \subseteq R^m$ be a language. We say $\mathcal{L}$ is a degree d language if there exists an m -variate polynomial f of total degree d such that $\vec{x} \in \mathcal{L} \iff f(\vec{x}) = 0$.*

Definition 6 (Public-coin zk-FLIOP [7]). *A public-coin fully linear interactive proof system over R with ρ -rounds, ℓ -queries and message lengths $(u_1, \dots, u_\rho) \in \mathbb{N}^\rho$, consists of a randomized prover algorithm P and a deterministic verifier algorithm V . Let the input to P be $\vec{x} \in R^m$ and let $r_0 = \perp$. In each round $i \in [\rho]$:*

1. *P outputs a proof $\vec{\pi}_i \in R^{u_i}$, computed as a function of $\vec{x}$, $r_1, \dots, r_{i-1}$ and its private randomness.*
2. *A random public challenge $r_i \leftarrow_{\$} S_i$ is sampled.*
3. *ℓ linear oracle queries $\vec{q}_1^i, \dots, \vec{q}_\ell^i \in R^{m+u_i}$ are determined based on $r_1, \dots, r_i$. Then, V receives ℓ answers $(\langle \vec{q}_1^i, x \mid \vec{\pi}_i \rangle, \dots, \langle \vec{q}_\ell^i, x \mid \vec{\pi}_i \rangle)$.*

At the end of round ρ , V outputs accept or reject based on the random challenges and all the answers to the queries.

Let $\mathcal{L} \subseteq R^m$ be a language. We say that ρ -round ℓ -query interactive fully linear protocol $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$ over R is zero-knowledge fully linear interactive oracle proof system for $\mathcal{L}$ with soundness error ε if it satisfies the following properties:

- **COMPLETENESS:** *If $x \in \mathcal{L}$, then V_{FLIOP} always outputs accept*
- **SOUNDNESS:** *If $x \notin \mathcal{L}$, then for all P^* , the probability that V_{FLIOP} outputs accept is at most $2^{-\varepsilon}$.*
- **ZERO KNOWLEDGE:** *There exists a simulator $\mathcal{S}_{\text{FLIOP}}$ such that for all $x \in \mathcal{L}$ it holds that $\mathcal{S}_{\text{FLIOP}} \equiv \text{view}_{[P_{\text{FLIOP}}(x), V_{\text{FLIOP}}]}(V_{\text{FLIOP}})$ (where the verifier's view $\text{view}_{[P_{\text{FLIOP}}(x), V_{\text{FLIOP}}]}(V_{\text{FLIOP}})$ consists of $\{r_i\}$ and $(\langle \vec{q}_1^i, \vec{x} \mid \vec{\pi}_i \rangle, \dots, \langle \vec{q}_\ell^i, \vec{x} \mid \vec{\pi}_i \rangle)$ for all $i \in [\rho]$).*

2.3.1 Concrete Instantiations for Degree-2 Languages

In Section B, we present a concrete instantiation for proving degree-2 languages, based on the constructions given in [7], where the size of the proof is sublinear in the size of the statement. Specifically, our instantiation is for a prover P who wants to prove to a verifier V that an input $(c, \vec{a}, \vec{b})$, where $\vec{a}$ and $\vec{b}$ are vectors of size m , is in the following language:

$$\mathcal{L}_{\text{prod}} = \left\{ (c, \vec{a}, \vec{b}) \in R^{1+2m} \mid c - \langle \vec{a}, \vec{b} \rangle = 0 \right\}.$$

Concretely, we show two instantiations: one with a constant number of rounds and a proof of square-root size, and one with a logarithmic number of rounds and a logarithmic proof size. For the constant-round proof, the size of the proof is $4\sqrt{m} + 1$ (over R) and the query complexity is $2\sqrt{m} + 2$. For the multi-round proof, the proof size is $(\log m - 1) \cdot 3 + 7$ and query complexity is $\log m + 3$.

Proving correctness of multiplication triples. We conclude with the following trivial claim, that the above construction can be used to prove correctness of a set of multiplication triples. Indeed, beyond sublinearity, the corresponding concrete costs are very low.

Claim 2.1. *There exists zk-FLIOP for proving $x \in R^{3m}$ is in $\mathcal{L}_{mult} = \left\{ (\alpha_j, \beta_j, \gamma_j)_{j=1}^m \mid \forall j \in [m] : \gamma_j = \alpha_j \cdot \beta_j \right\}$ where the proof size and the query complexity are sublinear in m .*

Proof: Note that the prover can prove that $\sum_{j=1}^m \theta_j \cdot (\gamma_j - \alpha_j \cdot \beta_j) = 0$ for random θ_j s chosen jointly by the prover and the verifier. Now, let $c = \sum_{j=1}^m \theta_j \cdot \gamma_j$, $\vec{a} = (\theta_1 \cdot \alpha_1, \dots, \theta_m \cdot \alpha_m)$ and $\vec{b} = (\beta_1, \dots, \beta_m)$. Thus, the prover's task goal is to prove that $c - \vec{a} \cdot \vec{b} = 0$. Hence, we can use the construction we mentioned in this section to prove that $x \in \mathcal{L}_{prod}$ with the desired complexity measures. ■

2.4 Distributed Zero-Knowledge Proofs (DZKP)

Boneh et al. [7] showed how to use zk-FLIOP to prove a statement shared across n verifiers in a distributed way. We present their protocol for the setting where either the prover *or* any subset of the verifiers is corrupted, which suffices for our $(n + 1)$ -party protocol constructions. Specifically, the verifiers hold a linear secret sharing of the input $\vec{x}$, and the zero knowledge property further guarantees that a subset of colluding verifiers learns nothing new about $\vec{x}$.

The idea of [7] is for the prover to simply additively secret share the zk-FLIOP proof to the n verifiers. Since the FLIOP queries are linear, it follows that each of the verifiers can run the linear queries on its shares of the proof and the statement, and then interact only to reconstruct the answers.

Protocol 7 (Π_{DZKP} - Distributed ZK proofs: malicious prover *or* verifiers).

Let σ be a statistical security parameter. Let P be a prover and $V_1, \dots, V_n$ be n verifiers. Let $C : R^m \rightarrow R$ be an arithmetic circuit over a ring R , and let $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$ be a zk-FLIOP protocol with ρ rounds, ℓ -queries per round, σ -soundness error and message length $u_1, \dots, u_\rho \in \mathbb{N}$ and challenge sets $S_1, \dots, S_\rho$ for the language $\mathcal{L}_C = \{\vec{x} \in R^m \mid C(\vec{x}) = 0\}$.

- **Input:** The prover P holds $\vec{x}$. Each verifier V_i holds $\vec{x}^i$ such that $x[j] = \sum_{i=1}^n x^i[j]$ for each $j \in [m]$.
- **The protocol:**
 1. For each round j of the zk-FLIOP:
 - (a) If $j = 1$, the prover P lets $\vec{\pi}_j = P_{\text{FLIOP}}(\vec{x}, \perp)$. Otherwise, it lets $\vec{\pi}_j = P_{\text{FLIOP}}(\vec{x}, r_1, \dots, r_{j-1})$.
 - (b) P chooses $\vec{\pi}_j^1, \dots, \vec{\pi}_j^n$ s.t. $\vec{\pi}_j = \sum_{i=1}^n \vec{\pi}_j^i$ and gives $\vec{\pi}_j^i$ to V_i .
 - (c) The parties call $\mathcal{F}_{\text{coin}}$ to receive $r_j \in S_j$.
 - (d) The parties let $\vec{q}_{j,1}, \dots, \vec{q}_{j,\ell}$ be the query vector determined by V_{FLIOP} based on $r_1, \dots, r_j$. Then, V_i computes the answers $a_{j,1}^i, \dots, a_{j,\ell}^i \leftarrow \langle \vec{q}_{j,1}, \vec{x}^i \mid \vec{\pi}_j^i \rangle, \dots, \langle \vec{q}_{j,\ell}, \vec{x}^i \mid \vec{\pi}_j^i \rangle$.
 - (e) The verifiers send their answers to each other to compute $a_{j,1}, \dots, a_{j,\ell} \leftarrow \sum_{i=1}^n a_{j,1}^i, \dots, \sum_{i=1}^n a_{j,\ell}^i$. (We remark that this can be deferred to the end of the execution)
 2. The verifiers run the decision predicate of V_{FLIOP} on all query answers received. If any party received reject, it outputs reject. Otherwise, it outputs accept.

Note that the communication in the protocol consists of the proof sent by the prover to each verifier, and messages exchanged between the verifiers, to reconstruct the queries' answers. For languages expressible by degree-2 circuits, the construction from Appendix B yields sublinear communication in the statement size, since both the proof size and number of queries (which determine the amount of communication between the verifiers) are sublinear.

Non-interactive DZKP. It was also shown in [7] how to make the above protocol non-interactive using the Fiat-Shamir transform [59]. Note that this is not immediate, since the transform works by setting the verifier randomness via a random oracle applied on the joint public state, whereas in the distributed setting, no such joint state exists. To overcome this, following [7], the prover publishes short statistically-hiding commitments for each party's view. Then, the parties can call the random oracle over the public commitments. For completeness, we give a formal description in Protocol 9. Note that communication remains roughly the same compared to the interactive version, with small sublinear additive overhead.

Lemma 8 ([7]). *For any poly-time adversary $\mathcal{A}$ controlling a subset of the online verifiers in the non-interactive DZKP, then in the random oracle model there exists a simulator $\mathcal{S}_{\text{NI-DZKP}}^{\mathcal{A}}$ that can simulate the view of $\mathcal{A}$ in the protocol with negligible statistical distance.*

Protocol 9 ($\Pi_{\text{DZKP}}^{\text{NI}}$ - Non-interactive Distributed ZK Proofs).

Let κ be statistical and computational security parameters, respectively and let P be the prover and $V_1, \dots, V_n$ be the verifiers. Let $C : R^m \rightarrow R$ be an arithmetic circuit over a ring R , and let $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$ be a zk-FLIOP protocol with ρ rounds, ℓ -queries per round, κ -soundness error and message length $u_1, \dots, u_\rho \in \mathbb{N}$ and challenge sets $S_1, \dots, S_\rho$, for proving that $C(\vec{x}) = 0$.

- **Input:** Same as in Protocol 7.
 - **The protocol:**
 1. The prover P does the following:
 - (a) For each round j of the zk-FLIOP:
 - i. If $j = 1$, the prover P lets $\vec{\pi}_j = P_{\text{FLIOP}}(\vec{x}, \perp)$. Otherwise, it lets $\vec{\pi}_j = P_{\text{FLIOP}}(\vec{x}, \vec{\pi}_{j-1}, r_{j-1})$.
 - ii. P Chooses $\vec{\pi}_j^1, \dots, \vec{\pi}_j^n$ such that $\vec{\pi}_j = \sum_{i=1}^n \vec{\pi}_j^i$.
Then, it chooses random $t_j^i \in \{0, 1\}^{\text{poly}(\kappa)}$ and computes $c_j^i = \text{Com}(\vec{\pi}_j^i, t_j^i)$ for each $i \in [n]$.
 - iii. P Sends $c_1^1 || \dots || c_1^n || \dots || c_j^1 || \dots || c_j^n$ to $\mathcal{F}_{\text{RO}}$ to receive $r_j \in S_j$.
 - (b) P sends $\vec{\pi}_i = \{(\vec{\pi}_j^i, t_j^i, c_j^1, \dots, c_j^n)\}_{j \in [\rho]}$ to V_i .
 2. Each verifier V_i does the following:
 - (a) For each round j in the zk-FLIOP:
 - i. V_i checks that $c_j^i = \text{Com}(\vec{\pi}_j^i, t_j^i)$. If it holds, V_i send $c_1^1 || \dots || c_1^n || \dots || c_j^1 || \dots || c_j^n$ to $\mathcal{F}_{\text{RO}}$ to receive r_j . Otherwise, it sends **abort** to the other verifier and aborts the protocol. V_i computes the queries and the answers as in Protocol 7.
 - (b) Each V_i sends their answers to the other verifiers and run the decision predicate of V_{FLIOP} on the queries' answers. If any party received **reject**, then it outputs **reject**. Otherwise, it outputs **accept**.
-

3 Semi-Honest Computation in the $n + 1$ Setting

In this section we present a simple and secure computation for arithmetic circuits over finite rings in the $n + 1$ setting under semi-honest corruptions. That is, our protocol works in the offline-online setting where an offline dealer distributes correlated randomness to the n “online” parties that is later consumed by them during an online phase. The protocol is secure against semi-honest adversaries corrupting either the dealer or any subset of the n parties. For simplicity, when securely evaluating a circuit C over $\mathcal{R}$, we denote by i_j the number of input wires provided by party P_j to that circuit. We also assume that input wires are ordered by the party index that is providing them, so that the first i_1 inputs are provided by P_1 , the following i_2 inputs are provided by P_2 and so on. Formally, the protocol is parameterized by a ring $\mathcal{R}$ and securely computes the functionality $\mathcal{F}_{\text{SH}}$ described below. Essentially, this is an $n + 1$ -party functionality, taking input from a dealer describing the size of the circuit to be computed in the online phase. Each online party’s input is a set of values in $\mathcal{R}$ for the appropriate input wires in C associated with that party. The functionality evaluates the circuit using the inputs provided by the parties and hands the parties the outputs of the circuit.

Functionality 10 (The Ideal Functionality $\mathcal{F}_{\text{SH}}$).

The functionality works with $n + 1$ parties, one of which is a dealer $\mathcal{D}$ and the remaining n parties are referred to as “online parties”. The functionality is parameterized by a ring $\mathcal{R}$ and a circuit C of size m over $\mathcal{R}$ with i input wires and o output wires.

1. The dealer $\mathcal{D}$ sends the functionality the message **init**.
 2. Upon receiving a message **init** from the dealer, the functionality sends the n online parties the message **ready**. It also sends the dealer its *empty* output message (**output**, $\perp$).
 3. After receiving a message **ready** from the functionality, each online party P_j sends the functionality the message (**input**, $\vec{x}_j$) where $\vec{x}_j$ is a vector containing party P_j ’s input to the circuit.
 4. After receiving a message (**input**, $\vec{x}_j$) from party P_j the functionality stores $\vec{x}_j$.
 5. After receiving all n input messages from n online parties the functionality concatenates all input vectors to compute an input vector $\vec{x}$. The functionality computes an output vector $\vec{y} = C(\vec{x})$ by evaluating the circuit C .
 6. The functionality sends all online parties the message (**output**, $\vec{y}$).
-

In Protocol 12 we present the protocol Π_{SH} for securely realizing $\mathcal{F}_{\text{SH}}$ in the $n + 1$ setting under passive corruptions. The protocol works in the offline-online setting where in the offline setting the dealer hands to the parties m multiplication triples over the ring $\mathcal{R}$, as well as shares of a mask for each output wire value. The triples are compressed using a PCG seed using an appropriate PCG for multiplication triples over the ring $\mathcal{R}$. Next, in the online phase, each online party P_j holds a circuit C and a vector $\vec{x}_j$ containing its input to the wires associated with it.

Throughout the computation, the parties maintain an additive sharing of values on the cir-

cuit's wires. Hence, linear gates can be computed without any interaction. On the other hand, multiplication gates require interaction. We recall the semi-honest multiplication protocol based on multiplication triples [60]. To multiply two additively shared inputs x_1 and x_2 over some ring $\mathcal{R}$, i.e., $\llbracket x_1 \rrbracket$ and $\llbracket x_2 \rrbracket$, the parties make use of a shared random triple $\llbracket r_1 \rrbracket, \llbracket r_2 \rrbracket, \llbracket r_1 \cdot r_2 \rrbracket \in \mathcal{R}^3$, distributed to them by the dealer. Then:

1. The parties reveal $\hat{x}_1 = x_1 - r_1$ and $\hat{x}_2 = x_2 - r_2$ (by sending their masked shares to each other).
2. The parties locally compute ³

$$\llbracket x_1 \cdot x_2 \rrbracket = \hat{x}_1 \cdot \hat{x}_2 + \hat{x}_1 \cdot \llbracket r_2 \rrbracket + \hat{x}_2 \cdot \llbracket r_1 \rrbracket + \llbracket r_1 \cdot r_2 \rrbracket \quad (1)$$

It is easy to verify that if all parties act honestly, the output will be correct. We note that over Boolean circuits, the addition operation is replaced by XOR and multiplication is replaced by AND.

Remark 11 (Circuit Independent Offline Phase). *Our protocol is described by the dealer distributing circuit independent multiplication triples using an appropriate PCG, this makes the dealer and the offline phase of the protocol circuit independent (i.e. depending only on the circuit size). In the circuit-dependent model [61, 62], it is possible to cut the communication by half (in this case, the parties interact only to reveal the masked output).*

We provide a formal description of the semi-honest protocol Π_{SH} in Protocol 12 which follows the protocol of [60], but with a dealer who hands the parties the multiplication triples in a compressed manner using PCGs.

Theorem 13. *Let C be a circuit over a ring $\mathcal{R}$ with m multiplication triples, i input wires and o output wires and let n be a positive integer. Protocol 12, Π_{SH} parameterized by $\mathcal{R}, n, i, o, m$ and C securely evaluates the functionality $\mathcal{F}_{\text{SH}}$ in the offline-online model under the $(n + 1)$ setting in the presence of semi-honest adversaries corrupting either the offline dealer or any subset of the n online parties.*

Proof. We provide a proof by presenting a simulator $\mathcal{S}_{\mathcal{D}}$ for simulating the view of a passively corrupted dealer and a simulator $\mathcal{S}_T$ for simulating the view of any passively corrupted subset T of the online parties.

Starting with the simpler case, the simulator $\mathcal{S}_T$ receives from the dealer all its messages in the offline phase, sends the `init` message to the functionality $\mathcal{F}_{\text{SH}}$, receives the dealer's empty output (`output`, $\perp$) from the functionality and passes $\perp$ to the dealer as its output and terminates.

We now move to simulating the view of any subset T of the online parties such that $|T| \leq n - 1$. We denote by $\bar{T}$ the set of honest online parties, so $T \cup \bar{T} = \{P_1, \dots, P_n\}$. We present a simulator $\mathcal{S}_{\text{SH}}^T$ that can simulate the view of the parties in T in the real execution of Π_{SH} from interacting with the functionality $\mathcal{F}_{\text{SH}}$.

³ Note that the term $\hat{x}_1 \cdot \hat{x}_2$ is actually added only by one of the parties. This follows since computing $c + \llbracket x \rrbracket$ for some public constant c , where x is additively shared, requires only one of parties to add c to its share of x .

Protocol 12 (Π_{SH} - Semi-honest Computation of an Arithmetic Circuit with a Dealer).

Parameters: A ring $\mathcal{R}$, number of parties n and upper bounds i, o, m on the number of input and output wires and multiplication gates in the online circuit to be computed. A PCG scheme (Gen, Expand) for generating m pseudorandom multiplication triples over $\mathcal{R}$.

Offline Phase

- 1: $\triangleright$ The dealer $\mathcal{D}$ generates n -party PCG seeds for the semi-honest multiplication triples: $\triangleleft$
 - 2: $(\text{SH}_1, \dots, \text{SH}_n) \leftarrow \text{PCG.Gen}(1^\kappa, n, m)$.
 - 3: The dealer hands to each party P_j its semi-honest PCG seed for multiplication triples SH_j .
 - 4: For each output wire w of C the dealer samples a mask r_w and hands the parties an additive secret sharing $\llbracket r_w \rrbracket$.
 - 5: For each input wire w of C provided by party P_j , the party P_j samples a random sharing of $m_w = \llbracket 0_{\mathcal{R}} \rrbracket$ and hands to each party P_k for $k \in [n]$ its share $\llbracket m_w \rrbracket_k$.
 - 6: Party P_j outputs SH_j as well as $\llbracket r_w \rrbracket_j$ for each output wire w as well as $\llbracket m_w \rrbracket_j$ for each input wire w .
 - 7: $\triangleright$ If untrusted dealer (i.e. protocol Π_{SH}^*), the dealer and online parties execute protocol $\Pi_{\text{DZKP}}^{\text{NI}}$ to generate a non-interactive proof for the correctness of the authenticated triples as shown in Section 5.3.1. $\triangleleft$
-

Online Phase (Computation)

Input:

- A circuit C with i input wires, o output wires, and m multiplication gates over $\mathcal{R}$.
 - Each party P_j holds its input $\vec{x}_w$ for each input wire w associated with it in C .
- 1: Each party P_j expands its PCG seed SH_j to obtain its share of m multiplication triples over $\mathcal{R}$.
 - 2: **for** each input wire w provided by party P_j **do**
 - 3: Party P_j sets its share of the input wire $\llbracket x_w \rrbracket = \llbracket x_w \rrbracket + \llbracket m_w \rrbracket_j$.
 - 4: Each party P_k for $k \neq j$ sets $\llbracket x_w \rrbracket \leftarrow \llbracket m_w \rrbracket_k$.
 - 5: $\triangleright$ Iterate the gates in topological ordering $\triangleleft$
 - 6: **for** each gate g in C with input wires u, v and output wire w **do**
 - 7: **if** g is an addition gate **then**
 - 8: Each party P_j computes $\llbracket x_w \rrbracket_j \leftarrow \llbracket x_u \rrbracket_j + \llbracket x_v \rrbracket_j$.
 - 9: **else** $\triangleright g$ is a multiplication gate
 - 10: Let $(\llbracket r_1 \rrbracket, \llbracket r_2 \rrbracket, \llbracket r_1 \cdot r_2 \rrbracket)$ be a fresh multiplication triple shared across the parties.
 - 11: Each P_j computes $\llbracket \hat{x}_u \rrbracket_j \leftarrow \llbracket x_u \rrbracket_j - \llbracket r_1 \rrbracket_j$ and $\llbracket \hat{x}_v \rrbracket_j \leftarrow \llbracket x_v \rrbracket_j - \llbracket r_2 \rrbracket_j$ and broadcasts $\llbracket \hat{x}_u \rrbracket_j$ and $\llbracket \hat{x}_v \rrbracket_j$.
 - 12: Each party reconstructs $\hat{x}_u$ and $\hat{x}_v$ by summing the shares it received.
 - 13: Party P_1 computes $\llbracket x_w \rrbracket \leftarrow \hat{x}_u \cdot \hat{x}_v + \hat{x}_u \cdot \llbracket r_2 \rrbracket + \llbracket r_1 \rrbracket \cdot \hat{x}_v + \llbracket r_1 \cdot r_2 \rrbracket$.
 - 14: All parties except P_1 compute $\llbracket x_w \rrbracket \leftarrow \hat{x}_u \cdot \llbracket r_2 \rrbracket + \llbracket r_1 \rrbracket \cdot \hat{x}_v + \llbracket r_1 \cdot r_2 \rrbracket$.
 - 15: **for** each output wire w with mask $\langle\langle r_w \rangle\rangle$ **do**
 - 16: Each party P_j computes $\llbracket \hat{x}_w \rrbracket_j \leftarrow \llbracket x_w \rrbracket_j - \llbracket r_w \rrbracket_j$.
 - 17: Each party reconstructs $\hat{x}_w$ by summing the shares it received.
 - 18: Each party P_j outputs $\llbracket \hat{x}_w \rrbracket_k$ for each output wire w and each input wire to a multiplication gate that it received from each party P_k .
-

Output Opening

- 1: **for** each output wire w with mask $\llbracket r_w \rrbracket$ **do**
 - 2: The parties open their shares of r_w
 - 3: Each party reconstructs $x_w \leftarrow \hat{x}_w + r_w$.
 - 4: Each party outputs the reconstructed value of each output wire.
-

Simulating the offline phase. Upon receiving the ready message from the functionality, the simulator $\mathcal{S}_{\text{SH}}^T$ follows the role of an honest dealer who samples a PCG seed for generating m multiplication triples over the ring $\mathcal{R}$ and $\llbracket r_w \rrbracket$ of a random r_w for each output wire w and hands to each party in T its PCG seed SH_j and its share $\llbracket r_w \rrbracket_j$ of each r_w . For each input wire w provided by party $P_j \in T$ the simulator receives the shares of $m_w = 0_{\mathcal{R}}$ of all honest parties. Let $\llbracket m_w \rrbracket_{\bar{T}} = \sum_{P_j \in \bar{T}} \llbracket m_w \rrbracket_j$. The simulator also sends for each input wire w provided by an honest party $P_j \in \bar{T}$ a random share $\llbracket m_w \rrbracket_k$ for each party $P_k \in T$ to the adversary.

Simulating the online phase. In the online phase the simulator knows the inputs of the parties in T . It sends the messages $(\text{input}, \vec{x}_j)$ to the functionality for each party $P_j \in T$. We emphasize that since we do not consider malicious adversary we do not need to extract corrupted parties' inputs before sending the inputs to the functionality. The simulator receives from the functionality the output $\vec{y}$, and stores it for the output opening phase. Now, to simulate the view during the protocol we need only to simulate the evaluation of multiplication gates. In this process for each multiplication gate g with input wires u, v we simulate honest parties $P_j \in \bar{T}$ messages of $\hat{x}_u$ and $\hat{x}_v$ by sending randomly distributed values. To simulate the output opening, let w be an output wire and let $\vec{y}_w$ be the value from $\vec{y}$ (received from the functionality) that should be output on output wire x_w . The simulator $\mathcal{S}_T^{\text{SH}}$ knows the shares held by the adversary on the wire x_w and let $\llbracket r_w \rrbracket_T = \sum_{P_j \in T} \llbracket r_w \rrbracket_j$ and be the sum of the shares of the mask of the output wire w held by the corrupted parties in T . In the output opening phase the simulator sends the shares of $\llbracket r_w \rrbracket$ opened by the honest parties in $\bar{T}$ as random values in $\mathcal{R}$ under the constraint their sum equals $y_w - \hat{x}_w - \llbracket r_w \rrbracket_T$. That way, the resulting opened value on wire w will be y_w as necessary.

Since the functionality is deterministic it is only needed to show that the view of the adversary in the ideal world execution and the real world execution are indistinguishable. In the offline phase the parties in T are given PCG seeds that are perfect simulation of the real world execution: The simulator computes them using exactly the same algorithm as the honest dealer in the real execution. The shares of each m_w handed by an honest party is also simulated perfectly as random shares are given to the adversary. In the online phase for each multiplication gate g with input wires u, v the view in the ideal world of the shares of $\llbracket \hat{x}_u \rrbracket, \llbracket \hat{x}_v \rrbracket$ are sampled randomly by the simulator. In the real world execution, the shares of $\llbracket \hat{x}_u \rrbracket, \llbracket \hat{x}_v \rrbracket$ are not uniformly distributed and a computationally unbounded distinguisher can find the honest parties PCG shares and distinguish between the real and ideal world distributions. However, if a computationally bounded distinguisher could distinguish between the ideal and real world executions, we can show that this implies that the underlying PCG is insecure by construct a view of the corrupted parties using the corrupted parties' shares and the sum of shares of the honest parties for the semi-honest protocol. This can be done by taking the triples and generating an appropriate view of the corrupted parties in the protocol; if the distinguisher decides view is sampled from the real world then the PCG distinguisher will decide that its sample is a result of a PCG key expansion, otherwise, if the distinguisher finds the view to be generated from an ideal-world execution, the PCG distinguisher will decide that its sample is a result of reverse sampling the correlation based on the PCG seed it received. ■

4 $n+1$ MPC: SPDZ Approach

In this section, we present an efficient method to adapt the SPDZ paradigm [11] to the $n+1$ setting, providing a lightweight protocol for online parties to jointly verify the validity of alleged

SPDZ authenticated triple correlations.

The SPDZ paradigm and its variants (e.g., [36, 63, 39, 14]) are considered the mainstream approach for dishonest-majority secure computation. While works in this line typically focus on secure generation of the offline correlation, it is natural to outsource the generation of this data to an external dealer—particularly if the parties could *verify* that this information was indeed generated correctly. To the best of our knowledge, however, SPDZ with an *untrusted* dealer has yet to be considered. We show that the correctness of the correlated randomness required for the SPDZ protocol can be verified efficiently by the online parties using zk-FLIOP, resulting in minimal additional communication (logarithmic in the circuit size, with good concrete constants), and lightweight verification amongst the n parties.

In the following, we show how to verify the circuit-independent variant; adapting to the circuit-dependent version is straightforward. The SPDZ protocol with a dealer is built on top of the semi-honest protocol presented in the previous section where the dealer merely serves as a source of a (possibly untrusted but non-colluding) source of correlation. We begin with describing the differences from the semi-honest protocol assuming a trusted dealer. Next, we show how to verify the correlations provided by the dealer are correct. In a nutshell, the SPDZ protocol achieves malicious security by maintaining the invariant that on each wire the parties hold the pair $\llbracket x \rrbracket, \llbracket \Delta \cdot x \rrbracket$, for some random secret Δ which is shared to the parties. Since Δ is unknown, $\Delta \cdot x$ can be viewed as a MAC on x , which enables cheating detection when revealing x . That is, the computation in the online phase of SPDZ proceeds exactly as in the online phase of the semi-honest protocol Π_{SH} from Protocol 12 with the only difference that parties maintain not only additive shares of the values on the wires, but they leverage authenticated shares of the values on the wires instead. To compute multiplication gates, the parties use *authenticated* random triples. Specifically, for each multiplication gate, the parties are given the tuple: $(\llbracket r_1 \rrbracket, \llbracket r_2 \rrbracket, \llbracket r_1 \cdot r_2 \rrbracket, \llbracket \Delta \cdot r_1 \rrbracket, \llbracket \Delta \cdot r_2 \rrbracket, \llbracket \Delta \cdot (r_1 \cdot r_2) \rrbracket)$. Given these triples, the online communication stays the same as before. So that in each multiplication gate the parties open their shares of masked wire values from which and using a fresh authenticated multiplication triple they can obtain authenticated shares of the result of the multiplication. At the end of the protocol the parties only need to perform a small constant-cost check at the end of the protocol, before the outputs are revealed. One last difference from the semi-honest protocol is that the output wires' masks now need to be authenticated to prevent malicious parties from providing incorrect shares of the masks.

Verifying the SPDZ correlation. We observe that correct structure of the SPDZ correlation for m multiplication gates can be expressed as a collection of $4m$ constraints of degree 2 (over some field $\mathbb{F}$ or a binary finite ring). Concretely, let $(\lambda_\alpha^k, \lambda_\beta^k, \lambda_{\alpha\beta}^k, \lambda_{\Delta \cdot \alpha}^k, \lambda_{\Delta \cdot \beta}^k, \lambda_{\Delta(\alpha\beta)}^k)$ be the additive reconstruction of the k -th tuple shares received by the parties. Then, the parties wish to verify that the following constraints hold (i.e., verifying correctness of the semi-honest triple, and proper Δ -scaling between the two triples): (i) $\lambda_{\alpha\beta}^k - \lambda_\alpha^k \cdot \lambda_\beta^k = 0$, (ii) $\lambda_{\Delta \cdot \alpha}^k - \Delta \cdot \lambda_\alpha^k = 0$, (iii) $\lambda_{\Delta \cdot \beta}^k - \Delta \cdot \lambda_\beta^k = 0$ and (iv) $\lambda_{\Delta(\alpha\beta)}^k - \Delta \cdot \lambda_{\alpha\beta}^k = 0$. By Claim 2.1, there exists a zk-FLIOP for proving that a list of multiplication triples is correct, with proof size and query complexity that are of size $O(\log m)$. Also note that each input is additively shared between the online parties. Hence, by invoking Protocol 9 with this zk-FLIOP construction, we obtain a non-interactive protocol to prove that the SPDZ correlation is correct with low cost as required.

5 $n+1$ MPC: FLIOP-Based Approach

In this section, we present our FLIOP-based approach for achieving maliciously secure MPC of any circuit C over a ring R in the $(n+1)$ setting.

As in the SPDZ approach, we use the semi-honest Beaver-based multiplication protocol. However, we verify correctness using DZKP, instead of a MAC over each wire. As a consequence, the online DZKP-based verification protocol will have sublinear (possibly polylogarithmic in the circuit size) communication cost instead of a constant as in the SPDZ approach. Nevertheless, the verification cost is still amortized away for large enough circuits. The big advantage of our protocol lies in the size of the dealer’s message. Specifically, both the correlated randomness and the proof of its correctness will all have size that is polylogarithmic in the computed circuit’s size. As we discuss later in Section 6, these carry over to significant concrete savings.

Overview. In Figure 2 we present a schematic visual overview of the protocol with the building blocks comprising it and reference to the appropriate sections in which each building block is discussed. In our protocol, the dealer’s single message to each party P_i consists of four items:

- i) Semi-honest correlations, i.e., shared multiplication triples, denoted SH_i ; These are generated according to Section 2.2.
- ii) A set of n instances *FLIOP correlations* which are used by the parties in the online verification protocol and allows for each party to both prove it acted honestly during the semi-honest protocol (this part of the FLIOP correlation is denoted by $\mathbf{F}_i^P$) and also to verify that other parties behaved honest (this part of the FLIOP correlation is denoted by the set $\{\mathbf{F}_j^V\}_{j \neq i}$); These “FLIOP correlations” will be defined and presented in greater depth throughout this section.
- iii) Proof of correctness for the semi-honest correlations denoted by $\vec{\pi}_i^{\text{SH}}$.
- iv) Proof of correctness for the FLIOP correlations denoted by $\vec{\pi}_i^P$ and the set $\{\vec{\pi}_i^V\}_{j \neq i}$.

Following section 2.2 we observe that the sizes of SH_i is polylogarithmic in the circuit size. In section 5.2 we will show, by following the Boyle et al. [8] approach, that the size of the FLIOP correlations can be made polylogarithmic in the size of the circuit. Finally, in sections 5.3.1 and 5.3.2 we show that the sizes of $\vec{\pi}_i^{\text{SH}}$, $\vec{\pi}_i^P$ and $\vec{\pi}_i^V$ are also polylogarithmic in the size of the circuit. Following the above we reach the following corollary.

Corollary 14. *The size of the dealer’s message is polylogarithmic in the size of the circuit.*

We make the instructive decision to provide the following a high-level overview of our main protocol for secure computation in the $(n+1)$ setting before diving into the details of the protocol’s building blocks. Then, in section 5.4, after building blocks have been presented, we provide a more detailed construction of our main protocol.

Offline Phase In the offline phase, the dealer hands the parties items (i)–(iv) listed above. Before the online protocol begins, the parties expand the PCG seeds from part (i) to shared multiplication triples, and verify them using the proof of correctness from part (iii). Next, they assert the correctness of the FLIOP correlations’ from part (ii) using the proof of correctness from

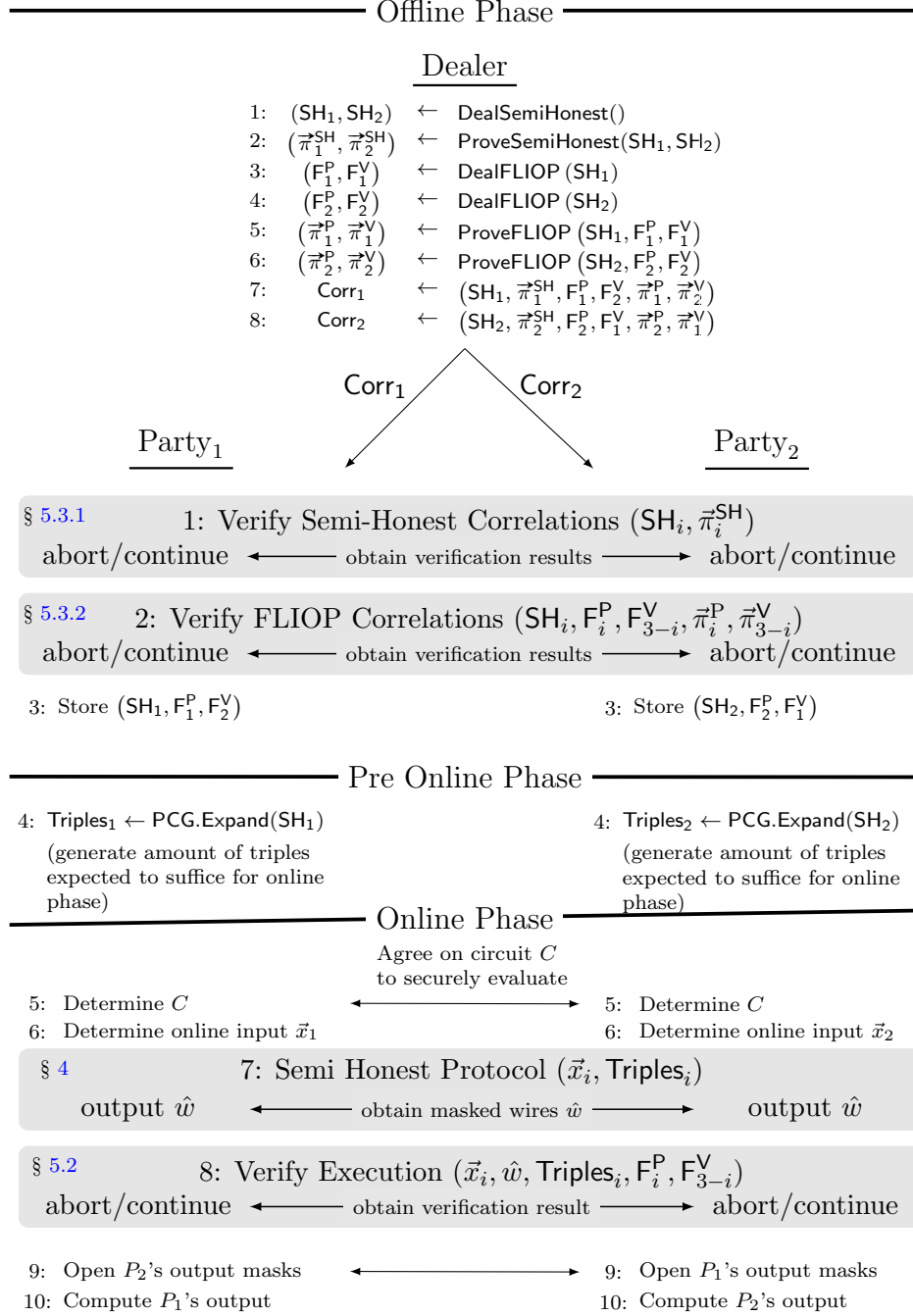


Figure 2: Schematic overview of the main secure computation protocol in the $(n + 1)$ setting. This figure uses two online parties, i.e. $n = 2$.

part (iv). If the dealer's proof is accepted, then the parties can dismiss the proofs of correctness (i.e parts (iii) and (iv) of the dealer's message) and proceed to the online phase.

Online Phase In the online phase, the parties first compute the circuit until they reach the output wires using the semi-honest protocol, while consuming the random triples from part (i) of the dealer's message. Then, *before* the output is revealed, they verify the semi-honest computation using the FLIOP correlations from part (ii) of the dealer's message. In this step, each party proves it sent the correct messages during the semi-honest computation to the other parties. If the verification ends successfully (i.e., all proofs are accepted), the parties can proceed to reveal the output.

5.1 FLIOP Correlations

In this section we introduce the notion of FLIOP correlations and provide the relevant definitions. FLIOP correlations will be used, as presented later in Section 5.2, to verify the correctness of a semi-honest secure computation in the $(n + 1)$ model.

Recall the FLIOP-based DZKP protocol (Protocol 7). This protocol involves a prover P and two verifiers, denoted by V_1, V_2 . At the beginning of the protocol the prover holds an input vector $\vec{x}$ and each verifier V_i holds a share $[\![\vec{x}]\!]_i$. The goal of the prover is to convince the verifiers that $\vec{x}$ satisfies some simple property $C(\vec{x}) = 0$, which in our case will be captured by degree-2 relations between the entries. This protocol (Protocol 18) proceeds in ρ rounds where in each round $j \in [\rho]$:

1. The prover P computes the round proof $\vec{\pi}_j$ and additively secret-shares it across the verifiers.
2. The parties sample a round challenge r_j and derive the round queries $\vec{q}_j$ from all round challenges sampled up to this point $r_1, \dots, r_j$. See Protocol 7 for more details about the sampling procedure.
3. Each verifier V_i computes its share of query answers $[\![\vec{a}_j]\!]_i = \langle \vec{q}_j, [\![\vec{x}]\!]_i | [\![\vec{\pi}_j]\!]_i \rangle$ to the queries $\vec{q}_j$.
4. The verifiers exchange their shares of the query answers and send r_j to the prover.

Informally, the goal of an FLIOP correlation is to replace one of the verifiers, without loss of generality V_1 , by an offline dealer who distributes circuit-independent correlated randomness to the prover and the online verifier V_2 . The high-level idea is to precompute the offline input share $[\![\vec{x}]\!]_1$ of V_1 as part of the correlation, and determine the online share $[\![\vec{x}]\!]_2$ based on the input $\vec{x}$ and $[\![\vec{x}]\!]_1$.

That is, only in a subsequent online phase the value of the second share of the input $[\![\vec{x}]\!]_2$ will be determined (and therefore of the shared input vector $\vec{x}$ itself) and known to the prover and the online verifier, such that the online verifier and the prover can accomplish the DZKP protocol and assert that $C(\vec{x}) = 0$. Following the above sketch of Protocol 7, for the offline verifier to compute its part in the DZKP, the offline verifier needs to know in advance the round challenges r_j and its share of the round proofs $\vec{\pi}_j$ from which it can compute its share of the query answers $[\![\vec{a}_j]\!]_1$ using its predetermined share of the input $[\![\vec{x}]\!]_1$. The FLIOP correlations, therefore, are handed to the prover P and the online verifier V_2 such that:

- **The Verifier's Share:** The online verifier's share of the FLIOP correlation is made of the round challenges r_j and the shares of the query answers of the offline verifier for each DZKP round: $\{[\![\vec{a}_j]\!]_1 | j \in [\rho]\}$.
- **The Prover's Share:** The prover P is given the shares of the round proofs used by the offline verifiers. That is, for each round j the prover's FLIOP correlation contains $\{[\![\vec{\pi}_j]\!]_1 | j \in [\rho]\}$.

That way the prover can provide the remaining online verifier V_2 with the shares of the round proofs subject to the constraint that the sum of both shares used by the offline and online verifiers is equal to the round proof. To enforce honest behavior of the verifiers when they open the round challenges, the prover is also given a commitment (or an authenticated secret share) to each round challenge that is verified when the round challenge is opened.

The above description assumes a prover and the two verifiers that do not collude. We in fact consider a stronger notion of FLIOP correlations that allows for a *set* of online verifiers T holding, in the online phase, a common share of the input $[\vec{x}]_2$ to each verify that $C(\vec{x}) = 0$ in a way that is robust even if some of the online verifiers in T and the prover are corrupted. In this case, since some of the online verifiers in T may be corrupted and collude with the prover, we cannot hand the challenge nor the query answers to the verifiers as before. Instead, each verifier's share of the FLIOP correlation contains an *authenticated share* of the round challenges r_j and query answers $[\vec{a}_j]_i$. The prover, in that case, is given a commitment to the authenticated share of each party for each round challenge. Then, both the round challenges and the query answers are revealed (by having each online verifier broadcasting its share of the round challenge and query answers), only when they should be revealed, as we later show in this section. The fact that the prover is given a commitment (or an authenticated secret share) to the authenticated shares of the round challenges and query answers ensures that the prover can verify that maliciously acting verifiers cannot select the round challenges adversarially. This is important, in particular, because zk-FLIOP schemes are proven to possess the zero-knowledge property only in presence of honest verifiers (i.e. Honest-Verifier Zero-Knowledge).

We detail the FLIOP correlation sampling algorithm **SampleFLIOP** in Algorithm 1. The algorithm outputs an $n + 1$ tuple of FLIOP correlations, where the first element is the prover's share and the remaining n elements are the online verifiers' shares.

We now provide a formal definition of the FLIOP correlations.

Definition 15 (FLIOP Correlations). *Let $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$ be a ρ -round zk-FLIOP protocol with ℓ queries per round and message length $u_1, \dots, u_\rho \in \mathbb{N}$. Let Π_{DZKP} be the two-verifier instantiation of the DZKP protocol of Protocol 7 for circuit $C : R^m \rightarrow R$ parameterized by the zk-FLIOP protocol $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$. Let $\vec{x} \in R^m$, let $n \in \mathbb{N}$ and let σ be a statistical secure parameter. An n -verifier σ -sound FLIOP correlation for Π_{DZKP} with offline input $\vec{x}$ is an output of the randomized procedure $\text{SampleFLIOP}(1^\sigma, 1^n, R, m, C, \vec{x}, (P_{\text{FLIOP}}, V_{\text{FLIOP}}))$ of Algorithm 1.*

Remark 16. *We emphasize to following subtlety in the definition above: The FLIOP correlation itself is n -verifier in the sense that it allows a set of n -verifiers to verify a certain statement. However, it provides the means for each of the online n verifiers to execute a two-verifier DZKP with the offline verifier (i.e. the dealer).*

Next, we show in Protocol 18 how a prover and a set of n verifiers sharing an FLIOP correlation provided by an *honest* dealer with offline input $[\vec{x}]_1$ can execute a DZKP protocol to verify that $C(\vec{x}) = 0$ for some $[\vec{x}]_2$ that is only determined in an online phase. The protocol works exactly like the DZKP protocol of Protocol 7 with the exception that the dealer's part is moved to the offline phase and its security (against a corrupted prover who may collude a corrupted subset of the verifiers) follows directly from the security of the zk-FLIOP protocol 7.

Theorem 17. *Let σ be a statistical security parameter, R a ring, $m, n \in \mathbb{N}$, and $C : R^m \rightarrow R$ be a degree d circuit. Let $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$ be a ρ -round σ -sound zk-FLIOP protocol with ℓ queries per*

Algorithm 1 SampleFLIOP ($\vec{x}_{\text{off}}$)

Parameters:

- 1^σ : statistical security parameter.
- 1^n : the number of online verifiers.
- R : A ring.
- m : An integer.
- C : A circuit mapping $R^m \rightarrow R$.
- $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$: The prover and verifier algorithms of a ρ round zk-FLIOP protocol with ℓ queries per round and message length $u_1, \dots, u_\rho \in \mathbb{N}$.

Input: $\vec{x}_{\text{off}}$: The DZKP offline input in R^m .

Output: An n -verifier FLIOP correlation with offline input $\vec{x}_{\text{off}}$.

```
1: for  $i \in [\rho]$  do
2:   Let  $S_i$  be the challenge set of the  $i$ -th zk-FLIOP round
3:    $r_i \leftarrow \$ S_i$ 
4:    $\vec{\pi}_i \leftarrow \$ R^{u_i}$ 
5:   Compute the  $i$ -th round queries  $\vec{q}_i^1, \dots, \vec{q}_i^\ell$  from  $r_1, \dots, r_i$ 
6:   For  $j \in [\ell]$  set  $a_i^j \leftarrow \langle \vec{q}_i^j, \vec{x}_{\text{off}} || \vec{\pi}_i \rangle$ 
7:   Let  $A_i = \{a_i^1, \dots, a_i^\ell\}$ 
8:    $\triangleright$  Compute the authenticated secret sharing of the round challenge  $r_i$  and query answer's
      shares  $A_i$   $\triangleleft$ 
9:    $(\langle\langle A_i, r_i \rangle\rangle_0, \dots, \langle\langle A_i, r_i \rangle\rangle_n) \leftarrow \text{AuthShare}((A_i, r_i), n+1)$ 
10:  $C_P \leftarrow (\vec{\pi}_1, \dots, \vec{\pi}_\rho, \langle\langle A_1, r_1 \rangle\rangle_0, \dots, \langle\langle A_\rho, r_\rho \rangle\rangle_0)$ 
11: for  $j \in [n]$  do
12:    $C_V^j \leftarrow (\langle\langle A_1, r_1 \rangle\rangle_j, \dots, \langle\langle A_\rho, r_\rho \rangle\rangle_j)$ 
output  $(C_P, C_V^1, \dots, C_V^n)$ 
```

round and message length $u_1, \dots, u_\rho \in \mathbb{N}$ for the degree d language $\mathcal{L}_C = \{\vec{x} \in R^m \mid C(\vec{x}) = 0\}$. Then, assuming an honest dealer, the protocol $\Pi_{\text{DZKP}}^{\text{off}}$ (Protocol 18) instantiated by the parameters σ, κ, C, R, n and m satisfies the following properties:

1. **Correctness:** If all parties act honestly, for every $\vec{x} \in R^m$ such that $C(\vec{x}) = 0$, the protocol $\Pi_{\text{DZKP}}^{\text{off}}$ outputs accept with probability 1.
2. **Soundness:** For every $\vec{x} \in R^m$ such that $C(\vec{x}) \neq 0$, the probability that an honest verifier outputs accept is negligible in σ .
3. **Zero-Knowledge:** For every poly-time adversary $\mathcal{A}$ controlling a subset of at most $n-1$ online verifiers, there exists a simulator $\mathcal{S}_{\text{DZKP}}^{\mathcal{A}}(\llbracket \vec{x} \rrbracket_2)$ that can perfectly simulate the view of $\mathcal{A}$ in $\Pi_{\text{DZKP}}^{\text{off}}$, that is:

$$(\mathcal{S}_{\text{DZKP}}^{\mathcal{A}}(\llbracket \vec{x} \rrbracket_2)) \equiv \left(\left\{ \text{view}_{\mathcal{A}}^{\Pi_{\text{DZKP}}^{\text{off}}}, \text{out}_{\text{Honest}}^{\Pi_{\text{DZKP}}^{\text{off}}} \right\} \right)$$

Proof. The correctness and soundness of the protocol follow directly from the correctness and soundness of the zk-FLIOP protocol $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$. In particular, an adversary capable of

successfully proving a false statement in the DZKP protocol would be able to break the soundness of the underlying zk-FLIOP protocol. Let $\llbracket \pi_i \rrbracket_1, \llbracket \pi_i \rrbracket_2$ be the sharing of the prover's proof in round i in the DZKP protocol. In short, by using the proof $\pi_i = \llbracket \pi_i \rrbracket_1 + \llbracket \pi_i \rrbracket_2$ in the zk-FLIOP, the prover can successfully prove the exact same false statement with the same success probability.

Moving to the proof of the zero-knowledge property, let $\mathcal{A}$ be a real-world adversary controlling a subset of at most $n - 1$ online verifiers. We show how to build a simulator $\mathcal{S}_{\text{DZKP}}^{\mathcal{A}}(\llbracket \vec{x} \rrbracket_2)$ that can simulate the view of $\mathcal{A}$ in $\Pi_{\text{DZKP}}^{\text{off}}$. In the offline phase the adversary is given the corresponding verifier shares of the FLIOP correlation generated by the dealer, according to the set of verifiers it controls. Playing the role of an honest dealer, the simulation of the verifier's share of the FLIOP correlation requires simulating the round challenges and the query answers. The simulator runs the zk-FLIOP simulator to simulate the joint distribution of FLIOP queries and answers. Let the query vector of the j -th round be $\vec{q}_j$ and the answer vector for the j -th round be $\vec{a}_j$. Next the simulator samples the online verifier's share of the proof as a random vector $\vec{\pi}_j$ for the j -th round proof share and sets the dealer's query answer for the j -th round as the vector whose i -th entry is $\vec{a}_{j,i} = \vec{a}_{j,i} - \langle \vec{q}_{j,i}, \llbracket \vec{x}_2 \rrbracket || \vec{\pi}_j \rangle$. The simulator adds these answers to the verifier's FLIOP correlations. In the online phase, the adversary's view is made of the proof shares sent by the prover. We use $\vec{\pi}_j$ for the share in round j . Then, the sum of the verifier's query answer and the dealer's query answer for the i -th query in the j -th round is $\vec{a}_{j,i} + \langle \vec{q}_{j,i}, \llbracket \vec{x}_2 \rrbracket || \vec{\pi}_j \rangle = \hat{a}_{j,i}$ as needed. The simulation is perfect since in both the real and ideal execution the challenges and query are sampled from the same distribution (since the zk-FLIOP is perfectly simulating the view of the FLIOP verifier) and the shares of the proofs are uniformly sampled, last the share of the dealer satisfies the constraint that the sum of the query answers of the dealer and verifiers equals the FLIOP query response, as expected by the FLIOP verifier. ■

5.2 The Online Verification Protocol

In this section we present the verification protocol that the parties execute in the online phase of our main protocol. This protocol is executed right after the semi-honest protocol is completed and before the output is revealed and is therefore intended to verify that no cheating has taken place during the semi-honest computation. This protocol consumes the FLIOP correlations described in Section 5.1. Our verification protocol follows the ideas of Boyle et al. [8] ("BGIN"), which make use of the (offline) dealer as an additional verifier in the DZKP protocol to obtain robustness. Therefore, we begin by providing an overview of their protocol.

The BGIN compiler [8]. Recall that in the semi-honest protocol described in Section 4, the parties see masked values on the input wires to each multiplication gate and on each circuit output wire. Given that a masked value $\hat{x}_w = x_w - r_w$ is public, we can in fact view x_w as being additively shared between each party and the dealer, since $x_w = \hat{x}_w + r_w$ while $\hat{x}_w$ is known to all parties and the random mask r_w is known to the dealer (and shared across the parties). In the BGIN compiler, the parties verify that each input to each multiplication gate as well as each circuit output are consistent with the inputs to the gates in the previous layer. By "consistent" we mean that it is computed correctly, as the sum of the outputs of the multiplication gates that feed it.⁴ The

⁴The BGIN compiler uses arithmetic circuits that are made of addition and multiplication gates only, this is why each input to each multiplication gate is necessarily a *sum* of outputs of previous layer's multiplication gates.

Protocol 18 ($\Pi_{\text{DZKP}}^{\text{off}}$ - DZKP with an offline Dealer).

Parameters:

- 1^σ - statistical security parameter.
 - $C : R^m \rightarrow R$ be an arithmetic circuit over a ring R , and let $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$ be a zk-FLIOP protocol with ρ rounds, ℓ -queries per round, σ -soundness error and message length $u_1, \dots, u_\rho \in \mathbb{N}$ and challenge sets $S_1, \dots, S_\rho$ for the language $\mathcal{L}_C = \{\vec{x} \in R^m \mid C(\vec{x}) = 0\}$.
 - n - the number of online verifiers $V_1, \dots, V_n$.
-

Offline Phase

Input: The dealer's offline input $[\vec{x}]_1 \in R^m$.

- 1: The dealer computes $(C_P, \{C_V^i\}_{i \in [n]}) \leftarrow \text{SampleFLIOP}([\vec{x}]_1)$ with statistical security parameter σ .
 - 2: The dealer hands C_P to the prover P .
 - 3: **for** $i \in [n]$ **do**
 - 4: The dealer hands V_i the FLIOP correlation share C_V^i .
 - 5: $\triangleright$ *If untrusted dealer (i.e. protocol Π_{DZKP}^*), the dealer and parties run protocol $\Pi_{\text{DZKP}}^{\text{NI}}$ to assert the correctness of the FLIOP correlations.* $\triangleleft$
-

Online Phase

Input:

- Each online verifier V_i holds a verifier share of an FLIOP correlation C_V^i with a prover P holding the prover's share of the FLIOP correlation C_P and an offline input $[\vec{x}]_1$. The correlation is provided by the dealer with offline input $[\vec{x}]_1$ to circuit C and FLIOP protocol $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$.
- The online verifiers and the prover hold the online share of the input $[\vec{x}]_2$.

Output: Each verifier outputs **accept** if $C(\vec{x}) = 0$ and **reject** otherwise.

- 1: **for** $j \in [\rho]$ **do**
 - 2: **if** $j = 1$ **then**
 - 3: $\vec{\pi}_j \leftarrow P_{\text{FLIOP}}(\vec{x}, \perp)$ $\triangleright$ Computed by the prover P
 - 4: **else**
 - 5: $\vec{\pi}_j \leftarrow P_{\text{FLIOP}}(\vec{x}, r_1, \dots, r_{j-1})$ $\triangleright$ Computed by the prover P
 - 6: $[\vec{\pi}_j]_2 \leftarrow \vec{\pi}_j - [\vec{\pi}_j]_1$ $\triangleright$ Computed by the prover P , $[\vec{\pi}_j]_1$ is taken from C_P
 - 7: The prover hands $V_1, \dots, V_n$ the proof share $[\vec{\pi}_j]_2$
 - 8: The parties broadcast their authenticated shares of r_j and A_j from their FLIOP correlations.
 - 9: Each party reconstructs r_j and A_j by executing the **Reconstruct** algorithm of the authenticated secret sharing scheme.
 - 10: The prover, using its commitment to the authenticated shares of r_j , verifies the opening of r_j
 - 11: **if** the authenticated opening of r_j or A_j fails **then**
 - 12: **return reject**
 - 13: The verifiers parse A_j as the dealer's query responses in round j : $([a_j^1]_1, \dots, [a_j^\ell]_1) \leftarrow A_j$
 - 14: The verifiers compute $\vec{q}_{j,1}, \dots, \vec{q}_{j,\ell}$ be the query vectors determined by V_{FLIOP} based on $r_1, \dots, r_j$.
 - 15: The verifiers compute the online verifier's query responses in the DZKP:
 $([a_j^1]_2, \dots, [a_j^\ell]_2) \leftarrow \langle \vec{q}_{j,1}, [\vec{x}]_2 \mid [\vec{\pi}_j]_2 \rangle, \dots, \langle \vec{q}_{j,\ell}, [\vec{x}]_2 \mid [\vec{\pi}_j]_2 \rangle$.
 - 16: **for** $i \in [\ell]$ **do**
 - 17: $a_j^i \leftarrow [a_j^i]_1 + [a_j^i]_2$ $\triangleright$ Computed by the verifiers
 - 18: The verifiers run the decision predicate of V_{FLIOP} on all query answers received. If any party received **reject**, it outputs **reject**. Otherwise, it outputs **accept**.
-

equation to verify for each such wire w is

$$\hat{x}_w + r_w = \sum_{g \in G_w} (\hat{x}_1^g + r_1^g) \cdot (\hat{x}_2^g + r_2^g) \quad (2)$$

where G_w is the set of multiplication gates whose outputs feed wire w (i.e., the value on wire w is computed as the sum of outputs of the multiplication gates in G_w). In the BGIN verification protocol, the parties simply embed these equations in a sufficiently large extension field, take a random linear combination of these equations and check their equality to 0. Notice that since Equation 2 is of degree two, the resulting linear computation is also a degree two equation over inputs that are known to *all* parties (e.g. masked wire values) or to the dealer (e.g. random masks). If the dealer was available in the online phase, then as explained in Section 2.4, the online parties and the dealer could jointly execute a DZKP protocol to prove and verify the correctness of these degree-2 equation with sublinear (and even polylogarithmic) communication.

However, the dealer is only present during the offline phase. Consequently, the final step in the BGIN compiler is to move the dealer's actions as an online verifier within such a DZKP execution to the *offline* phase. This can be done by having the dealer providing the parties information that allows them to carry the execution of a DZKP protocol without the presence of the dealer — this is exactly where FLIOP correlations (Definition 15) come into play. Unfortunately, note that since the DZKP challenges are chosen by the dealer in advance as part of the FLIOP correlations, we cannot use the Fiat-Shamir transform and make the DZKP protocol non-interactive.

Limitations of the BGIN compiler. In the BGIN compiler, the dealer also needs to compute, in addition to the FLIOP correlations listed above, a sum of random coefficients (each coefficient corresponds to a single wire) in a way that depends on the circuit's structure. This makes the correlated randomness *circuit-dependent*, requiring the dealer to know the circuit to be computed in advance.

The security of the BGIN protocol also heavily relies on the correct formation of these FLIOP correlations, namely, assuming an honest trusted dealer. Beyond this, if cheating is detected in the protocol, the parties only know that some wire value is incorrect; however, they are not able to identify the party (or dealer) who cheated and caused the error. In the following we show how to address these limitations.

Our protocol. Instead of verifying the correctness of values on the wires we take a different route: We verify the *messages* sent by each party in the semi-honest computation. Intuitively, this will also aid later in cheater identification, since rejecting a proof indicates that the proving party may have cheated and sent an incorrect message during the semi-honest computation.

We now describe the equations that verify the correctness of the messages sent by each party in the semi-honest protocol: Let G_w be the set of multiplication gates that feed wire w . For each $g \in G_w$, let $\hat{x}_1^g, \hat{x}_2^g$ be the two masked inputs to g . Party P_i needs to prove for each wire $w \in W$ that it published the correct share of its masked values, that is:

$$\underbrace{\hat{x}_{w,i} + r_{w,i}}_{P_i \text{'s share of } x_w} - \sum_{g \in G_w} \overbrace{\left(\hat{x}_1^g \cdot \hat{x}_2^g + \hat{x}_1^g \cdot r_{2,i}^g + \hat{x}_2^g \cdot r_{1,i}^g + (r_1^g \cdot r_2^g)_i \right)}^{P_i \text{'s share of the output of gate } g} = 0$$

where $r_{1,i}^g, r_{2,i}^g$ and $(r_1^g \cdot r_2^g)_i$ are P_i 's *shares* of the random triple $(r_1^g, r_2^g, r_1^g \cdot r_2^g)$ respectively (note that only one party actually adds $\hat{x}_1^g \cdot \hat{x}_2^g$; see Footnote 3). Recall that this expression corresponds to the computation that Party P_i is instructed to perform as part of the semi-honest protocol execution (where the $\hat{x}_{w,i}$ values are communicated). To prove the correctness for the entire circuit, we can assign each wire w a random coefficient $\alpha_w \in R_\sigma$ and take a random linear combination of all equations where R_σ is an extension field/ring of R of size $\geq 2^\sigma$ (where σ is a statistical security parameter). By doing so, we embed the equations in the respective extension field/ring R_σ .

Observe that each gate g may feed multiple gates and so may appear several times in the final, combined equation. Let mult denote the set of multiplication gates in the circuit. For each multiplication gate $g_\ell \in \text{mult}$, let W^{g_ℓ} be the set of wires w for which $g_\ell \in G_w$ (i.e., that g_ℓ 's output feeds these wires). Then, let $\gamma_\ell = \sum_{w \in W^{g_\ell}} \alpha_w$. By reordering terms, we have the final equation that

P_i is required to prove is thus

$$\sum_{w \in W} \alpha_w (\hat{x}_{w,i} + r_{w,i}) - \sum_{g_\ell \in \text{mult}} \gamma_\ell \left(\hat{x}_1^{g_\ell} \cdot \hat{x}_2^{g_\ell} + \hat{x}_1^{g_\ell} \cdot r_{2,i}^{g_\ell} + \hat{x}_2^{g_\ell} \cdot r_{1,i}^{g_\ell} + (r_1^{g_\ell} \cdot r_2^{g_\ell})_i \right) = 0 \quad (3)$$

Now, setting

$$T^i = \sum_{w \in W} \alpha_w \cdot \hat{x}_{w,i} - \sum_{g_\ell \in \text{mult}} \gamma_\ell \cdot (\hat{x}_1^{g_\ell} \cdot \hat{x}_2^{g_\ell}) \quad (4)$$

$$\vec{Q}^i = ((-\alpha_w)_{w \in W}, (\gamma_\ell \cdot \hat{x}_1^{g_\ell}, \gamma_\ell \cdot \hat{x}_2^{g_\ell}, \gamma_\ell)_{g_\ell \in \text{mult}}) \quad (5)$$

and

$$\vec{V}^i = ((r_{w,i})_{w \in W}, (r_{2,i}^{g_\ell}, r_{1,i}^{g_\ell}, (r_1^{g_\ell} \cdot r_2^{g_\ell})_i)_{g_\ell \in \text{mult}}) \quad (6)$$

we have that Eq. (3) can be written simply as $T^i - \vec{Q}^i \cdot \vec{V}^i = 0$ (where $\cdot$ denotes inner product). That is, to verify the correctness of the messages sent by party P_i in the semi-honest protocol, we need to verify that $T^i - \vec{Q}^i \cdot \vec{V}^i = 0$ which can be done using a degree-two circuit $C_m(T^i, \vec{Q}^i, \vec{V}^i)$ with $1 + 2m$ inputs where m is the number of wires in the circuit that are either input or output wires of a multiplication gate or output wires of the circuit (this is because the length of both $\vec{Q}^i$ and $\vec{V}^i$ is m , each). Note that T^i and $\vec{Q}^i$ are known by all online parties, whereas $\vec{V}^i$ is known to the dealer and party P_i . In the rest of this paper we will refer to the circuit $C_m^\mathcal{R}(z, \vec{x}, \vec{y}) = z - \langle \vec{x}, \vec{y} \rangle$ as the m -sized *verification circuit over ring $\mathcal{R}$* where $\vec{x}, \vec{y} \in \mathcal{R}^m$ and $z \in \mathcal{R}$. When $\mathcal{R}$ is clear from the context, we will simply refer to it as the m -sized *verification circuit* and denote it by C_m .

In our verification protocol, the parties participate in n instances of Protocol 18 with an offline dealer and $n - 1$ online verifiers. In the i -th instance, party P_i acts as the prover in the DZKP protocol (Protocol 18), the dealer acts as an offline verifier and the remaining $n - 1$ parties act as the online verifiers.

In each instance of Protocol 18 the parties shall consume a single FLIOP correlation. These correlations consists of dealer's share of the query answers, which it can compute in advance during the offline phase. Notice that the dealer's offline input in this DZKP is $\vec{V}_i$ from Eq. (6) which does not depend on the circuit's structure but only on its size, thereby achieving the desired circuit-independence property. In protocol 19 we provide a formal description of our protocol followed by a full security analysis.

We now proceed to the security proof of the verification protocol.

Protocol 19 (Π_{vrfy} - verification with a dealer via DZKP).

Parameters:

- Number of online parties n denoted $P_1, \dots, P_n$.
 - A statistical and computational security parameters $1^\sigma, 1^\kappa$ and FLIOP soundness parameter ε .
 - Upper bound m on the number of wires that are either connected to a multiplication gate or are output wires of the circuit to be verified in the online phase.
 - A field $\mathbb{F}$ and an extension field $\mathbb{F}_\sigma$ of $\mathbb{F}$ with $|\mathbb{F}_\sigma| \geq 2^\sigma + \varepsilon$.
 - A $\rho = \rho(m)$ -round ε -sound FLIOP protocol $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$ over a field $\mathbb{F}_\sigma$ with ℓ queries per round and message length $u_1, \dots, u_\rho \in \mathbb{N}$ and challenges sets $\mathbb{F}_\sigma$ for the circuit C_m .
-

Offline Phase

Input: The dealer $\mathcal{D}$ holds for each $i \in [n]$ the masks $r_{w,i}$ for output wires and the shares $r_{1,i}^g, r_{2,i}^g, (r_1^g \cdot r_2^g)_i$ for each multiplication gate g .

- 1: **for** $i \in [n]$ **do** $\triangleright$ This loop is computed by the dealer $\mathcal{D}$
 - 2: Computes $\vec{V}^i$ as in Eq. (6).
 - 3: Computes its offline input $\vec{y}_i^{\mathcal{D}} = (V^i[1], 0, \dots, V^i[m], 0)$
 - 4: Executes the offline phase of the $\Pi_{\text{DZKP}}^{\text{off}}$ protocol with offline input $\vec{y}_i^{\mathcal{D}}$ parameterized by $n-1$ online verifiers, circuit C_m with input size $1 + 2m$ over field $\mathbb{F}_\sigma$ and FLIOP protocol $(P_{\text{FLIOP}}, V_{\text{FLIOP}})$.
 - 5: $\triangleright$ If untrusted dealer (i.e. protocol Π_{vrfy}^*), the above step instead calls to Π_{DZKP}^* . If this call returns reject, return reject. $\triangleleft$
-

Online Phase

Input:

- A circuit C over finite field $\mathbb{F}$ with m wires connected either to a multiplication gate or are output wires of the circuit.
 - Each party P_i ($i \in [n]$) in the online phase holds $\{\hat{x}_{w,j}\}_{j=1}^n \in \mathbb{F}$ for each wire w which is either an input to a multiplication gate or a circuit's output wire.
 - Each party P_i holds a mask $r_{w,i}$ for each circuit's output wire w and shares of a random multiplication triple $r_{1,i}^g, r_{2,i}^g, (r_1^g \cdot r_2^g)_i$ for each multiplication gate g with two input wires.
 - Each party P_i holds a share of the FLIOP correlations $(\mathbf{P}^i, \{\mathbf{V}^{j,i}\}_{j \neq i})$ obtained in the offline phase.
- 1: The parties jointly call $\mathcal{F}_{\text{coin}}$ to obtain $\alpha \in \mathbb{F}_\sigma$, from which they derive α_w for each $w \in W$ (this can be done by setting $\alpha_w = \alpha^w$).
 - 2: Each party P_i computes $T^i, \vec{Q}^i$ (defined in Eq. (4) and (5)) where γ_ℓ coefficients are computed based on the input circuit C .
 - 3: **for** $i \in [n]$ **do** $\triangleright$ Execute all iterations in parallel
 - 4: Party P_i computes the DZKP full input $\vec{y}_i = (V^i[1], Q^i[1], \dots, V^i[m], Q^i[m], T^i)$
 - 5: The parties P_j for $j \neq i$ compute $\vec{Q}^i$ as in Eq. (5).
 - 6: The parties P_j for $j \neq i$ compute the DZKP online input $\vec{y}_i^V = (0, Q^i[1], \dots, 0, Q^i[m], T^i)$
 - 7: The parties execute the online phase of the i -th instance of $\Pi_{\text{DZKP}}^{\text{off}}$. P_i acts as the prover with input $\vec{y}_i$, while P_j for $j \neq i$ act as online verifiers with input $\vec{y}_i^V$ to verify the degree-2 equation $C_m(\vec{y}_i) := T^i - \vec{Q}^i \cdot \vec{V}^i = 0$
 - 8: **if** the protocol outputs reject **then**
 - 9: **return** reject
 - 10: **return** accept
-

Correctness. The correctness requirement states that if Eq. (3) holds for each $i \in [n]$, then an honest execution of the verification protocol will succeed. Therefore, assuming that the equation holds and that the parties act honestly, in the i -th execution of Protocol 18 the verification protocol will succeed and since this holds for all $i \in [n]$, all parties will output **accept**.

Soundness. We next compute the probability that a corrupted party P_i successfully cheats. That is, assume that Eq. (3) does not hold for i and that P_i is corrupted. Assume that P_i injected errors when sending $\hat{x}_{w,i}$ for some wire w during the semi-honest protocol (this is the only place it can cheat). Then, there are two events where the parties may output **accept**:

- The random coefficients α_w cause Eq. (3) to be equal to 0, by canceling all the errors added by P_i .
- The verifiers output **accept** even though the statement to be proven is incorrect.

The probability for the first event depends on the way α_w is generated. Assume that $\alpha_w = \alpha^w$ for some random α . Then, Eq. (3) is a polynomial of degree W over a field with at least 2^σ elements. Hence, the probability of equality to 0 is the probability of α being a root of this polynomial. Over finite fields, the probability is $\frac{W}{|\mathbb{F}_\sigma|}$ which is negligible in σ . The probability of the second event is simply the soundness error of the DZKP which equals the soundness of the zk-FLIOP. For a zk-FLIOP with soundness error ε , we have that the overall cheating probability is bounded by $\frac{W}{2^\sigma} + \varepsilon$.

Remark 20 (Extending Π_{vrfy} to rings). *Although Π_{vrfy} is presented over finite fields it can be extended to work over $\mathbb{Z}_{2^k}$ rings. If the statement is defined over $\mathbb{Z}_{2^k}$, then the verification protocol can be executed over an extension ring as shown in [7]. When the degree extension is d , it can be shown that the success cheating probability is bounded by $\frac{W}{2^d} + \varepsilon$.*

Security. To argue security, we prove that a simulator that knows the corrupted party's inputs, can generate a transcript of Π_{vrfy} that is statistically close to a real execution of Π_{vrfy} (Protocol 19), i.e., the simulator's output looks the same as the adversary's view and the honest parties' output in a real execution.

Proposition 21. *Let ϵ be the soundness error of Π_{vrfy} (Protocol 19). Then, for every adversary $\mathcal{A}$ controlling any subset of parties with set of indices $T \subset [n]$ where $|T| \leq n - 1$, there exists a simulator $\mathcal{S}_{\text{vrfy}}$ such that*

$$SD \left(\mathcal{S}_{\text{vrfy}} \left(\begin{array}{c} \{\hat{x}_{w,k}, \hat{x}_w, r_{w,k}\}_{w \in W, k \in [T]} \\ \{(r_1^{g_\ell} \cdot r_2^{g_\ell})_k\}_{g_\ell \in \text{mult}, k \in [T]} \end{array} \right), \left\{ \text{view}_{\mathcal{A}}^{\Pi_{\text{vrfy}}}, \text{out}_{\text{Honest}}^{\Pi_{\text{vrfy}}} \right\} \right) \leq \epsilon$$

where $SD(X, Y)$ is the statistical distance of X from Y .

Proof: We present a simulator $\mathcal{S}_{\text{vrfy}}$ that plays the role of the honest online parties, the dealer $\mathcal{D}$ and the ideal functionality $\mathcal{F}_{\text{coin}}$, while interacting with the real-world adversary $\mathcal{A}$. As our protocol essentially consists of n instances of the DZKP protocol $\Pi_{\text{DZKP}}^{\text{off}}$, the simulation of the offline phase incorporates the execution of n instances of $\mathcal{S}_{\text{DZKP}}^{\text{off}}$, receiving from them the simulated view of the malicious parties in the offline phase of the n executions of $\Pi_{\text{DZKP}}^{\text{off}}$. In the online phase, playing the role of $\mathcal{F}_{\text{coin}}$, the simulator $\mathcal{S}_{\text{vrfy}}$ first hands a randomly and uniformly chosen $\alpha \in \mathbb{F}_\sigma$ to $\mathcal{A}$.

This allows $\mathcal{S}_{\text{vrfy}}$ to compute T^i and $\tilde{Q}^i$ for each party P_i and $\tilde{V}^k$ for each corrupted party P_k . This allows the simulator to compute the inputs to the online phases of the DZKP protocols $\Pi_{\text{DZKP}}^{\text{off}}$ with which it can execute the online phase of the n executions of the simulator $\mathcal{S}_{\text{DZKP}}^{\text{off}}$. The statistical distance arises from the fact that a malicious prover might succeed in convincing honest verifiers in a false statement (i.e. that a dishonest semi-honest protocol was carried honestly) with negligible probability — this is exactly ϵ the soundness error of Π_{vrfy} . In this case the simulator of $\mathcal{S}_{\text{DZKP}}^{\text{off}}$ will fail to generate a correct simulation of the view of the malicious parties — while the simulator aborts in the ideal world, the protocol terminates with acceptance in the real world.

■

5.3 Proving Correlated Randomness is Valid

In this section, we show how the online parties can verify the correctness of the correlated randomness received from the offline dealer. We split this into two parts: correlated randomness for the semi-honest execution, which consists mainly of random multiplication triples, and correlated randomness for the verification protocol, which contains mainly the dealer’s FLIOP correlations.

5.3.1 Verifying Semi-Honest Correlations

The correlated randomness for the semi-honest execution of the circuit C over $\mathcal{R}$ contains: (1) For each multiplication gate $g \in \text{mult}$: a triple $(\llbracket r_1^g \rrbracket, \llbracket r_2^g \rrbracket, \llbracket r_1^g \cdot r_2^g \rrbracket) \in \mathcal{R}^3$; (2) For each circuit’s output wire w : an authenticated additive sharing $\langle\langle r_w \rangle\rangle$. Note that the authenticated sharing of the output masks are required for correctly opening the output (although the authentication is needed to protect against malicious behavior, we count it as part of the semi-honest correlations for simplicity).

Compressing semi-honest correlations via PCGs. We can use *pseudorandom correlation generators* (PCGs) [4] from Section 2.2 to generate the semi-honest correlated randomness. Here each party receives a short seed from which it locally derives its share of the desired correlation—in our case, random multiplication triples over some ring or field $\mathcal{R}$. In particular, for secure 2-party computation of Boolean circuits, each triple can be locally generated using 2 random OTs (oblivious transfer), where the latter can be efficiently compressed using fast PCGs for OT [4, 64, 65, 66]. We remark that concretely efficient constructions for PCGs do not exist for *authenticated* triples in the multi-party setting. Thus, one could not directly compress the SPDZ correlation concretely efficiently using PCGs.

Proving correctness of semi-honest correlations. The dealer also sends a proof that the multiplication triples are correct. By Claim 2.1, there exists a zk-FLIOP with proof and query complexity of logarithmic size (in the number of triples). The parties can thus run Protocol 9 (non-interactive DZKP) to verify the correctness of the triples with communication of polylogarithmic size in the number of multiplication triples. When compressing the triples with PCGs, the dealer expands the seeds to compute the proof, and then sends the seeds together with the proof. The verifiers expand the seeds to generate the shares of the triples and then verify correctness. If the proof is accepted, then the parties can discard the triples and keep only the compressed PCG seeds for future online protocols. (This expansion and verification can also be performed on blocks of

triples at a time, as opposed to all at once. This would indeed be the structure for preprocessing for life.)

To verify $\langle\langle r_w \rangle\rangle$ for each output wire w , recall that each party holds a commitment for its share and for the shares of other parties. Thus, each party simply verifies the correctness of the commitment for its own share and sends a complaint in case the commitment is not valid. If the dealer sent invalid commitments, then in the corruption model all online parties are honest, in which case cheating will be detected.

We therefore consider a modified version of the semi-honest protocol (Protocol 12) denoted by Π_{SH}^* that works exactly like Π_{SH} except that the offline phase has the following two steps added:

1. After handing the PCG seeds to the parties, the dealer expands the seeds and generates a non-interactive proof for the correctness of the multiplication triples and hands each online party P_i its share of the proof π_i^{SH} .
2. After receiving the correlated randomness and the shares of the proof from the dealer, the parties run the non-interactive DZKP protocol to verify the correctness of the multiplication triples.

These steps are added to the description of Π_{SH}^* in Step 7 of the offline phase of Protocol 12.

This extension of providing security against a malicious dealer in the semi-honest protocol and will be used as a building block in our full protocol presented in Section 5.4.

5.3.2 Verifying FLIOP Correlations

In this section we proceed to the more challenging task: proving that the FLIOP correlations sent by the dealer are correct. Our protocol in this section uses an *offline* non-interactive DZKP to verify the generated FLIOP correlations, sampled using the **SampleFLIOP** algorithm. The dealer, who generated the FLIOP correlations, will act as the non-interactive DZKP prover, while the remaining parties will act as the verifiers, thereby ensuring that the FLIOP correlations are correct. The idea is to closely follow the dealer's computation for generating the FLIOP correlations and pointing out the exact equations the need to be proven correct from the FLIOP correlations. Then, we can batch these equations into a single equation by taking a random linear combination of them, similar to the approach taken for batching the equations to verify all multiplication triples in Section 5.3.1. While in the $2+1$ setting, surprisingly, the depth of the verification circuit is as low as 2, in the multi-party setting, the depth of the verification circuit depends on the underlying FLIOP and is logarithmic in the *online* circuit size in our instantiation of the FLIOP of [7]. Therefore, the only remaining point to be taken care of is the depth of the circuit which directly affects the concrete efficiency of the proof generating and the verification of the non-interactive DZKP. We show two alternative methods to structuring the FLIOP correlations that reduce the depth of the circuit and discuss their trade-offs.

Verifying FLIOP Correlations in the $(2+1)$ Setting. We begin by showing how to verify the FLIOP correlations in the $2+1$ setting, where the dealer generates the FLIOP correlations for a single verifier. Let us first recall the computation carried by the dealer in order to generate the FLIOP correlations, as presented in Algorithm 1. Working in the $2+1$ setting we assume (without loss of generality) that P_1 is the party that is given the prover's share of the correlation (C_P) and P_2 being the party that is given the verifier's share of the correlation (C_V). At the beginning of

the protocol the dealer holds its offline input $\vec{x}^D$. For each round $j \in [\rho]$ of the (online) DZKP, the dealer samples the round challenge r_j from which it computes the round queries $\{\vec{q}_{j,i}\}_{i \in [\ell]}$ (where ℓ is the number of queries carried by the underlying zk-FLIOP protocol), next the dealer samples its share of the round proof $\vec{\pi}_j^D$ and computes its share of the zk-FLIOP query answers $\{a_{j,i}^D\}_{i \in [\ell]}$ where:

$$a_{j,i}^D = \langle \vec{q}_{j,i}, \vec{x}^D || \vec{\pi}_j^D \rangle \quad (7)$$

Now, the correlation is generated by handing the prover P_1 the proof masks $\vec{\pi}_j$ and the verifier P_2 receives the round challenge r_j and the query answers $\{a_{j,i}^D\}_{i \in [\ell]}$.

Hence, the dealer needs to prove that $a_{j,k}^D$ shared to the parties P_1 and P_2 was computed correctly, given $\vec{x}^D$, $\vec{\pi}_j^D$ and $\vec{q}_{j,k}$ (namely, $r_1, \dots, r_j$). Therefore, towards verifying the FLIOP correlated randomness by leveraging DZKPs, we consider the following degree-two circuit:

$$C_{\text{FLIOP}}^D(a_{j,k}^D, \vec{x}^D, \vec{\pi}_j^D, \vec{q}_{j,k}) := a_{j,k}^D - \langle \vec{q}_{j,k}, (\vec{x}^D || \vec{\pi}_j^D) \rangle$$

We observe that $C_{\text{FLIOP}}^D(a_{j,k}^D, \vec{x}^D, \vec{\pi}_j^D) = 0$ if and only if the inputs to the circuit satisfy Eq. 7. Moreover, if the circuit's inputs (i.e. $a_{j,k}^D, \vec{x}^D, \vec{\pi}_j^D, \vec{q}_{j,k}$) are linearly shared across the parties, then this would introduce an efficient DZKP-based verification protocol immediately. First, the query answer $a_{j,k}^D$ is indeed shared across the parties by the dealer, in fact, it is known in its entirety to P_2 . Second, note that $\vec{x}^D$ and $\vec{\pi}_j^D$ are known to one of the parties, namely, the party P_1 that acts as a prover. If $\vec{q}_{j,k}$ were shared between the parties as well, C_{FLIOP}^D was indeed a degree-two circuit with secret shared inputs. However, in the $(2+1)$ setting, party P_2 is given only the round challenges. Luckily, we overcome this hurdle by observing that P_2 can locally compute the query vectors $\vec{q}_{j,k}$ from the round challenges $r_1, \dots, r_j$, without interaction. Therefore, the prover can generate a non-interactive DZKP (Protocol 9) to prove the correctness of the dealer's correlations using the circuit C_{FLIOP}^D . This proof is concretely efficient to realize: It is a degree two circuit in its inputs that are shared across the parties P_1 and P_2 and the proof size is logarithmic in the size of the circuit to be computed in the online phase.

Remark 22 (Batching Proofs). *So far, the dealer needs to generate a single proof for each round of the FLIOP protocol per FLIOP correlation which means that the size of all proofs together scale linearly in the number of parties. Instead, we recall that all these proofs can be batched into a single DZKP by taking a random linear combination of all degree-two equations (Eq. 7, one per round per FLIOP correlation) to be proven correct. This allows the dealer to generate a single DZKP for all FLIOP correlations, which is as concretely efficient to verify*

Verifying FLIOP Correlations in the $(n+1)$ Setting. Following the same approach as in the $(2+1)$ setting, we now consider the case where the dealer generates the FLIOP correlations for n verifiers. According to the **SampleFLIOP** algorithm, while in the $(2+1)$ setting the query answers $a_{j,k}^D$ and round challenges r_j were given in their entirety to the single DZKP online verifier, now they are linearly shared across the online verifiers in an authenticated manner. To employ the same approach as in the $(2+1)$ setting, we need to ensure that all inputs to the verification circuit C_{FLIOP}^D are linearly shared across the verifiers. Since linear sharing of circuit inputs is enough, the (authenticated) linear sharing of the query answers is not an issue and the DZKP protocol is still compatible with this sharing. The more challenging part is the sharing of the round challenges

Approach	Online Verification			Offline		
	Prover Time	Verifiers Time	Communication	Dealer Time	Parties Time	Communication
Naive	$O(m)$	$O(n \cdot m)$	$O(n^2 \log m) + O(n^3)$	$\tilde{O}(nm \log m)$	$O(n \cdot m)$	$O(n \log^2 m) + O(n^2 \log m)$
Pairwise Correlations	$O(n \cdot m)$	$O(n \cdot m)$	$O(n^2 \log m)$	$\tilde{O}(n^2 \cdot m)$	$O(n \cdot m)$	$O(n^2 \log m)$
Multiplicative Sharing	$O(m)$	$O(n \cdot m)$	$O(n^2 \log m) + O(n^3)$	$\tilde{O}(n^2 \cdot m)$	$O(n \cdot m)$	$O(n^2 \log m)$

Table 1: Comparison of the costs associated with the verification protocol between the three different approaches for proving the zk-FLIOP correlations by the offline dealer to n online parties with the underlying FLIOP instantiated based on [7]. These costs include the cost of running the online DZKP protocols for verifying a circuit with m multiplication gates by n parties, and the offline cost of generating the correlations by the dealer, distributing them and proving their correctness.

r_j . In the $(2 + 1)$ setting, the single DZKP online verifier could have computed the query vectors $q_{j,k}$ from the round challenges $r_1, \dots, r_j$ without interaction. However, in the $(n + 1)$ setting, the verifiers do not open the round challenges r_j until the online phase and therefore cannot compute the round queries $q_{j,k}$.

Before diving into our solution, we introduce some notation: For each FLIOP round $j \in [\rho]$ and each round query $k \in [\ell]$ we denote by $c_{j,k}(r_1, \dots, r_j)$ the function that takes round challenges $r_1, \dots, r_j$ and computes the k -th query of the j -th FLIOP round $q_{j,k}$. We denote the multiplicative depth of $c_{j,k}$ by $d_{j,k}$. For simplicity we make the natural assumption that all $d_{j,k}$ are bounded by some depth d .

Based on the above notation, we present three approaches for generating and verifying the FLIOP correlations, and in particular, sharing the round challenges r_j and discuss their tradeoffs. In our description we also consider concretely the FLIOP instantiation of [7] (see Appendix B) where $\ell = 1, \rho = \log m + 1$ and $d = \rho + 1$. Based on this instantiation we summarize the costs of our three approaches in Table 1. All approaches rely on the dealer of the FLIOP correlations generating a non-interactive DZKP for the correctness of the FLIOP correlations $\tilde{\pi}$ and sharing it across the verifiers, while the online parties verifying this proof in the offline phase. Since FLIOP correlations are generated in the offline phase of $\Pi_{\text{DZKP}}^{\text{off}}$ we consider a protocol Π_{DZKP}^* that is similar to $\Pi_{\text{DZKP}}^{\text{off}}$ but after sending the FLIOP correlations the dealer generates a non-interactive DZKP for the correctness of the FLIOP correlations and sends it to the online parties and the online parties verify this proof in the offline phase. This extra step is shown in Step 5 of Π_{DZKP}^* in Protocol 18. We also consider an altered version of the verification protocol Π_{vrfy} that invokes the protocol Π_{DZKP}^* to verify the FLIOP correlations instead of $\Pi_{\text{DZKP}}^{\text{off}}$ and denote it by Π_{vrfy}^* . This modified step is shown in Step 5 of Π_{vrfy}^* in Protocol 19.

We now present the three approaches for proving and verifying the correctness of the FLIOP correlations.

Solution 1: Using a Deeper Verification Circuit. Rewriting the verification equation of Eq. 7 in terms of the round challenges $r_1, \dots, r_j$ we need to verify the equivalent following equation

in which we simply replace $\vec{q}_{j,k}$ with $c_{j,k}(r_1, \dots, r_j)$:

$$a_{j,i}^{\mathcal{D}} = \langle c_{j,i}(r_1, \dots, r_j), \vec{x}^{\mathcal{D}} || \vec{\pi}_j^{\mathcal{D}} \rangle \quad (8)$$

This equation is of depth $d_{j,k} + 1 \leq d + 1$ and therefore the corresponding verification circuit $C_{\text{FLIOP}}^{\mathcal{D}}$ is of depth $\leq d + 1$:

$$C_{\text{FLIOP}}^{\mathcal{D},j,k}(a_{j,k}^{\mathcal{D}}, \vec{x}^{\mathcal{D}}, \vec{\pi}_j^{\mathcal{D}}, r_1, \dots, r_j) := a_{j,k}^{\mathcal{D}} - \langle c_{j,i}(r_1, \dots, r_j), (\vec{x}^{\mathcal{D}} || \vec{\pi}_j^{\mathcal{D}}) \rangle$$

We notice that now all circuit inputs are linearly shared across the online parties, which introduces a non-interactive DZKP for the language of degree $\leq d + 1$ which is logarithmic in the online circuit size when the FLIOP protocol is instantiated from [7].

Solution 2: Using Pairwise FLIOP Correlations. An alternative approach would be for the dealer to generate a *single verifier* FLIOP correlation for each pair of online parties. That is, for each pair of parties P_i, P_j the dealer generates the FLIOP correlation as if P_i is the prover and P_j is the verifier for asserting the honest participation of P_i in the semi-honest protocol exactly in the same way described at the beginning of this section for the $(2 + 1)$ setting. By taking this approach we benefit from the fact that now the online verifier in each FLIOP correlation holds the round challenges r_j (rather than just shares of them) and can compute the round queries $q_{j,k}$ locally and therefore invoke the efficient solution for the $(2 + 1)$ setting also in the $(n + 1)$ setting. This approach, however, incurs significant computational costs: First, the dealer needs to generate $\Theta(n^2)$ FLIOP correlations and the proofs of correctness thereof, rather than just $O(n)$ in the previous approach. Moreover, in the online phase each party needs to generate $\Theta(n)$ DZKP proofs: one for each other online party, rather than just a single proof as in the original approach. This approach does benefit, however, from faster computation per proof, as the verification circuit for the FLIOP correlation is of depth 2 rather than logarithmic in the online circuit size. This is why expect this approach to be more efficient for a small number of parties.

Solution 3: Using Multiplicative Sharing. A main limitation of the first approach is that in the $(n + 1)$ setting for $n > 2$ the verification circuit's depth is logarithmic in the circuit size even for a constant, small number of parties which might be costly. The second approach might also turn out to be costly for a relatively small number of parties: Even as little as $n = 11$ parties require generating as many as 110 FLIOP correlations, which is 10× higher than the number of correlations generated in the first approach.

Therefore we consider a third approach — using multiplicative sharing. While the round challenges themselves will be shared additively like before, some linear function of the round challenges, as shown in the following, will be shared multiplicatively. In this approach for n online parties the verification circuit is of depth $\log n + 2$ rather than $\log m + 1$, which is a significant improvement in common settings where circuits are much larger than the number of parties.

First, due to this unique sharing of the round challenges, this approach is limited for secure computation over fields (rather than arbitrary rings). Second, this approach might not be suitable for all FLIOP instantiations but we have found it to be suitable for the FLIOP of [7]. With that being said, this approach can provide the best of both previous approaches, in the appropriate settings. To keep this explanation simple we focus on the protocol of [7] and provide a short background on the necessary part and refer the reader to Appendix B and to [7] for more details.

In short, their protocol has $\rho = \log m + 1$ rounds and each round proof is of constant size and a single query each round that is applied *only* to that round proof (we call these ρ queries - “proof queries”). In the last round, and in the last round only, there is another single query that is applied to the full input and the proof (we call this query - “final query”). In the following we further explain on how each of these two types of queries is computed from the round challenges and how the parties can leverage multiplicative sharing of the round proofs to verify the FLIOP correlations.

Query Type 1: Proof Queries Since proof queries are made only to the proof which are of constant size, the dealer can provide authenticated shares of these queries explicitly which are opened in the online phase. The only remaining task is to verify that the shared queries were computed correctly from the round challenge. In the protocol of [7] the “proof query” of the i -th round is a degree 2 function of that round challenge r_i except for the last proof query which is a degree 4 function of the round challenge r_ρ . Since the n parties hold additive shares of r_i the verification circuit can additively reconstruct the round challenge and then ensure that the query was computed correctly using a degree ≤ 4 circuit. Since each round proof is of depth ≤ 4 and there are $\log m + 1$ such proofs, they can be batched into a single proof using the same technique as in Remark 22. Since the full statement size in this case is $O(\log m)$ the size of the proof is $O(\log \log m)$. This leads us to the following corollary:

Corollary 23. *Verifying the proof queries can be done using non-interactive DZKP for a degree 4 statement.*

Query Type 2: Final Query This is the more challenging part as the final query is computed from the round challenges $r_1, \dots, r_\rho$. Towards computing the final query, the dealer considers three functions $L_1(x), L_2(x), L_3(x)$ ⁵ of the round challenges $r_1, \dots, r_\rho$. Concretely, the dealer computes for each round challenge r_i the following two values: $L_1(r_i), L_2(r_i)$. For the last round challenge r_ρ the dealer computes also $L_3(r_\rho)$. In a nutshell, this final query is of size $3m$ and each entry in the query is computed picking for each round challenge r_j one of $L_1(r_j), L_2(r_j)$ (and for the last round also $L_3(r_j)$) and multiplying them together.

Slightly more formally, for each round $i \in [\rho]$ the dealer computes the set $S_i = \{L_1(r_i), L_2(r_i)\}$ and for the last round also $S_\rho = \{L_1(r_\rho), L_2(r_\rho), L_3(r_\rho)\}$. The final query is then computed by taking all the elements from the product set $S_1 \times S_2 \times \dots \times S_\rho$ which is exactly of size $3m$ as needed. Which leads us to the following corollary:

Corollary 24. *Let d_L be an upper bound on the degree of L_1, L_2 and L_3 , then there exists a circuit $C_{\text{final}}(r_1, \dots, r_\rho)$ that computes the final query of [7] and is of degree $< 3(d_L - 1) + \rho = \log m + 3d_L - 2$*

In [7] the degree of L_1, L_2, L_3 is ≤ 2 and therefore the degree of C_{final} is from Corollary 24 is $< \log m + 4$ and in any case $\geq \log m + 1$ which might result in concrete computational bottlenecks.

Our solution is to have the dealer share not only the round challenges $r_1, \dots, r_\rho$ but also multiplicatively share the values $L_1(r_i), L_2(r_i)$ for each round $i \in [\rho]$ and $L_3(r_\rho)$ for the last round. This way, each party can compute for each entry in the final query a multiplicative share of that query. Then, a circuit that only takes the multiplicative shares of the n parties and reconstructs the final query, which is a degree n circuit can be used to verify the final query according to Equation 8, resulting in a degree $n + 1$ verification circuit, rather than a degree $\geq \log m + 1$ circuit.

⁵In [7] these functions are simply the computation of a Lagrange coefficient at a specific point for a degree 2 polynomial, except for the last round where the Lagrange coefficient is computed for a degree 3 polynomial.

One last subtlety that needs to be taken care of is that the dealer needs to multiplicatively share the values $L_1(r_i)$, $L_2(r_i)$ and $L_3(r_\rho)$ and in fact might cheat when multiplicatively sharing them. We utilize the fact that the degree of L_1, L_2 and L_3 to the parties to verify that the product of their multiplicative shares (a degree n computation) equals the correct computation on the round challenges $r_1, \dots, r_\rho$. More formally, let $j \in [\rho]$ be a round and let r_j be the j -th round challenge. Let $i \in \{1, 2, 3\}$ and let $t_{j,i,k}$ be the multiplicative sharing of $L_i(r_j)$ handed to party P_k . The parties need to verify, therefore, that the following equation holds for all i, j :

$$\prod_{k \in [n]} t_{j,i,k} = L_i(r_j) \quad (9)$$

Since parties hold $t_{j,i,k}$ and $\llbracket r_j \rrbracket$, let $C_{i,j}^{\text{mult}}(t_{i,j,1}, \dots, t_{i,j,n}, r_j) = \prod_{k \in [n]} t_{j,i,k} - L_i(r_j)$ be a degree n circuit.⁶ Verifying the correctness of Equation 9 can be done by running a non-interactive DZKP for the degree n language using the circuit $C_{i,j}^{\text{mult}}$. While there are $2\rho + 1 = 2\log m + 3$ such equations, they can be batched into a single proof using the same technique as in Remark 22 and with negligible statistical soundness error.

5.4 Description of the Full Protocol

We are now ready to describe the full protocol for computing arithmetic circuits. The protocol $\Pi = (\Pi_{\text{off}}, \Pi_{\text{on}})$ is given in Protocol 25.

We now proceed to the proof of security. We note that the protocol has computational security only because we use PCGs to compress the semi-honest triples, and must restrict the number of random oracle calls made by the adversary for the Fiat-Shamir heuristic. If we give up on the dealer's sub-linearity requirement, and let it distribute triples instead of PCG seeds, the protocol becomes statistically secure assuming a polynomial bound on random oracle queries.

Theorem 26. *Let R be a finite field or the ring $\mathbb{Z}_{2^k}$ and let $\mathcal{F}$ be an $(n+1)$ -functionality over R . Then, our protocol $\Pi = (\Pi_{\text{off}}, \Pi_{\text{on}})$ securely realizes $\mathcal{F}$ with abort in the offline-online $(\mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{RO}})$ -hybrid model and with statistical error in the presence of computationally-bounded malicious adversaries controlling either P_0 or any subset of parties $\{P_k\}_{k \in T}$ where $T \subset [n]$ and $|T| \leq n - 1$.*

Proof: Let $\mathcal{A}$ be the real-world adversary; we construct an ideal-world simulator $\mathcal{S}$. In the simulation, $\mathcal{S}$ interacts with $\mathcal{A}$ by playing the role of the honest parties and the ideal functionalities $\mathcal{F}_{\text{coin}}$ and $\mathcal{F}_{\text{RO}}$. We distinguish between two cases:

CASE 1: DEALER IS MALICIOUS. In this case, $\mathcal{S}$ only receives one message from $\mathcal{A}$ and then does not interact with $\mathcal{A}$ at all. (Namely, in the real world, the parties do not send any messages back to the dealer.) Hence, $\mathcal{S}$ only needs to simulate the output of the honest parties. Note that since it plays the role of the n verifiers, it learns the entire statement and knows whether the correlated randomness is correct or not. If the correlated randomness is incorrect, it sends **abort** to the trusted party computing $\mathcal{F}$. Otherwise, it sends **continue**, allowing the honest parties in the ideal world to learn their output.

Note that the only difference between the simulation and a real execution in this case, is the event that can occur in a real execution, where honest parties accept incorrect correlated randomness.

⁶Technically it is of degree $\leq \max(n, d_{L_i})$ where d_{L_i} is an upper bound on the degree of L_i but since in [7] $d_{L_i} \leq 2$ and $n \geq 2$ we can safely assume that $d_{L_i} \leq n$

Protocol 25 ($\Pi = (\Pi_{\text{off}}, \Pi_{\text{on}})$) - Secure n -Party Computation of an Arithmetic Circuit).

Parameters:

- $1^\sigma, 1^\kappa$ - statistical and computational security parameters.
 - n - the number of online parties $P_1, \dots, P_n$.
 - m, i, o - upper bounds on the number of multiplication gates, input wires and output wires of the circuit to be computed in the online phase.
-

Π_{off} (Offline Phase)

- 1: The dealer and parties $P_1, \dots, P_n$ invoke the offline phase of the semi-honest protocol Π_{SH}^* for a size m circuit with i input wires and o output wires. Let SH_i be the correlated randomness for party P_i
 - 2: $\triangleright$ *During the instances above of Π_{SH}^* , the dealer also proves the validity of the semi-honest correlations* $\triangleleft$
 - 3: For each output wire w with mask r_w generated by Π_{SH}^* and for each party P_i the dealer samples a commitment randomness $s_{w,i}$ computes for each party P_i a commitment $c_{w,i} = \text{Com}(\llbracket r_w \rrbracket_i; s_{w,i})$
 - 4: The dealer hands $c_{w,i}$ to all the parties and $s_{w,i}$ to party P_i .
 - 5: The parties verify the authenticated secret sharing of the output wires' masks by having each party P_i compute the commitment $c_{w,i}$ and sends it to the other parties who verify it is equal to the commitment received from the dealer, otherwise they abort.
 - 6: The dealer and parties $P_1, \dots, P_n$ invoke the offline phase of the verification protocol Π_{vrfy}^*
 - 7: $\triangleright$ *During the instances above of Π_{vrfy}^* , the dealer also proves the validity of the FLIOP correlations* $\triangleleft$
 - 8: Denote by $\mathbf{C}_i = \left(C_P^i, \left\{ C_V^{j,i} \right\}_{j \neq i} \right)$ be the FLIOP correlated randomness for party P_i .
 - 9: If the offline phase of Π_{SH}^* or Π_{vrfy}^* outputs **reject**, then abort.
 - 10: Party P_i outputs $(\text{SH}_i, \mathbf{C}_i)$.
-

Π_{on} (Online Phase)

Input:

- $C : R^i \rightarrow R^o$ be an arithmetic circuit over a ring R with i and o being the number of input and output wires, respectively. Let m be the number of multiplication gates in the circuit.
- Each party P_i holds for each input wire w_j in C that is associated with P_i a private input y_j . Let $\vec{y} \in R^i$ be the vector of all parties' inputs.

Output: $C(\vec{y})$

- 1: The parties execute the online (computation) phase of Π_{SH}^* to obtain for each party P_j its shares of the masked wire values $\llbracket \hat{x}_w \rrbracket_j$ for each input wire in the circuit that is either an input to a multiplication gate or an output wire.
 - 2: The parties execute the online phase of Π_{vrfy}^* to verify all parties' honest execution of Π_{SH}^* openings of the masked values $\llbracket \hat{x}_w \rrbracket_i$. In this protocol party P_j 's input is the circuit C , the shares of openings of all wires' masked values $\llbracket \hat{x}_w \rrbracket_i$ for all $i \in [n]$ as well as P_j 's correlated randomness SH_j expanded into multiplication triples and its FLIOP correlations $\mathbf{C}_j$.
 - 3: **if** Π_{vrfy}^* outputs **reject** **then**
 - 4: $\perp$ **return abort.**
 - 5: The parties execute the Output Opening phase of Π_{SH}^* to obtain the output of $C(\vec{y})$.
-

However, this can happen with only negligible probability based on the soundness of Protocol 9 and the underlying FLIOP, as desired.

CASE 2: ONLINE PARTY IS MALICIOUS. In this case, $\mathcal{S}$ plays the role of the dealer and the honest online parties. As a dealer, it first sends the correlated randomness and proof of correctness to $\mathcal{A}$. Note this contains mainly PCG seeds and random shares, and so can be simulated by $\mathcal{S}$ by running, and thereby receiving the output of an honest dealer, and distributing its outputs to the malicious parties accordingly. To simulate the verification procedure of the correlated randomness, $\mathcal{S}$ runs $\mathcal{S}_{\text{vrfy}}$ (from the security proof in Section 5.2) to obtain queries' answers. Then, given that it knows the shares of the answers held by $\mathcal{A}$, it can set the shares held by the honest parties such that they will reconstruct to the queries' answers (same as in the proof of Proposition 21 for the case where the prover is honest). If $\mathcal{A}$ sent incorrect shares, causing the verification to fail, $\mathcal{S}$ sends the command **abort** to the trusted party computing $\mathcal{F}$.

Otherwise, the simulation proceeds to the online execution. Until reaching the verification protocol, the simulator $\mathcal{S}$ works as follows. To simulate the input sharing step, it sends random messages on behalf of the honest parties to $\mathcal{A}$ for sharing their inputs. When receiving the masked inputs from $\mathcal{A}$, $\mathcal{S}$ uses the fact that it knows the mask (given by him when playing the dealer) to unmask and extract the inputs. During the circuit's computation, the interaction is for opening the masked inputs to multiplication gates and the masked circuit's output. Again, for the messages sent by the honest parties on each wire w , $\mathcal{S}$ chooses a random message $\hat{x}_{w,i}$ for each honest party P_i and hands it to $\mathcal{A}$. The simulator $\mathcal{S}$ records the message $\hat{x}_{w,k}$ sent from $\mathcal{A}$ for each corrupted party P_k and the masked values $\hat{x}_w$ on each wire and stores it for the verification. Upon reaching the verification step, $\mathcal{S}$ runs the simulator $\mathcal{S}_{\text{vrfy}}$ from Proposition 21 on $\{\hat{x}_{w,k}, \hat{x}_w, r_{w,k}\}_{w \in W}, \{(r_1^{g_\ell} \cdot r_2^{g_\ell})_k\}_{g_\ell \in \text{mult}}$, where $k \in T$ (T is the set of corrupted parties). If the output of $\mathcal{S}_{\text{vrfy}}$ is **reject**, then it sends **abort** to the trusted party computing $\mathcal{F}$ and halts. Otherwise, it proceeds to the output revealing step.

To simulate the final step, $\mathcal{S}$ receives the corrupted parties' output from the trusted party. Then, given a masked $\hat{x}_w$ on wire w intended to a corrupted party, $\mathcal{S}$ computes $r_w = x_w - \hat{x}_w$, where x_w is the output given by the trusted party computing $\mathcal{F}$ and $\hat{x}_w$ is the simulated masked value. Then, given $r_{w,k}$ held by each party corrupted party P_k which were dealt in the offline phase, $\mathcal{S}$ chooses a random share $r_{w,j}$ for each honest party P_j under the constraint that $r_w = \sum_{i=1}^n r_{w,i}$ and hands all $r_{w,j}$ values to $\mathcal{A}$, so that the output of the corrupted party will be x_w . For the commitment over $r_{w,j}$, $\mathcal{S}$ hands $\mathcal{A}$ some randomness, and then plays the role of $\mathcal{F}_{\text{RO}}$ handing $\mathcal{A}$ a string that matches the commitment of $r_{w,j}$ held by $\mathcal{A}$. For an output wire w with output intended to an honest party P_j , if $\mathcal{A}$ causes the opening of the mask to fail (by handing an incorrect share), then $\mathcal{S}$ sends **abort_j** to the trusted party computing $\mathcal{F}$. Otherwise, it sends **continue**, allowing the honest party to learn its output.

Observe that the simulation of the offline phase is statistically close to a real execution by the zero-knowledge property of the zk-FLIOP and since the simulation of the correlated randomness sent by the dealer is perfect. Note that the simulation for computing the circuit is *computationally* indistinguishable from an honest execution, since in the simulation $\mathcal{A}$ sees values uniformly distributed over the ring, whereas in the real execution these values are produced using PCGs. This follows from the reverse sampleability property of the correlations generated by the PCG, that is, multiplication triples can be reverse-sampled based on the PCG seeds help by the adversary. Following the reverse samplability of the correlations and the full security definition of PCGs (see security definition for PCGs in [4]) the distributions are computationally indistinguishable. By Proposition 21, the distribution of the simulation for the verification step is statistically close to

the real execution of this step. Finally, in the output reconstruction step, in both executions, $\mathcal{A}$ sees random values that yield the reconstruction of the correct output, and hence the simulation is perfect. This concludes the proof. ■

5.5 Achieving Identifiable Abort (IA)

In this section, we show how to augment our protocol from the previous section to security with identifiable abort, meaning that if abort takes place then the honest parties also consistently output an index of a corrupted party. Our protocol assumes a PKI between the parties in the offline protocol and a broadcast channel, which are essential for achieving this notion of security (which in particular implies consensus). We note however that the broadcast channel allows the participating parties to agree on the sent messages, but the messages are not public and should not be seen by external parties. This means that in the offline, the broadcast channel enables the $n + 1$ parties to agree on the sent messages, but in the online the broadcast channel is accessible to the n online parties only (i.e. excluding the dealer). To minimize the broadcast calls and shorten the broadcast messages one can use the known method of “broadcast extension” [67, 68]. Hence, the expensive broadcast channel is implemented only for sending a hash of all messages and in case someone cheated.

Overview. In the offline phase of Π_{off} , the dealer now broadcasts an encryption of the correlated randomness handed to each party P_i under P_i ’s encryption key from the PKI. This is done for (1) the semi-honest correlations dealt in the offline phase of Π_{SH}^* ; (2) the FLIOP correlations dealt in the offline phase of Π_{vrfy} (3) the shares of the proofs of correctness of the semi-honest correlations $\llbracket \pi_{\text{SH}} \rrbracket$; and (4) the shares of the proofs of correctness of the FLIOP correlations $\llbracket \pi_{\text{vrfy}} \rrbracket$. Note that since all of these have logarithmic size in our protocol, we can afford using this idea, which only blows up the communication from the dealer by a factor that depends on the security parameter.

Naively, the parties could have now decrypted their correlated randomness and executed the verification protocol of the non-interactive DZKP for the correctness of the correlated randomness, but taking approach ends up the following problem: The non-interactive DZKP requires the parties to compute their shares of the results of the FLIOP linear queries and the only parties (i.e. the non-interactive DZKP verifiers) might actually cheat when opening their shares of the query to the other verifiers which in turn are left with the problem of being unable to tell whether the dealer (i.e. the non-interactive DZKP prover) cheated by trying to prove a false statement or some verifiers tried to cheat by falsely opening their shares of the queries’ answers in an attempt to incriminate the dealer.

In an attempt to settle the issue presented above we present a variant of the non-interactive DZKP protocol (Protocol 9) that allows to identify the cheater in case of abort. The core of the idea is that now the dealer as part of non-interactive DZKP shares also broadcasts the expected shares of the zk-FLIOP queries’ answers of each party in the clear. Now, each online party P_i can decrypt its correlated randomness both for the semi-honest protocol (for Π_{SH}^* and the online verification protocol Π_{vrfy}). And locally assert that both: (1) the shares of the queries’ answers published by the dealer on behalf of P_i are consistent with the shares of the queries it computed; and (2) that verifying the non-interactive DZKP using the queries’ answers the dealer published is successful. If some party P_i finds that the shares of the queries’ answers published by the dealer do not equal to the shares it computed (i.e. the (1) doesn’t hold), then P_i publishes a complaint. This complaint can be verified by the rest of the parties by having P_i to publish the encryption key of

the broadcast correlated randomness. This allows the remaining parties to verify that the queries the dealer published are indeed incorrect and blame the dealer as the cheater (or otherwise blame the complaining party).

Remark 27. *We note that by publishing the encryption key as part of the complaint the PKI can't be reused. Instead, we can leverage public-key cryptosystems which are committing (such as El-Gamal cryptosystem [69]) in the sense that for a given public key pk , message m , encryption randomness r and ciphertext $c = \text{Enc}_{pk}(m; r)$ a (possibly computationally bounded) adversary can't find another message m^* encryption randomness r^* such that $c = \text{Enc}_{pk}(m^*; r^*)$. In that case upon receiving a complaint the dealer can “decommit” to the encrypted message by publishing the encryption randomness r and the message m parties can verify that the complaint is valid. We stress that, however, this loses the property of a non-interactive dealer. An alternative approach is to leverage zk-PoK for the statement that the encrypted message is indeed the correlated randomness provided by the dealer which achieves the best of both worlds — non interactive dealer and reusable PKI. We do not provide further detail on this approach as it is not essential for the main idea of the protocol.*

The fact that each party holds encryptions of other parties' private data is what enables them to identify the cheater. This requires the PKI to satisfy a notion of *commitment* — that is, that the encryption of a message is binding, meaning that the encryption of a message is unique. We note that since our protocol is non-interactive, and since the dealer's message is succinct (in the size of the online circuit) is what allows us to use such a simple and generic solution. Indeed, we formalize our solution to the offline protocol as a generic protocol to non-interactively distribute correlated randomness.

Cheated identification in the online phase is even simpler. The only modification to our protocol from the previous section is sending all messages over a broadcast channel (again, using broadcast extension to minimize overhead). In our protocol, there are two possible abort events. The first can happen when opening the shares of the dealer's messages in the verification protocol. We show that if such opening fails, then due to the way these values are shared, we can attribute the failure to a specific party. The second abort event is if a proof is rejected. In this case, we show that a proof can be rejected only when the prover is corrupted, and so the parties can identify the prover as the cheater.

As a final note, we remark that the simplicity of achieving IA for our protocol demonstrates the advantage of the zk-FLIOP approach where each party proves it sent the correct messages. This allows us to easily identify cheaters, as a rejected proof in our particular setting implies immediately that the prover is the cheater. In contrast, when using the approach of [8], it is more challenging to identify cheaters since the parties only check the consistency of values on the circuit's wires with respect to values on wires that precede them. In this case, when the verification fails, it is unclear how to attribute the failure to a specific party. Similarly, when using the SPDZ approach, previous works [19] required storing and computing an *information-theoretic homomorphic signature* for each message in the protocol. With our approach, this complexity is avoided.

5.5.1 Distributing Correlated Randomness with IA

In this section, we present a protocol for distributing correlated randomness by a dealer to n “online” parties with identifiable abort where the dealer is non-interactive. This protocol can be used as the offline phase of our main protocol Π_{off} where the correlated randomness being dealt

is the semi-honest multiplication triples (generated in the offline phase of Π_{SH}^*) and the FLIOP correlations (generated in the offline phase of Π_{vrfy}^*). The protocol provides security against a malicious dealer *or* any subset of “online” parties. This means that if the dealer is cheating, then all online parties will detect it and identify the dealer as the cheater. On the other hand, if the dealer is honest, a corrupted online party may try to incriminate the dealer. In this case, we want the honest online parties to identify this online party as a cheater. Our protocol is highly generic; it works for any form of correlated randomness which can be verified via zero-knowledge proofs over distributed data based on zk-FLIOPs (as defined in Definition 6), regardless of the way the correlated randomness is generated. The protocol requires only one broadcast message from the dealer to the parties, and in case cheating takes place one more broadcast message is exchanged between the verifiers.

Our protocol realizes the ideal functionality $\mathcal{F}_{\text{dealer}}^{\text{IA}}$ defined in Functionality 28. The functionality either samples the correlated randomness using an algorithm `CorrSample` or allows the adversary to choose it (if the dealer is corrupted). In the former case, it then allows the adversary to decide whether the honest parties receive their randomness or not. If not, then the adversary must send an index of a corrupted party that will be handed to the honest parties together with the abort command. In the latter case, the adversary samples the correlated randomness which is then verified by the functionality using the `CorrVrfy` algorithm. If the correlated randomness is correct, then it is forwarded to the parties. Otherwise, the parties are instructed to abort, while the dealer is blamed as the cheater.

Functionality 28 (The Ideal Functionality $\mathcal{F}_{\text{dealer}}^{\text{IA}}$).

The functionality works with $n + 1$ parties, one of which is a dealer $\mathcal{D}$ and the remaining n parties are referred to as “receiving parties”. The functionality also works with an ideal world adversary $\mathcal{S}$ controlling $\mathcal{D}$ or a set T of receiving parties with $|T| \leq n - 1$.

The functionality is parameterized by the following: (1) m , which denotes the online circuit’s size. (2) A PPT algorithm `CorrSample` which given m and the number of parties n , outputs a vector of uniformly sampled correlated randomness $\text{CR} = (\text{CR}^1, \dots, \text{CR}^n)$ and a deterministic algorithm `CorrVrfy`, which given a vector CR outputs a bit $b \in \{0, 1\}$. If $b = 0$ then CR is well-formed, otherwise it is not.

- If the dealer $\mathcal{D}$ is honest: the functionality computes $\text{CR} = \text{CorrSample}(m, n)$ and hands it to $\mathcal{D}$. Then, it hands CR^i for each $P_i \in T$ to $\mathcal{S}$. If it receives back `continue` from $\mathcal{S}$, then it hands CR^j to each honest party P_j . Otherwise, it receives back an index $i \in T$ from $\mathcal{S}$, and sends the parties `(abort, i)`.
 - If the dealer $\mathcal{D}$ is controlled by $\mathcal{S}$: Upon receiving a vector of correlated randomness CR from $\mathcal{D}$, the functionality computes `CorrVrfy(CR)` to receive b . If $b = 1$, then it sends `(abort, D)` to the parties. If $b = 0$, then it lets $\mathcal{S}$ decide whether to hand CR^i to party P_i or to send `(abort, D)` to all parties.
-

For our protocol to realize $\mathcal{F}_{\text{dealer}}^{\text{IA}}$, we assume that each verifier has a PKI with the dealer. This allows the dealer to broadcast the encrypted correlated randomness (including correlated

randomness for the semi-honest protocol and the FLIOP correlations) together with shares of the proof of correctness *to all verifiers*. Each verifier hence can decrypt its correlated randomness and keep the encryption of the other parties' randomness. In addition, the dealer also broadcasts the expected shares zk-FLIOP queries' answers to all parties on behalf of each party. Now, each party verifies that both the shares of the queries' answers the dealer published on its behalf are correct and that the verification of the non-interactive DZKP for the correctness of both the correlated randomness passes. If any of the above verifications fail for some party P_i then P_i publishes a complaint by opening its correlated randomness either via publishing its encryption key or by following Remark 27 allowing the remaining parties to tell whether P_i is the cheater or the dealer is the cheater. Our protocol is formally described in Protocol 31. As above, by opening the encryption, parties can prove that the public commitment is not valid. Overall, our result is summarized in the following theorem:

Theorem 29. *Let $\text{CR} = (\text{CR}^1, \dots, \text{CR}^n)$ be a vector of correlated random strings over a ring R intended to n parties, whose correctness can be verified using a degree- d circuit C , and let M be the number of distinct monomials in the representation of C as a polynomial. Then, assuming PKI and the existence of a broadcast channel, there exists a protocol to distribute CR by a dealer D to n parties which realizes $\mathcal{F}_{\text{dealer}}^{\text{IA}}$ with computational security, where the size of the dealer's single message is $O(|\text{CR}| + \log(M) \cdot n \cdot d)$. In particular the dealer is non-interactive in this protocol.*

Now, if we consider our protocol Π from the previous section, it is possible to send the correlated randomness to the parties in a compressed way, such that its size is reduced to $O(\log m)$. In that case, the algorithm `CorrVrfy` verifies the correctness of the semi-honest multiplication triples (i.e. that each shares triple (a, b, c) satisfies $c = a \cdot b$) and also verifies the correctness of the FLIOP correlations as specified in section 5.3.2 and Equation 7 and an honest dealer will sample the correlated randomness uniformly and provide it as input to the distribution protocol $\Pi_{\text{Dist}}^{\text{IA}}$. Hence, overall, the size of the dealer's message is $O(\log(m) \cdot n \cdot d)$, where d depends on the method used for verifying the zk-FLIOP correlations. For the $2+1$ setting, however, $d = 2$ and so we obtain the following:

Corollary 30. *Let m be the number of multiplication gates in the online circuit and let Π be the $2+1$ online protocol from Section 5 to compute the circuit. Then, assuming a PKI and a broadcast channel, there exists a protocol to distribute the correlated randomness required for Π by a dealer to two parties, which realizes $\mathcal{F}_{\text{dealer}}^{\text{IA}}$ in which the dealer is non-interactive with computational security, where the size of the dealer's single message is $O(\text{poly}(\kappa) \log(m))$ and the size of honest parties' messages (given a complaint) is $O(\text{poly}(\kappa))$.*

Proof: [Proof of Theorem 29] We show that Protocol 31 satisfies the theorem statement. We first prove security. Let $\mathcal{S}$ be the ideal world simulator and let $\mathcal{A}$ be the real-world adversary. We distinguish between two cases:

- *Case 1: Dealer is malicious.* In this case, $\mathcal{S}$ simply receives the correlated randomness CR from $\mathcal{A}$ and forwards it to $\mathcal{F}_{\text{dealer}}^{\text{IA}}$. Since $\mathcal{S}$ plays the role of the honest parties as the verifiers, it can perfectly simulate the honest parties by following the protocol instructions. If the honest parties abort in the simulation, then it sends $(\text{abort}, \mathcal{D})$ to the trusted party computing $\mathcal{F}_{\text{dealer}}^{\text{IA}}$. Otherwise, it allows the trusted party to hand the honest parties in the ideal world their randomness. We conclude that in this case the simulation is perfect.

Protocol 31 ($\Pi_{\text{Dist}}^{\text{IA}}$ - A Protocol for Distributing Correlated Randomness with Identifiable Abort).

Parameters:

- $1^\sigma, 1^\kappa$ - statistical and computational security parameters.
 - n - the number of online parties $P_1, \dots, P_n$.
 - **CorrSample** - a PPT algorithm which samples vector of correlated randomness $\vec{\text{CR}} = (\text{CR}^1, \dots, \text{CR}^n)$.
 - **CorrVrfy** : $R^m \rightarrow R$ - an arithmetic circuit over a ring R for checking correctness of the correlated randomness CR
 - $(\text{P}_{\text{FLIOP}}, \text{V}_{\text{FLIOP}})$ - a zk-FLIOP protocol with ρ rounds, ℓ -queries per round and message length $u_1, \dots, u_\rho \in \mathbb{N}$ for the language $\mathcal{L}_{\text{CorrVrfy}} = \{\vec{\text{CR}} \mid \text{CorrVrfy}(\vec{\text{CR}}) = 0\}$
 - Let $(\text{Gen}, \text{Enc}, \text{Dec})$ be a semantically secure public-key encryption scheme which is computationally binding (see Remark 27 for a definition of this property) and **Com** be a commitment scheme, as defined in Section 2.
-

Input:

- Each party P_i holds a pair of secret and public keys $(\text{sk}_i, \text{pk}_i)$, where pk_i is known to all other parties.
 - The dealer $\mathcal{D}$ holds an integer m .
- 1: The dealer samples an n -party correlated randomness $\vec{\text{CR}} = (\text{CR}^1, \dots, \text{CR}^n)$ using **CorrSample**(m, n).
 - 2: The dealer generates an n -party non-interactive DZKP $\vec{\pi}$ for the statement $\vec{\text{CR}} \in \mathcal{L}_{\text{CorrVrfy}}$ using $\Pi_{\text{DZKP}}^{\text{NI}}$ (Protocol 9). Let $\pi_1, \dots, \pi_n$ be the shares of the proof $\vec{\pi}$.
 - 3: The dealer computes as part of generating the proof $\vec{\pi}$ the expected shares of the zk-FLIOP queries' answers $\vec{a}_i$ for each party P_i as instructed in Step 2.(b). of Protocol 9.
 - 4: The dealer $\mathcal{D}$ broadcasts $\text{Enc}_{\text{pk}_i}(\text{CR}^i, \pi_i)$ and $\vec{a}_i$ (not encrypted) for each $i \in [n]$.
 - 5: Each party P_i decrypts $\text{Enc}_{\text{pk}_i}(\text{CR}^i, \pi_i)$ to receive CR^i and π_i .
 - 6: Each party P_i verifies the correctness of the non-interactive DZKP $\vec{\pi}$ by following Protocol 9 using its share of the proof π_i except for the following change: Let $\vec{a}_i^*$ be the query answers' shares computed by P_i . Instead of sending $\vec{a}_i^*$ as part of the Protocol 9 and receiving other parties' shares of query responses, P_i uses the query answers published by the dealer $\mathcal{D}$ to verify the proof.
 - 7: If the modified Protocol 9 outputs **abort** or if $\vec{a}_i^* \neq \vec{a}_i$, then P_i publishes a complaint by broadcasting its secret key sk_i and outputs **(abort, $\mathcal{D}$)**.
 - 8: $\triangleright$ A reusable PKI can be achieved as explained in Remark 27. $\triangleleft$
 - 9: All other parties receiving the complaint decrypt $\text{Enc}_{\text{pk}_i}(\text{CR}^i, \pi_i)$ to receive CR^i and π_i and verify whether the complaint is valid by executing the protocol on behalf of P_i .
 - 10: If the execution on behalf of P_i outputs **abort**, all other parties P_j output **(abort, $\mathcal{D}$)**. Otherwise, they output **(abort, P_i)**.
-

- *Case 2: Dealer is honest.* In this case, $\mathcal{S}$ receives CR^i for each corrupted P_i from $\mathcal{F}_{\text{dealer}}^{\text{IA}}$ and hands C_i (encryption of (CR^i, π_i)) for each corrupted P_i to $\mathcal{A}$. In addition, $\mathcal{S}$ follows the protocol instructions for computing $\vec{a}_i$ for each corrupted P_i . Then, it invokes the zk-FLIOP simulator $\mathcal{S}_{\text{FLIOP}}$ (see Definition 6) to obtain queries' answers $a_{j,1}, \dots, a_{j,\ell}$ for each round $j \in [\rho]$ where $a_{j,i}$ is the zk-FLIOP i -th query answer response in the j -th round. Given $a_{j,1}, \dots, a_{j,\ell}$ and the corrupted parties shares $\llbracket a_{j,1} \rrbracket_i, \dots, \llbracket a_{j,\ell} \rrbracket_i$, the simulator $\mathcal{S}$ chooses random share $\llbracket a_{j,1} \rrbracket_k, \dots, \llbracket a_{j,\ell} \rrbracket_k$ for each honest party P_k under the constraint that $a_{j,i} = \sum_{k=1}^n \llbracket a_{j,i} \rrbracket_k$ for each $i \in [\ell]$ and $j \in [\rho]$. From the above, $\mathcal{S}$ can compute $\vec{a}_i$ for each honest party P_i using P_i 's share of all queries answer's

across all rounds and hand to the adversary $\mathcal{A}$ as well. Finally, $\mathcal{S}$ sends `continue` to $\mathcal{F}_{\text{dealer}}^{\text{IA}}$.

Next, note since the dealer is honest, $\mathcal{S}$ does not need to send any complaint on behalf of the honest parties. If $\mathcal{A}$ sends a complaint about cheating from party P_k , $\mathcal{S}$ sends `(abort,  $k$ )` to the trusted party computing $\mathcal{F}_{\text{dealer}}^{\text{IA}}$ and halts.

Note that there are three main differences between the simulation and a real execution. First, $\mathcal{S}$ sends encryptions for garbage instead of real values regarding the correlated randomness handed to the honest parties. By the semantic security of the encryption scheme and the hiding property of the commitment scheme, it follows that any computationally bounded adversary cannot distinguish between the two executions. Second, the answers to zk-FLIOP queries were generated using the zk-FLIOP simulator, without knowing the honest parties' inputs. However, by the zero-knowledge property of the zk-FLIOP scheme, the two executions are statistically close. Finally, in the real execution, an event where the honest parties abort while blaming the honest dealer may occur, whereas in the simulation the honest parties always output an index of a corrupted party. However, such a bad event can occur if a cheating party is able to open an encryption or a commitment to a different value than the actual plaintext or committed value. By the binding property of the encryption and commitment schemes, this can happen only with negligible probability in the security parameter κ , that is allowed by the theorem. We conclude that the real execution and simulation are computationally close and so security follows.

We proceed to show that our protocol satisfies the complexity measure. Note that the dealer sends an encryption of the correlated randomness, implying communication of $|\text{CR}|$ (in particular, if hybrid encryption is used as suggested in Remark 32) and $O(|\text{CR}|)$ in any case. In addition, it sends encryption of the proof, commitments to shares of the proof and encryption of the randomness used for the commitments. The size of these depends on the size of the proof and the number of rounds. Finally, it sends the queries' answers. Since the circuit C to verify the correctness of the correlated randomness has degree- d and M monomials in its representation as a polynomial, then there exists a zk-FLIOP where the size of the proof, the number of rounds, the challenge length and the query complexity are all bounded by $O(\log(M) \cdot d)$ (see Theorem 5.8 in [7]). Hence, encrypting each party's share of these will blow up the cost by a factor of n . Overall, the communication complexity from the size of the dealer is $|\text{CR}| + O(\log(M) \cdot d \cdot n)$. This concludes the proof. ■

Remark 32 (Reducing computational and communication costs). *It is possible to reduce the cost of using encryption in Protocol 31 via standard hybrid encryption. Specifically, the prover can send $\text{Enc}_{\text{pk}_i}(k_i)$ to verifier P_i and then use k_i to encrypt the correlated randomness and the proof via cheap rate-1 symmetric encryption (e.g., using AES).*

Remark 33 (Public verifiability). *Note that if the prover signs all its messages, it is possible to publish a public certificate that shows the dealer has cheated. In particular, the verifiers can publish a signed transcript which contains the dealer's signed messages together with the correlated randomness and proof, accompanied with a zk proof that these are the plaintexts encrypted by the dealer. Then, any outsider who reads this transcript can be convinced that the dealer cannot be trusted. This is possible due to the non-interactivity of the dealer in the protocol: if the dealer sends malformed correlated randomness, this message suffices (together with the parties PKI keys of zk-PoKs as in Remark 32) to publicly prove that the dealer is indeed cheating. Without the interactivity a dealer that is asked respond to further inquiries after a complaint is published may not cooperate leaving the parties unable to publicly claim the dealer cheated.*

5.5.2 Online Protocol with IA

We now proceed to augment the online protocol from Section 5 to security with identifiable abort. Our solution is extremely simple: given our protocol from the offline phase, we claim that sending all messages in the online protocol over a broadcast channel suffices to achieve identifiable abort. Let us first recall the three types of messages sent by parties in our protocol:

1. *Semi-honest computation*: each party sends one message to each other party in the computation of each multiplication gate.
2. *Verification phase - prover's message*: each party sends a masked proof to all parties in each round of the zk-FLIOP emulation, where the mask is known by the offline dealer.
3. *Verification phase - verifier's message*: each party sends its share of the offline dealer's masks of the zk-FLIOP queries' answers. These masks are shared via authenticated additive secret sharing as defined in Section 2, i.e., each party holds commitments to other parties' shares. Hence, each party broadcasts its share and the randomness used for the commitment to the other parties. Then, each party can check that all shares it receives are consistent with the commitments it holds.

By requiring a broadcast channel for the semi-honest computation, we prevent corrupted parties from sending different messages to honest parties. This ensures that parties will hold the same statement when entering the verification step. By having each party broadcast the masked proof and the shares of the dealer's message as a verifier, we ensure that all parties will reach the same decision of whether to accept or reject the proof. Now, there are two events of abort that might occur in the verification protocol. First, the parties may abort if the shares sent by some verifier are not consistent with the commitment held by them for these shares. In this event, we instruct the parties to abort while identifying that verifier as the cheater. Note that if this party were honest, it means that the commitment distributed by the dealer was invalid; in this case, that party would have complained already in the offline protocol (and thus the parties would not have reached the online protocol execution). Hence, it must hold that this verifier is indeed corrupt. The second event where abort may occur is when a proof is rejected. In this case, we instruct the parties to abort while identifying the prover as a cheater. We claim that if an online party is honest, the proof will always be accepted. This holds by the property of our verification protocol (Protocol 19): the statement, the proof and the queries' answers are shared via a 2-out-of-2 additive secret sharing between each party and the dealer. Hence, any dishonest verifier cannot change the output of an honest verifier.

To summarize, our online protocol is augmented to IA by running all messages over a broadcast channel and adding the following:

- If any opening of a dealer's message shared as $\langle\langle \cdot \rangle\rangle$ fails due to party P_i sending a share that is inconsistent with the commitment held by the other parties, output (abort, i) .
- If a proof by proving party P_i is rejected, then output (abort, i) .

We remark also that for the protocol to be secure with identifiable abort, one must implement the ideal functionality $\mathcal{F}_{\text{coin}}$ with a protocol that has this security guarantee (alternatively, we can let the dealer choose a random seed from which all coins are derived, and share it to the parties). Since we call $\mathcal{F}_{\text{coin}}$ only logarithmic number of times, any implementation will suffice without incurring

significant overhead. Finally, we note that the simulation for proving the security of this protocol is almost identical to the simulation in the security proof for our main protocol which achieves security with abort (Section 5.4). This is because the only change is the use of a broadcast channel. The only small addition to the simulation is simulating the identification of a cheater by honest parties. Since the simulator knows all the data held by the corrupted parties in the simulation (recall that if an online party is corrupted then the dealer is honest, and so the simulator plays the role of the dealer handing all randomness to the adversary), it knows exactly when cheating takes place and who sent an incorrect message.

5.6 Concrete Cost Analysis

In this subsection we provide a precise concrete cost analysis of our protocol over boolean circuits. We denote by m the extent of secure computation the dealer provides to the parties (i.e. the total number of triples that can be generated from the PCG keys dealt by the dealer and later verified using the FLIOP correlations the dealer provides). We consider only the $2 + 1$ setting due to the high-dimensionality of the $n + 1$ toward realizing PCGs and verifying FLIOP correlations. In terms of computational costs, we account both for PRG operations (as part of the PCG key expansion) and large-field multiplication operations as part of the FLIOP. Communication costs are measured in bits. The exact cost depends on the choice of FLIOP and PCG. We only mention the most significant additive term of the costs.

Choice of FLIOP. We use the FLIOP of [7] also given in Appendix B. For verifying evaluation of a circuit C of size m with statistical security parameter σ , the FLIOP correlation size is $10\sigma \log(2m)$. The computational cost of proving is $7m$ multiplications and the cost of verifying is $3m$ multiplications. The prover and verifier communication cost is $4\sigma \log m$.

Choice of PCG. We use the PCG of [15] based on Expand-Accumulate codes that is also used for benchmarking in Section 6 where more details are provided. The PCG key size in this setting is $37.5\kappa \log(2m/75)$ for computational security parameter κ . Expanding the PCG key requires $5m$ length-doubling PRG operations.

Offline Phase. From the above, we conclude the following costs for the offline phase: The dealer needs to generate the PCG keys and expand them, provide FLIOP correlations, and prove the keys and the FLIOPs are correct. The verifiers expand the PCG keys, verify them and the FLIOP correlations using the proof provided by the dealer, and store the PCG and FLIOP correlations for the online phase. In total, the dealer's computational cost is $5m$ PRGs and $20m$ multiplications and it sends $37.5\kappa \log(2m/75) + 10\sigma \log 2m + 3\sigma \log 4m$ bits per party for the PCG seed, FLIOP correlation and proofs of validity for them as well. The parties, receiving the correlation from the dealer expand the PCGs as well and verify the correlations. The computational cost is $5m$ PRGs and $6 \log 2m$ multiplications and $3\sigma \log 2m$ bits communicated between the parties.

Online Phase. In the online phase the parties expand the PCG keys (can be done ahead of time) and run the semi-honest and the verification protocols. Overall the computation cost is $2.5m$ PRG calls and $10m$ multiplications. Then each party sends $2m$ bits (as part of the semi honest

protocol). The verification protocol’s communication is sublinear in m and is dominated by the communication from the semi-honest online computation protocol.

5.7 (2+1)-PC VS. 3PC

In the $2 + 1$ setting, the corruption model is the same as standard 3PC: three parties of which one may be corrupted. A major advantage of the $2 + 1$ setting, which is critical to our work, is that the third party is only active during a one-time offline phase. In the following we briefly compare the efficiency of our $2 + 1$ protocol with state-of-the-art 3PC protocols. We emphasize again that following Section 2.1 our protocol is inherently limited to identifiable abort while some of the works listed below ([70, 49]) achieve full security. We consider only 3PC protocols where the online communication is ≤ 1 sent bits per party per AND gate. We divide these protocols into two categories:

Offline FLIOPs. In this category, which is represented by the SWIFT protocol [49], the parties use the offline phase to generate triples that are shared in a robust way, using replicated secret sharing. The parties use zk-FLIOPs to verify these triples. Relying on the robustness of the secret sharing, online malicious security is obtained via a simple check at the end of the protocol. This is what allows this work to achieve high performance. However, the 3 parties need to store robust shares of a multiplication triple for each gate. This makes the resulting correlation size *linear* in the circuit size, conflicting with our “preprocessing for life” goal. To give a more concrete comparison, on our benchmark machine from Section 6 it takes roughly 1ns to multiply an extension field element, amortized. Assuming a 100Mb/s link it takes 10ns to transmit a single bit (in terms of bandwidth), the cost per gate element field element in our protocol is therefore 10ns for multiplying field elements and 20ns for communication. In the SWIFT protocol each party sends a single bit per gate with negligible computation so the marginal cost of each gate in the computation in this setting is estimated to 10ns. Overall the end-to-end amortized cost per gate is 30ns in our protocol vs 10ns in the SWIFT protocol. Therefore, we expect in such setting for SWIFT to be $3\times$ faster than our protocol in the online phase. However, this advantage comes at a cost. For a circuit of size 10^7 , SWIFT’s correlation size is 7.5MB while ours is $< 20\text{KB}$, almost four orders of magnitude smaller.

Online FLIOPs. The works of [70, 12] do not require a separate offline phase. Instead, they first run a 3PC protocol with semi-honest security (excluding the final round), and then run three zk-FLIOPs to verify each party’s messages. Our $2+1$ protocol only requires two online zk-FLIOPs, at the cost of adding an interactive offline phase. A different variant of these 3PC protocols [7, 70] relies on an alternative semi-honest baseline, where one of the parties distributes multiplication triples to the other two parties, who carry out the online computation. The third party uses zk-FLIOP to prove the correctness of the triples in an offline phase, but also needs to rejoin the computation for the final step of verifying the correctness of the online messages. In contrast, in our $2+1$ protocol, the dealer speaks once and disappears.

		Correlation Density (Gates/MB)	Dealer (MGates/s)	Party (MGates/s)	End-to-End LAN (MGates/s)			End-to-End WAN (MGates/s)		
Cores			1	1	1	4.1	8	0.4	1	2
(2+1)-PC	FLIOP	$4.97 \cdot 10^8$	1.3	2.9	1.3	5.31	10.4	0.51	1.3	2.6
	SPDZ	$4.15 \cdot 10^4$	12.8	36	5.31	5.31	5.31	0.51	0.51	0.51
Trusted Dealer	FLIOP	$4.97 \cdot 10^8$	1.77	-	1.77	7.26	14.2	0.7	1.77	3.54
	SPDZ	$4.15 \cdot 10^4$	14.5	-	5.31	5.31	5.31	0.51	0.51	0.51

Table 2: **Offline Costs** in the (2+1)-PC (untrusted dealer) and the trusted dealer settings. *Correlation density* reports the extent of secure computation supported by one MB of storage. *Dealer* and *Party* columns report *local* computation throughput. *End-to-End LAN/WAN* measure the end-to-end protocol throughput with varying number of cores. Additional cores benefit FLIOP, which is compute bound, but not SPDZ since it is network bound. A fractional number of cores is a theoretic extrapolation of the number of cores for which FLIOP and SPDZ performance is equal.

6 Implementation and Benchmarks

In this section we evaluate the performance of our FLIOP-based protocol and compare it to the SPDZ protocol with a dealer. The evaluation demonstrates: (1) The FLIOP-based protocol with a trusted dealer incurs little latency overhead in the online phase compared to SPDZ; (2) In typical LAN/WAN settings, generating and distributing the correlations can be done concretely efficiently; (3) In turn, the storage and offline communication requirements are dramatically lower than SPDZ, by almost four orders of magnitude on our benchmarks; (4) Proving and verifying that the dealer’s correlations are well-formed can be done at little concrete cost; (5) Our protocol incurs lower communication than SPDZ in the offline phase leading to better end-to-end time than SPDZ depending on the number of cores used in parallel by each party. In the WAN setting, the improved performance is evident even if each party runs on a single core, while in the LAN setting the crossover point is five cores per party.

The code is available in a dedicated GitHub repository⁷. In Appendix C we describe additional implementation optimizations we employed.

6.1 Setting and Implementation Details

We implemented our protocol for the setting of $2 + 1$ MPC over *Boolean circuits*. The $2 + 1$ case is arguably the most natural instance of MPC for $n + 1$ parties, and using Boolean circuits has the advantage that multiplication triples for the semi-honest protocol can be compressed using highly efficient PCGs [4, 15, 16].

Circuit of Choice We evaluate our implementation by securely computing the AES-128 [71] encryption scheme, where parties hold additive shares of the key and the message. This use case serves as a standard benchmark for MPC protocols [72, 73, 74, 75, 76, 77]. The circuit consists of 28,176 XOR, 6,400 AND and 2,087 NOT gates, connected by 36,919 wires. The computation performs batches of $B = 1,024$ encryptions at a time. This is done so that a single batch would consume $\sim 10^7$ OTs, an amount commonly used to benchmark OT extension protocols [78, 65, 4, 66, 64, 79].

⁷<https://github.com/MatanHamilis/PreprocessingForLife>

Development Framework Our implementation is written in nightly Rust [80] version 1.72. The experiments were conducted using a single core of a laptop equipped with an 11th Gen Intel Core i7-11800H CPU with hyperthreading disabled and 32GB of DDR4 RAM @ 25.6GB/s.

Networking Environment We simulated two networking environments, a LAN and a WAN, using the `tc qdisc` command by adding a `qdisc` over the localhost interface through which parties communicate. This mechanism restricts the total network traffic of both parties to 1Gb/s with 0.1 millisecond unidirectional latency in the LAN setting and 100Mb/s with 80 millisecond unidirectional latency in the WAN setting.

Pseudorandom Correlation Generator We implement the PCG for OT correlations from [15] based on Expand-Accumulate (EA) codes. We use the aggressive set of parameters from [15], generating ~ 5 million binary Beaver triple correlations with EA code of weight 7 and based on the Learning Parity with Noise (LPN) assumption under regular noise with weight $t = 75$. We note that the recent work of [16] showed decreased security margins for these parameters compared to the estimate of [15], although no practical attack was demonstrated. [16] suggested an alternative construction with improved efficiency based on Expand-Convolute (EC) codes and therefore our measurements serve merely as an upper bound and can be improved by leveraging EC codes or the PCG of [42].

Finite Field for zk-FLIOP The computational bottleneck in the zk-FLIOP execution from Section 5.2 is the field multiplications over the field $GF(2^{64})$. In [12], Li et al. proposed using a prime field $GF(p)$ for the Mersenne prime $p = 2^{61} - 1$ leading to more multiplications, but at a cheaper per-multiplication cost. In the implementation of [12], the prime field $GF(p)$ achieved as much as $8\times$ better throughput than $GF(2^{64})$, resulting in $3.6\times$ faster zk-FLIOP protocols. However, our code uses $GF(2^{64})$ since running the field comparison code from [12] (translated to Rust) in our setting demonstrated a slight advantage in the opposite direction. The comparison task was to multiply 16 field elements by the same field element, where all elements lie sequentially in memory, introducing better caching and instruction-level parallelism, and it required 13.5ns for $GF(2^{64})$ against 14.1ns for $GF(p)$. This discrepancy is likely due to a change in architecture and binary field speedups.

6.2 Benchmark Results

6.2.1 Offline Phase

Our goal in the offline phase is to maximize *throughput*. That is, to generate random correlations for the secure evaluation of as many gates as possible per second. In the offline phase, the dealer: (1) Generates the correlated randomness; (2) Generates a proof for its validity; and (3) Sends it to the parties. The other parties verify that the correlation they received is valid. Table 2 summarizes the computation and communication costs for the offline phase, for both the $(2 + 1)$ setting (with untrusted dealer) and when the dealer is trusted.

Recall that our offline phase is circuit independent, which means that it only depends on the circuit *size*. Instead of referring to a specific circuit of predetermined size, we measure *throughput*. Hence, the communication costs, rather than being measured in bandwidth (in MB), are measured in *gates per MB*, which represents the “Correlation Density”, i.e., the *density* of the correlated

randomness. Then, we provide separate throughput measurements for the computational costs of dealer and the parties, to which we refer as “Dealer Throughput” and “Party Throughput”, as measured on a single CPU core.

In the rest of this section we use $\mathcal{P}$ to denote to a protocol (either FLIOP-based or the SPDZ-based protocol) with correlation density C , dealer throughput D , party throughput P operating under networking environment $\mathcal{N} \in \{LAN, WAN\}$ with network bandwidth $B_{\mathcal{N}}$, measured in Megabytes per second (MB/s). We assume links are symmetric and all parties operate under the same network conditions. The end-to-end throughput of the offline phase is determined by the execution bottleneck. This value can be analytically computed as $\min\left(P, D, \frac{C \cdot B_{\mathcal{N}}}{2}\right)$, where the last term $\frac{C \cdot B_{\mathcal{N}}}{2}$ includes a division by two since the dealer needs to send the correlated randomness *twice* — once per online party.

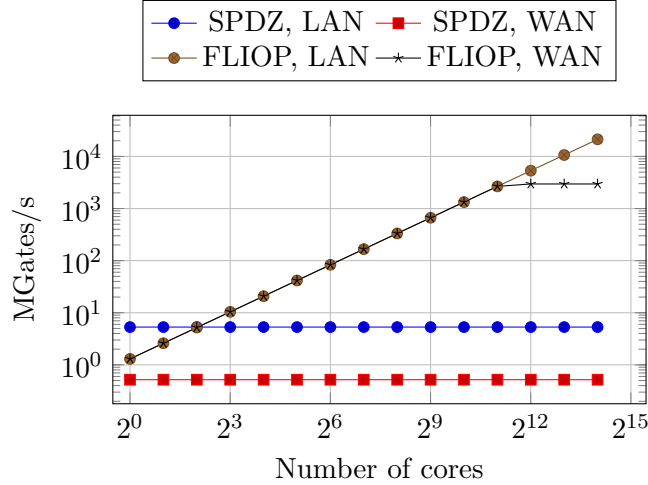


Figure 3: Offline phase gate throughput for 2+1 SPDZ and FLIOP approaches in WAN and LAN. The observed plateaus correspond to network saturation. Intersection points under the same network setting correspond to break-even points.

Link Saturation Clearly, increasing the dealer’s computational power will enable producing a larger amount of correlated randomness. However, the correlations still need to be communicated over a link of limited bandwidth. Therefore, the link bandwidth imposes an upper bound on the end-to-end throughput of the offline phase. Formally, for a positive number L , multiplying the computational power of the dealer and the parties by L , the end-to-end throughput of $\mathcal{P}$ in environment $\mathcal{N}$ is $\min\left(L \cdot P, L \cdot D, \frac{C \cdot B_{\mathcal{N}}}{2}\right)$. The number L can be seen as the number of CPU cores provided to the dealer/parties or a *Parallelization Level*. The *Saturation Point of $\mathcal{P}$ over a network N* is defined to be the parallelization level of the dealer that results in a saturation of the network link, and is equal to $\frac{C \cdot B_{\mathcal{N}}}{2D}$.

Breakeven Point We define the *breakeven point* as the parallelization level which results in an equal offline throughput for both protocols in the (2 + 1) setting. In Table 2, the end-to-end offline throughput measurements are provided for three different parallelization levels: (1) parallelization level of 1; (2) parallelization equals to the breakeven point in the respective networking environment

	Time LAN	Time WAN	Comm	Rounds
SPDZ	310 ms	6.1 s	1.68MB	68
This work	487 ms	7.4 s		84

Table 3: Online $(2 + 1)$ protocol benchmark results. Communication is per party.

	LAN + 1 core	WAN + 1 core	LAN + 8 cores	WAN + 8 cores
SPDZ	1.54 s	18.95 s	1.26 s	13.62 s
This work	5.53 s	12.93 s	0.69 s	1.55 s
Speedup	0.27×	1.46×	1.83×	8.79×

Table 4: End-to-end cost of SPDZ and our protocol in the $(2 + 1)$ setting with varying number of cores and networking environments. The speedup row computes the ratio between the time of SPDZ and our protocol.

for the $(2+1)$ -PC setting (this breakeven point is 4.1 under LAN and 0.4 under WAN settings); and (3) roughly $2\times$ (the value from (2)). From the table it is clear that even in the LAN setting, using 5 cores—commonly available in present computing setups—the FLIOP-based protocol provides better offline throughput. For the WAN setting, even with a single core, the FLIOP-based approach results in a $3\times$ better throughput. Figure 3 provides throughput measurements as a function of the number of cores. The figure was generated by analytically computing the resulting throughput for a given number of cores, network bandwidth and correlation density.

6.2.2 Online Phase

In the $(2+1)$ setting, we measure for the online phase the computation time and bytes communicated per party as well as the number of rounds. Results are in Table 3. All computation times we give relate to a computation conducted on a single core. We compare the protocols in the $(2 + 1)$ setting. We consider both LAN and WAN settings where in LAN the total time is dominated by the computation overhead of each protocol. In LAN our protocol is $1.57\times$ slower. The communication bandwidth is dominated by the semi-honest protocol. In WAN the total time is dominated by the number of rounds and is higher in the FLIOP-based protocol since the verification is interactive.

End-to-End Cost Analysis We conclude this section by combining both the offline and online benchmarks and providing an end-to-end cost analysis for both the SPDZ and our FLIOP-based protocols. We expect SPDZ to be faster when computation power is scarce and network is cheap, while our FLIOP-based protocol to be faster when computation power is abundant and network is expensive. We therefore provide four datapoints in Table 4. While in the LAN setting with a single core SPDZ incurs almost 4 times lower end-to-end latency, in all other settings benchmarked (LAN with 8 cores and WAN with 1 and 8 cores) our protocol incurs lower end-to-end latency by a factor of 1.5 to 8.8.

Acknowledgments Research done in part while the third and fourth authors were visiting NTT Research. E. Boyle was supported by AFOSR Award FA9550-21-1-0046 and ERC Project HSS (852952). M. Hamilis by ERC Project HSS (852952). Y. Ishai by ERC NTSC (742754), BSF

2022370, ISF 2774/20, and ISF-NSFC 3127/23 grants and N. Gilboa by ISF 2951/20 and MOST 8109282 grants.

References

- [1] A. C. Yao, “How to generate and exchange secrets (extended abstract),” in *FOCS*, 1986.
- [2] O. Goldreich, S. Micali, and A. Wigderson, “How to play any mental game or A completeness theorem for protocols with honest majority,” in *STOC*, 1987.
- [3] “MPC deployments dashboard,” accessed: June 6, 2024. [Online]. Available: <https://mpc.cs.berkeley.edu/>
- [4] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, and P. Scholl, “Efficient pseudorandom correlation generators: Silent OT extension and more,” in *CRYPTO*, 2019.
- [5] —, “Efficient pseudorandom correlation generators from Ring-LPN,” in *CRYPTO*, 2020.
- [6] Z. Li, C. Xing, Y. Yao, and C. Yuan, “Efficient pseudorandom correlation generators for any finite field,” *IACR Cryptol. ePrint Arch.*, p. 169, 2025, to appear in Eurocrypt 2025. [Online]. Available: <https://eprint.iacr.org/2025/169>
- [7] D. Boneh, E. Boyle, H. Corrigan-Gibbs, N. Gilboa, and Y. Ishai, “Zero-knowledge proofs on secret-shared data via fully linear PCPs,” in *CRYPTO*, 2019.
- [8] E. Boyle, N. Gilboa, Y. Ishai, and A. Nof, “Sublinear GMW-style compiler for MPC with preprocessing,” in *CRYPTO*, 2021.
- [9] E. Boyle, N. Gilboa, Y. Ishai, and A. Nof, “Secure multiparty computation with sublinear preprocessing,” in *EUROCRYPT*, 2022.
- [10] R. Bendlin, I. Damgård, C. Orlandi, and S. Zakarias, “Semi-homomorphic encryption and multiparty computation,” in *EUROCRYPT*, 2011.
- [11] I. Damgård, V. Pastro, N. P. Smart, and S. Zakarias, “Multiparty computation from somewhat homomorphic encryption,” in *CRYPTO*, 2012.
- [12] Y. Li, Y. Duan, Z. Huang, C. Hong, C. Zhang, and Y. Song, “Efficient 3PC for binary circuits with application to maliciously-secure DNN inference,” in *USENIX Security*, 2023.
- [13] I. Damgård, M. Keller, E. Larraia, V. Pastro, P. Scholl, and N. P. Smart, “Practical covertly secure MPC for dishonest majority - or: Breaking the SPDZ limits,” in *ESORICS*, 2013.
- [14] M. Keller, V. Pastro, and D. Rotaru, “Overdrive: Making SPDZ great again,” in *EUROCRYPT*, 2018.
- [15] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, N. Resch, and P. Scholl, “Correlated pseudorandomness from expand-accumulate codes,” in *CRYPTO*, 2022.
- [16] S. Raghuraman, P. Rindal, and T. Tanguy, “Expand-convolute codes for pseudorandom correlation generators from LPN,” in *CRYPTO*, 2023.

- [17] Y. Ishai, R. Ostrovsky, and V. Zikas, “Secure multi-party computation with identifiable abort,” in *CRYPTO*, 2014.
- [18] G. Spini and S. Fehr, “Cheater detection in SPDZ multiparty computation,” in *Information Theoretic Security: 9th ICITS*, 2016.
- [19] C. Baum, E. Orsini, and P. Scholl, “Efficient secure multiparty computation with identifiable abort,” in *Theory of Cryptography*, 2016.
- [20] C. Baum, E. Orsini, P. Scholl, and E. Soria-Vazquez, “Efficient constant-round mpc with identifiable abort and public verifiability,” in *CRYPTO*, 2020.
- [21] G. Kaissis, A. Ziller, J. Passerat-Palmbach, T. Ryffel, D. Usynin, A. Trask, I. Lima, J. Mancuso, F. Jungmann, M.-M. Steinborn, A. Saleh, M. Makowski, D. Rueckert, and R. Braren, “End-to-end privacy preserving deep learning on multi-institutional medical imaging,” *Nature Machine Intelligence*, 2021.
- [22] G. A. Kaissis, M. R. Makowski, D. Rückert, and R. F. Braren, “Secure, privacy-preserving and federated machine learning in medical imaging,” *Nature Machine Intelligence*, 2020.
- [23] P. Muth and S. Katzenbeisser, “Assisted MPC,” IACR Cryptology ePrint Archive, 2022.
- [24] W. Dong and C. Wang, “Poster: Towards lightweight tee-assisted mpc,” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023.
- [25] D. Demmler, T. Schneider, and M. Zohner, “{Ad-Hoc} secure {Two-Party} computation on mobile devices using hardware tokens,” in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014.
- [26] Y. Wu, X. Wang, W. Susilo, G. Yang, Z. L. Jiang, Q. Chen, and P. Xu, “Efficient server-aided secure two-party computation in heterogeneous mobile cloud computing,” *IEEE TDSC*, 2020.
- [27] M. Blanton and F. Bayatbabolghani, “Improving the security and efficiency of private genomic computation using server aid,” *IEEE S&P*, 2017.
- [28] N. Volgushev, M. Schwarzkopf, B. Getchell, M. Varia, A. Lapets, and A. Bestavros, “Conclave: secure multi-party computation on big data,” in *Proceedings of the Fourteenth EuroSys Conference*, 2019.
- [29] M. S. Riazi, C. Weinert, O. Tkachenko, E. M. Songhori, T. Schneider, and F. Koushanfar, “Chameleon: A hybrid secure computation framework for machine learning applications,” in *ASIACCS*, 2018.
- [30] T. Ryffel, P. Tholomiat, D. Pointcheval, and F. R. Bach, “Ariann: Low-interaction privacy-preserving deep learning via function secret sharing,” *Proc. Priv. Enhancing Technol.*, 2022.
- [31] B. Knott, S. Venkataraman, A. Y. Hannun, S. Sengupta, M. Ibrahim, and L. van der Maaten, “Crypten: Secure multi-party computation meets machine learning,” *CoRR*, 2021.
- [32] P. Huang, T. Hoang, Y. Li, E. Shi, and G. E. Suh, “Efficient privacy-preserving machine learning with lightweight trusted hardware,” in *IEEE S&P*, 2023.

- [33] B. Karmakar, N. Koti, A. Patra, S. Patranabis, P. Paul, and D. Ravi, “Asterisk: Super-fast mpc with a friend,” in *IEEE S&P*, 2024.
- [34] R. Bendlin and I. Damgård, “Threshold decryption and zero-knowledge proofs for lattice-based cryptosystems,” in *TCC*, 2010.
- [35] I. Cascudo and J. S. Gundersen, “A secret-sharing based MPC protocol for boolean circuits with good amortized complexity,” in *TCC*, 2020.
- [36] J. B. Nielsen, P. S. Nordholt, C. Orlandi, and S. S. Burra, “A new approach to practical active-secure two-party computation,” in *CRYPTO*, 2012.
- [37] T. K. Frederiksen, M. Keller, E. Orsini, and P. Scholl, “A unified approach to MPC with preprocessing using OT,” in *ASIACRYPT*, 2015.
- [38] S. S. Burra, E. Larraia, J. B. Nielsen, P. S. Nordholt, C. Orlandi, E. Orsini, P. Scholl, and N. P. Smart, “High-performance multi-party computation for binary circuits based on oblivious transfer,” *J. Cryptol.*, 2021.
- [39] M. Keller, E. Orsini, and P. Scholl, “MASCOT: faster malicious arithmetic secure computation with oblivious transfer,” in *ACM CCS*, 2016.
- [40] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, and P. Scholl, “Efficient pseudorandom correlation generators from ring-LPN,” in *Advances in Cryptology – CRYPTO*, 2020.
- [41] M. Bombar, G. Couteau, A. Couvreur, and C. Ducros, “Correlated pseudorandomness from the hardness of quasi-abelian decoding,” in *Advances in Cryptology - CRYPTO*, 2023.
- [42] M. Bombar, D. Bui, G. Couteau, A. Couvreur, C. Ducros, and S. Servan-Schreiber, “FOLEAGE: $\mathbb{F}_4$ OLE-based multi-party computation for boolean circuits,” Cryptology ePrint Archive, Paper 2024/429.
- [43] D. Abram and P. Scholl, “Low-communication multiparty triple generation for spdz from Ring-LPN,” in *PKC*, 2022.
- [44] N. Asokan, V. Shoup, and M. Waidner, “Optimistic fair exchange of digital signatures,” in *EUROCRYPT*, 1998.
- [45] H. Chaudhari, A. Choudhury, A. Patra, and A. Suresh, “ASTRA: high throughput 3PC over rings with application to secure prediction,” in *ACM CCSW@CCS*, 2019.
- [46] S. Wagh, D. Gupta, and N. Chandran, “SecureNN: 3-party secure computation for neural network training,” *PoPETs*, 2019.
- [47] A. Patra and A. Suresh, “BLAZE: blazing fast privacy-preserving machine learning,” in *NDSS*, 2020.
- [48] A. P. K. Dalskov, D. Escudero, and M. Keller, “Secure evaluation of quantized neural networks,” *Proc. Priv. Enhancing Technol.*, 2020.
- [49] N. Koti, M. Pancholi, A. Patra, and A. Suresh, “SWIFT: super-fast and robust privacy-preserving machine learning,” in *USENIX*, 2021.

- [50] P. Mohassel and P. Rindal, “ABY³: A mixed protocol framework for machine learning,” in *ACM CCS*, 2018.
- [51] S. Wagh, S. Tople, F. Benhamouda, E. Kushilevitz, P. Mittal, and T. Rabin, “Falcon: Honest-majority maliciously secure framework for private deep learning,” *Proc. Priv. Enhancing Technol.*, 2021.
- [52] D. Genkin, Y. Ishai, M. Prabhakaran, A. Sahai, and E. Tromer, “Circuits resilient to additive attacks with applications to secure computation,” in *STOC*, 2014.
- [53] S. Garg, A. Goel, A. Jain, G.-V. Policharla, and S. Sekar, “zkSaaS: Zero-Knowledge SNARKs as a service,” in *USENIX*, 2023.
- [54] D. Dolev and H. R. Strong, “Authenticated algorithms for byzantine agreement,” *SIAM J. Comput.*, vol. 12, no. 4, pp. 656–666, 1983.
- [55] R. Canetti, “Security and composition of multiparty cryptographic protocols,” *J. Cryptology*, 2000.
- [56] O. Goldreich, *The Foundations of Cryptography - Volume 2, Basic Applications*. Cambridge University Press, 2004.
- [57] R. Cleve, “Limits on the security of coin flips when half the processors are faulty (extended abstract),” in *STOC*, 1986.
- [58] Y. Ishai, R. Ostrovsky, and H. Seyalioglu, “Identifying cheaters without an honest majority,” in *TCC*, 2012.
- [59] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *CRYPTO*, 1986.
- [60] D. Beaver, “Efficient multiparty protocols using circuit randomization,” in *CRYPTO*, 1991.
- [61] A. Ben-Efraim, M. Nielsen, and E. Omri, “Turbospeedz: Double your online SPDZ! improving SPDZ using function dependent preprocessing,” in *ACNS*, 2019.
- [62] E. Boyle, N. Gilboa, and Y. Ishai, “Secure computation with preprocessing via function secret sharing,” in *TCC*, 2019.
- [63] I. Damgård and S. Zakarias, “Constant-overhead secure computation of boolean circuits using preprocessing,” in *TCC*, 2013.
- [64] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, P. Rindal, and P. Scholl, “Efficient two-round OT extension and silent non-interactive secure computation,” in *ACM CCS*, 2019.
- [65] K. Yang, C. Weng, X. Lan, J. Zhang, and X. Wang, “Ferret: Fast extension for correlated OT with small communication,” in *ACM CCS*, 2020.
- [66] G. Couteau, P. Rindal, and S. Raghuraman, “Silver: Silent VOLE and oblivious transfer from hardness of decoding structured LDPC codes,” in *CRYPTO*, 2021.

- [67] A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song, “The honey badger of BFT protocols,” in *ACM SIGSAC*, 2016.
- [68] K. Nayak, L. Ren, E. Shi, N. H. Vaidya, and Z. Xiang, “Improved extension protocols for byzantine broadcast and agreement,” in *DISC*, 2020.
- [69] T. ElGamal, “A public key cryptosystem and a signature scheme based on discrete logarithms,” *IEEE transactions on information theory*, 1985.
- [70] E. Boyle, N. Gilboa, Y. Ishai, and A. Nof, “Practical fully secure three-party computation via sublinear distributed zero-knowledge proofs,” in *ACM CCS*, 2019.
- [71] M. Dworkin, E. Barker, J. Nechvatal, J. Foti, L. Bassham, E. Roback, and J. Dray, “Advanced encryption standard (AES),” 2001.
- [72] M. Keller, E. Orsini, D. Rotaru, P. Scholl, E. Soria-Vazquez, and S. Vivek, “Faster secure multi-party computation of AES and DES using lookup tables,” in *ACNS*, 2017.
- [73] T. Araki, J. Furukawa, Y. Lindell, A. Nof, and K. Ohara, “High-throughput semi-honest secure three-party computation with an honest majority,” in *ACM SIGSAC*, 2016.
- [74] M. Albrecht, L. Grassi, C. Rechberger, A. Roy, and T. Tiessen, “Mimc: Efficient encryption and cryptographic hashing with minimal multiplicative complexity,” in *ASIACRYPT*, 2016.
- [75] I. Damgård, M. Keller, E. Larraia, C. Miles, and N. P. Smart, “Implementing aes via an actively/covertly secure dishonest-majority mpc protocol,” in *SCN*, 2012.
- [76] C. Hazay, P. Scholl, and E. Soria-Vazquez, “Low cost constant round MPC combining BMR and oblivious transfer,” *Journal of cryptology*, 2020.
- [77] A. Brüggemann, R. Hundt, T. Schneider, A. Suresh, and H. Yalame, “Flute: Fast and secure lookup table evaluations,” in *IEEE S&P*, 2023.
- [78] L. Roy, “SoftspokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model,” in *CRYPTO*, 2022.
- [79] M. Keller, E. Orsini, and P. Scholl, “Actively secure ot extension with optimal overhead,” in *CRYPTO*, 2015.
- [80] “Rust Programming Language,” <https://www.rust-lang.org/>.
- [81] J. Kilian, “Founding cryptography on oblivious transfer,” in *STOC*, 1988.
- [82] Y. Ishai, M. Prabhakaran, and A. Sahai, “Founding cryptography on oblivious transfer - efficiently,” in *CRYPTO*, 2008.

A Impossibility of Full Security with Correlated Randomness

Theorem 34. *There exists a two-party functionality $f(x, y)$ that cannot be realized with full (computational) security, even when the parties can access an arbitrary source of correlated randomness.*

Proof: (sketch) The first observation is that Cleve’s impossibility of fair coin-tossing [57] extends to the OT-hybrid model, namely where the two parties have access to an ideal oblivious transfer oracle. (Intuitively, this is because an OT delivers output to just one of the parties, and thus is not different from a standard communication channel.) This implies the impossibility of a fully secure protocol for the 2-party XOR functionality, where both parties receive the output, in the OT-hybrid model. The key observation is that if such a fully secure XOR protocol Π existed given correlated randomness, one could obtain a similar protocol Π' in the OT-hybrid model as follows. First, the parties use a secure-with-abort protocol π for generating the correlated randomness, which exists unconditionally in the OT-hybrid model [81, 82]. If π aborts, the honest party uses a default value for the other party’s input (this does not violate security, since the event of aborting is independent of the honest party’s input). Finally, Π' invokes Π using the correlated randomness generated by π . ■

B Concrete Instantiations for the zk-FLIOP

Consider a prover P who wants to prove to a verifier V that an input $(c, \vec{a}, \vec{b})$, where $\vec{a}$ and $\vec{b}$ are vectors of size m , is in the following language:

$$\mathcal{L}_{prod} = \{(c, \vec{a}, \vec{b}) \in R^{1+2m} \mid c - \langle \vec{a}, \vec{b} \rangle = 0\}.$$

To prove the correctness of the statement, the vectors $\vec{a}$ and $\vec{b}$ are divided into M vectors, i.e., $\vec{a} = \vec{a}_1 \parallel \dots \parallel \vec{a}_M$ and $\vec{b} = \vec{b}_1 \parallel \dots \parallel \vec{b}_M$. This means that the statement to be proven can be written as $c - \sum_{k=1}^M \vec{a}_k \cdot \vec{b}_k = 0$. Denote the number of elements in each vector by L , and so $L \cdot M = m$.

The first step of P is to choose random vectors $\vec{a}_0$ and $\vec{b}_0$. Then, define L polynomials of degree- M such that:

$$\forall e \in [L] : f_e(0) = \vec{a}_0[e], \dots, f_e(M) = \vec{a}_M[e].$$

That is, the evaluation of f_e on the point k is the e th entry of the vector $\vec{a}_k$. Similarly, define additional L polynomials of degree- M by setting:

$$\forall e \in [L] : g_e(0) = \vec{b}_0[e], \dots, g_e(M) = \vec{b}_M[e].$$

Next, define an additional polynomial q by letting

$$q(x) = \sum_{e=1}^L f_e(x) \cdot g_e(x).$$

By the way q is defined it follows that:

- For each $k \in \{0, \dots, M\}$ it holds that $q(k) = \vec{a}_k \cdot \vec{b}_k$
- The degree of q is $2M$.

To fully define q the prover P thus computes M more additional points on q . This can be done by first interpolating the f_e and g_e polynomials to compute these polynomials on the points $M + 1, \dots, 2M$ and then computing $q(x)$ for each $x \in \{M + 1, \dots, 2M\}$.

The protocol then proceeds as follows:

1. **Proof:** The prover P defines the proof by setting

$$\pi = (\{f_e(0)\}_{e=1}^L, \{g_e(0)\}_{e=1}^L, q(0), \dots, q(2M))$$

2. **Challenge:** A random point τ is chosen (such that $\tau \notin \{0, \dots, M\}$).
3. **Query:** The linear queries over the proof and inputs are defined such that the verifier V is given:
 - For each $e \in [L] : f_e(\tau)$ and $g_e(\tau)$.
 - $q(\tau)$
 - $c - \sum_{k=1}^M q(k)$

4. **Decision:** The verifier V checks that

$$\begin{aligned} \text{(a)} \quad & q(\tau) = \sum_{e=1}^L f_e(\tau) \cdot g_e(\tau) \\ \text{(b)} \quad & c - \sum_{k=1}^M q(k) = 0 \end{aligned}$$

If both equations hold, then V outputs **accept**. Otherwise, it outputs **reject**.

Observe that in check 4a, the verifier ensures that P defined the proof correctly, i.e., computed the polynomial q honestly. Then, it can verify that the statement holds via check 4b. Note also that privacy (zero-knowledge) is maintained in this protocol by the additional random point defined for each polynomial (i.e., $f_e(0)$ and $g_e(0)$). These random points make the evaluation of each polynomial on the point τ look completely random.

Soundness A malicious prover can succeed only if check 4a passes although $q(x)$ is not defined correctly. By the Schwartz-Zippel lemma, this event can happen when working over a finite field with probability bounded by $\frac{2M}{|\mathbb{F}| - M - 1}$. When the statement is defined over the ring $\mathbb{Z}_{2^k}$, then the proof is executed over the extension ring $\mathbb{Z}_{2^k}[x]/f(x)$. In this case, if f is of degree d , then the cheating probability is bounded by $\frac{2^{(k-1)d} \cdot (2M)}{2^{kd}} \approx \frac{2M}{2^d}$.

Concrete Costs Recall that L and M are parameters to choose under the constraint that $m = L \cdot M$. If we set $L = M = \sqrt{m}$, the cost becomes sublinear in m . Concretely, the size of the proof is $2L + 2M + 1 = 4\sqrt{m} + 1$. In addition, the query complexity is $2L + 2 = 2\sqrt{m} + 2$.

Reducing size of the proof via recursion Now let $M = 2$ and $L = m/2$. This means that we have now $\frac{m}{2}$ polynomials, each of degree-2. If use the above constant-round protocol, we obtain a proof of linear size. However, via recursion, we can reduce the size to being logarithmic. The observation is that in check 4a, instead of having the verifier check that $q(\tau) = \sum_{e=1}^L f_e(\tau) \cdot g_e(\tau)$, we can let the prover prove that this equation holds. Note that this statement has the same structure as the initial statement (i.e., a scalar minus the inner product between two vectors), albeit with

input of half size (the size of each vector is now $\frac{m}{2}$). Hence, the parties can repeat the process until the size of the statement is a small constant, and then let the verifier perform the final check as above. We note in this solution, the prover needs to add a random point to each polynomial only in the last round. This implies that the degree of the proof polynomial q in each round is 2, and so the proof consists of 3 elements. In the last round, the proof consists of 7 elements, since random points are added to the f and g polynomials, which makes the degree of q be 4. Overall, this construction yields the following parameters:

- Proof size: $(\log m - 1) \cdot 3 + 7$
- Query complexity: $\log m + 3$
- Soundness error is approximately $\frac{\log m}{|\mathbb{F}|}$ for finite fields and $\frac{\log m}{2^d}$ for the ring $\mathbb{Z}_{2^k}$.

C Implementation Optimizations

In this appendix we discuss several optimizations we introduced in our implementation.

Reducing Verifier Cost Recall in protocol 19 the parties verify the correctness of a shared proof for a shared statement. The protocol is based on the zk-FLIOP scheme introduced in section 2.3.1, where in each round of the protocol the parties view their share of the statement as evaluations of L polynomials of degree M each and interpolate them on a randomly chosen point. Following step 2 of the protocol, we make the following observation: Whenever the zk-FLIOP verifier algorithm is invoked, the shared statement that each verifier holds (either $\vec{y}_i^D$ or $\vec{y}_i^V$) contains *zeros* in all odd or all even indices of the shared statement. This leaves exactly $L/2$ of the L polynomials to be identically zero, and thereby eliminates the computational overhead of interpolating half of the polynomials, reducing the total cost of the verifier by 50%. We observed the same level of speedup in our implementation as well. We emphasize that this optimization applies to all settings where the FLIOP verifier is invoked, namely: 1) During the online phase when the parties verify the honest behavior of other parties; 2) During the offline phase when the dealer is acting as a verifier while generating the zk-FLIOP correlations; and 3) During the offline phase when parties verify the correct generation of the (possibly authenticated) multiplication triples.

zk-FLIOP Folding Factors From a theoretical perspective, the zk-FLIOP protocol from section 2.3.1 has the lowest computational overhead when setting $M = 2$. However, when it comes to *memory accesses*, each round of the zk-FLIOP requires accessing the elements in the statement sequentially, and therefore the total memory access cost for a length- N statement is $\sum_{i=0}^{(\log_M N)-2} \frac{N}{M^i}$, maximized for $M = 2$. Therefore, as observed by Li et al. [12], by choosing a larger value for M , we can achieve optimal balance between memory access bandwidth and computational cost. We ran experiments for varying values of M and found the value of $M = 8$ to result in 30% concrete performance improvement. In Figure 4 we provide measurements for proving and verifying zk-FLIOPs with for range of folding factors. From the figure it follows that in our setting, a folding factor of 8 for the LAN setting is optimal.

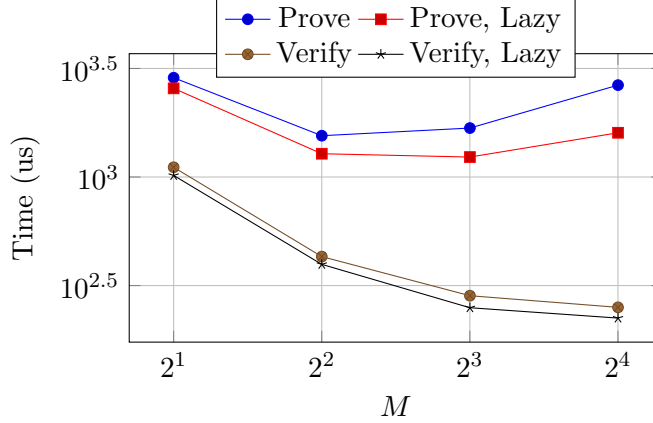


Figure 4: Performance over LAN of proving and verifying a zk-FLIOP statement of length 100,000 for varying values of the folding factor M with and without lazy modular reduction.

Lazy Modular Reduction The computation of a binary field multiplication is made of two sequential parts: (1) Multiplying two binary polynomials; (2) Reducing the resulting polynomial by an irreducible binary polynomial. Let $\vec{a}, \vec{b}$ be two vectors in $\mathbb{F}_{2^k}$ of length N , when computing the inner product $\langle \vec{a}, \vec{b} \rangle$ we can multiply each pair of elements $\vec{a}_i, \vec{b}_i$ and sum the results. This would require executing both the binary polynomial multiplication and the polynomial modular reduction steps of the finite field multiplication N times. We can reduce the cost of this computation to N binary polynomial multiplications and only a single reduction by first multiplying the N pairs of polynomials, summing the results and finally computing the modular reduction by the irreducible polynomial only once on the resulting polynomial. We benchmark the effect of this optimization on varying folding factors in the LAN setting, both for the computation of the zk-FLIOP prover and verifier and give the results in Figure 4.

Efficient Computation of Powers The construction of the input statement to protocol 7 requires computing a large number of powers of a random value α . That is, for a specific value α it is required to compute $\alpha^0, \alpha^1, \dots, \alpha^N$ for some large value N . A naive approach for generating this sequence of values is to repeatedly take the last generated value α^i , multiply it by α and output α^{i+1} and that is for any $i \in [0, N-1]$. Each core in a modern CPU is equipped with many independent “arithmetic logic units” (ALUs) that can carry computations concurrently. The naive approach does not allow the CPU core to use all its ALUs by instruction level parallelism as it introduces many dependencies between instructions. Concretely, each value α^{i+1} depends directly on the computation of α^i . We have taken a different approach by computing the powers in bulks of a small constant size K . This is done by initially filling an array of size K with powers of $\alpha^0, \dots, \alpha^{K-1}$, then computing $\alpha' = \alpha^K$ and iteratively outputting K consecutive powers at a time by multiplying each element in the array by α' . This allows us to carry K multiplications in parallel and thereby avoid stalls during the computation of powers. We benchmarked both approaches by evaluating 10 million consecutive powers of a random 64-bit field element. The naive approach took 61ms while the optimized approach, using $K = 16$, took 21ms, a $3\times$ speedup.